

Министерство просвещения Республики Башкортостан
Государственное автономное профессиональное образовательное учреждение
Уфимский колледж статистики, информатики и вычислительной техники

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____ 3.3. Курмашева
«___» _____ 2026 г.

«РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ УПРАВЛЕНИЯ АВИАПАРКОМ
И ЗАЯВКАМИ НА ВЫЛЕТ»

Пояснительная записка к дипломному проекту

Рецензент
_____ В.В. Будилов
«___» _____ 2026 г.

Руководитель
_____ 3.Р. Шагибалова
«___» _____ 2026 г.

Выпускник гр. 22П-1
_____ Д.А. Сальников
«___» _____ 2026 г.

АННОТАЦИЯ

Пояснительная записка к дипломному проекту содержит постановку и программу решения задачи «Разработка веб-приложения для управления авиапарком и заявками на вылет».

Веб-приложение написано на языке программирования TypeScript в среде программирования Microsoft Visual Studio Code. Серверная часть разработана на языке C# в среде программирования Microsoft Visual Studio 2022 с применением ASP.NET Core Web API и Entity Framework Core с использованием системы управления базами данных MySQL. Предназначено для работы в операционной системе MS Windows 10 и отлажено на данных контрольного примера.

					40.C2454-2026 09.02.07 ДП-ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка веб-приложения для управления авиапарком и заявками на вылет	Лит.	Лист	Листов
Разраб.		Сальников Д.А.					2	138
Провер.		Шагибалова З.Р.						
Реценз.		Будилов В.В.						
Н. Контр.		Каримова Р.Ф.						
Утверд.		Курмашева З.З.				УКСИВТ 22П-1		

СОДЕРЖАНИЕ

	ЛИСТ
Введение	4
1 Постановка задачи	7
1.1 Описание предметной области	7
1.2 Проектирование бизнес-процессов предметной области	14
1.3 Описание входной информации	15
1.4 Описание выходной информации	16
1.5 Общие требования к программному продукту	16
1.6 Описание структуры базы данных	18
1.7 Контрольный пример	27
2 Экспериментальный раздел	28
2.1 Описание программы	28
2.2 Протокол тестирования программного продукта	40
2.3 Руководство пользователя	47
Заключение	66
Приложение А Диаграмма прецедентов	68
Приложение Б Шаблон договора	69
Приложение В ER-диаграмма	74
Приложение Г Контрольный пример	75
Приложение Д Модульная схема приложения	84
Приложение Е Код программы	85
Список использованных источников	135

ВВЕДЕНИЕ

Авиаперевозки занимают значительное место в современной транспортной системе, обеспечивая мобильность людей и грузов. Основная особенность авиаперевозок заключается в оперативности. Они позволяют удобно и быстро перемещаться, что укрепляет связь между людьми по всему миру, а также открывают большие туристические возможности. Кроме того, авиаперевозки позволяют быстро доставлять различные товары, что способствует развитию торговли и бизнеса.

По данным Росавиации, в 2024 году количество пассажиров, обслуженных российскими авиакомпаниями, составило 111,7 млн человек (рост на 5,9% по сравнению с 2023 годом), а также было перевезено 489,4 тыс. тонн грузов (рост на 4,7% по сравнению с 2023 годом). Рост количества перевозок говорит о важной роли авиации в экономике. Поэтому активно внедряются цифровые технологии, которые помогают отслеживать и управлять полётами в реальном времени.

Актуальность темы обусловлена отсутствием на рынке информационных систем учёта и управления авиапарка, а также предложения услуг клиентам с доступностью для малого и среднего бизнеса не только для авиаперевозчиков, но и для аэротакси. Кроме того, данная система позволит частным лицам удобно искать и сравнивать услуги. Это не только минимизирует человеческий фактор, но и повышает удобство покупки авиабилетов и услуг.

На данный момент существуют современные решения, ориентирующиеся на учёт, управление авиапарка и качество обслуживания пассажиров, однако практически все подобные решения платные. Вот некоторые из них: «Portside Horizon», «AirplaneManager», «PAMC Software – Aircraft Charter Module», «ХО»

Portside Horizon – это облачная платформа для полного цикла управления чартерными и управляемыми флотовыми операциями: цифровое планирование,

					40.С2454-2026 09.02.07 ДП-ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

учёт вылетов, отслеживание финансовых показателей и отчётность в режиме реального времени

AirplaneManager – это web-сервис для расписания чартерных рейсов и взаимодействия пилотов, диспетчеров и клиентов: календарь вылетов, мобильное приложение для ввода данных “в полёте”, триггер уведомлений и отчётов, модуль запросов рейсов.

PAMC Software – Aircraft Charter Module – это интегрируемый модуль для учёта чартерных рейсов в рамках ERP: планирование, управление пассажирами, распределение затрат, ведение КП и сквозная связка с учётом поставщиков и выставлением счетов

ХО (ранее JetSmarter) – это онлайн-платформа и маркетплейс для поиска и бронирования чартерных рейсов, предлагает мгновенные котировки, управление заявками и программами членства, динамическое ценообразование, оформление брони, выставление счетов и сквозную отчётность.

Разработка такой информационной системы позволяет объединить в одном программном продукте основные процессы, связанные с организацией чартерных авиаперевозок: ведение данных об авиакомпаниях и воздушных судах, обработку заявок клиентов, расчёт параметров перелёта, контроль статусов вылетов и формирование сопутствующей документации. Это особенно важно для небольших авиаперевозчиков и сервисов аэротакси, которым требуется не перегруженное и доступное решение для повседневной работы, а также для клиентов, заинтересованных в быстром и понятном подборе услуг.

В этих условиях разработка информационной системы для организации чартерных авиаперевозок представляет собой актуальную и практически значимую задачу. Она требует рассмотрения вопросов анализа предметной области и проектирования структуры базы данных. Такой подход позволяет рассматривать создаваемый программный продукт как инструмент комплексной поддержки основных процессов, связанных с предоставлением чартерных авиауслуг.

При подготовке проекта были использованы теоретические источники по вопросам цифровой трансформации авиационной отрасли, развития сервисов обслуживания пассажирских авиаперевозок, проектирования баз данных и программных интерфейсов. В работе учтена документация по ASP.NET Core Web API, Entity Framework Core, MySQL, OpenAPI, React и Vite. Законодательно-нормативную основу работы составили ГОСТ 19.701-90, ГОСТ Р 2.105-2019, ГОСТ Р 7.0.108-2022 и ГОСТ Р 59793-2021.

Целью проекта является разработка веб-приложения, предназначенного для упрощения организации чартерных авиаперевозок, управления авиапарком и обработки заявок клиентов. Коммерческая ценность проекта заключается в возможности получения дохода за счёт автоматизации платных чартерных услуг: расчёта стоимости рейсов, обработки заявок, сопровождения вылетов и предоставления авиакомпаниям удобного инструмента для взаимодействия с клиентами.

Для достижения поставленной цели были определены и решены следующие задачи:

- изучить предметную область организации чартерных авиаперевозок;
- проанализировать существующие решения и определить основные требования к разрабатываемой системе;
- спроектировать структуру базы данных для хранения сведений об авиакомпаниях, самолётах, аэропортах, пользователях, пассажирах, заявках и вылетах;
- разработать серверную часть приложения с использованием ASP.NET Core Web API и Entity Framework Core;
- реализовать функции авторизации, просмотра каталога самолётов, расчёта параметров рейса, создания заявок и управления вылетами;
- провести тестирование основных функций программного продукта;
- подготовить пользовательскую и техническую документацию к разработанному программному продукту.

1 Постановка задачи

1.1 Описание предметной области

Требуется разработать информационную систему, для автоматизации учёта, чартерных авиаперевозок. Включая учёт имеющихся у авиакомпаний самолётов (авиапарк), создание и редактирование заявок на чартерный рейс.

Данная информационная система предполагает наличие лиц, ответственных за управление авиапарком. Ответственное лицо имеет возможность:

- управлять авиапарком (добавление, редактирование и удаление самолётов, доступных для заказа);
- одобрять или отклонять заказы рейсов;
- сопровождать вылеты;
- редактировать статус вылета.

Также данная информационная система предполагает наличие клиентов, имеющих возможность:

- просматривать частичную информацию о самолётах;
- просматривать примерные данные планируемого рейса;
- создавать заявки на чартерные рейсы;
- отменять созданные заявки;
- добавлять пассажиров на рейс;
- просматривать информацию о заказанных рейсах;
- отслеживать статус вылета.

Первое, что выбирает заказчик, это компанию, организующую вылет. У каждой компании есть представитель, отвечающий за управление авиапарком, работу с заявками и вылетами. Для авиакомпаний будет создан следующий справочник:

- уникальный номер авиакомпании;
- название авиакомпании;

- дата регистрации;
- полное наименование организации;
- сокращённое наименование организации;
- юридический адрес;
- почтовый адрес;
- номер телефона;
- электронная почта;
- базовая стоимость обслуживания;
- базовая стоимость одной пересадки;
- название банка;
- ИНН;
- КПП;
- ОГРН;
- расчётный счёт;
- корреспондентский счёт;
- БИК;
- логотип авиакомпании;
- город заключения договора;
- срок действия договора;
- срок оплаты;
- класс питания;
- время прибытия пассажиров до вылета;
- статус отображения авиакомпании в каталоге.

При заказе чартерного рейса заказчик обязан указать аэропорт взлёта и посадки. Для их учёта будет создан справочник аэропортов, хранящий информацию о нём, уникальные коды, которыми он обозначается, используемые во всём мире, а также ширину долготу для расчёта дальности, времени и стоимости:

					40.C2454-2026 09.02.07 ДП-ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

- уникальный номер аэропорта;
- название;
- город;
- страна;
- IATA (трёхбуквенный код, присваиваемый Международной ассоциацией воздушного транспорта);
- ICAO (четырёхбуквенный код, присваиваемый аэропортам международной организацией гражданской авиации);
- географическая широта;
- географическая долгота.

Также, при заказе чартерного рейса, клиент выбирает модель самолёта. Для учёта самолётов будет создан справочник, в котором для каждого самолёта будет храниться информация о нём с его некоторыми характеристиками:

- уникальный номер самолёта;
- номер авиакомпании, которой он принадлежит;
- название модели;
- максимальная дальность полёта;
- количество мест;
- крейсерская скорость;
- стоимость часа полёта;
- фотография самолёта.

Для вылетов будет создан справочник, хранящий всю необходимую информацию, в том числе для формирования билетов:

- уникальный номер вылета;
- номер заказчика чартерного рейса;
- номер самолёта;
- аэропорт вылета;
- аэропорт посадки;

- расстояние полёта;
- время полёта;
- цена;
- количество пересадок;
- запрошенное число и время вылета;
- документ договора;
- имя файла договора;
- тип содержимого файла договора;
- дата и время загрузки договора;
- уникальный номер пользователя, загрузившего договор.

Помимо этого, будет создан справочник, хранящий информацию о пассажирах, пользующихся услугами авиакомпании:

- уникальный номер пассажира;
- фамилия;
- имя;
- отчество;
- серия паспорта;
- номер паспорта;
- почта для уведомлений;
- дата рождения;
- адрес регистрации;
- фактический адрес;
- номер телефона;
- ИНН;
- название банка;
- расчётный счёт;
- корреспондентский счёт;
- БИК.

А также справочник, связывающий пассажиров и вылеты:

- уникальный номер пассажира;
- уникальный номер вылета.

Для отслеживания статусов вылетов будет создан отдельный справочник, хранящий:

- уникальный номер записи;
- уникальный номер вылета;
- уникальный номер статуса;
- дата и время установки статуса.

Соответственно для отслеживания статусов необходим справочник статусов, содержащий следующие поля:

- уникальный номер статуса;
- статус.

Для совершения заказа клиент должен зарегистрировать учётную запись. Также для управления вылетами сотрудники авиакомпании должны быть авторизованными. Для хранения информации о пользователях системы будет создан справочник, содержащий следующие поля:

- уникальный номер пользователя;
- уникальный номер пассажира;
- уникальный номер роли;
- уникальный номер авиакомпании;
- электронная почта;
- пароль;
- код подтверждения почты;
- время истечения кода подтверждения почты;
- статус подтверждения почты;
- статус активности пользователя.

Для разделения возможностей пользователей предусмотрены роли пользователей, хранящиеся в отдельном справочнике, содержащем следующие поля:

- уникальный номер роли;
- название роли.

Для обслуживания каждого вылета требуется целая команда. Все сотрудники, связанные с обслуживанием вылетов, хранятся в справочнике, содержащем следующие поля:

- уникальный номер вылета;
- уникальный номер сотрудника.

Для расчёта маршрута с пересадками будет создан справочник приоритетов аэропортов, содержащий следующие поля:

- уникальный номер аэропорта;
- приоритет аэропорта при построении маршрута;
- признак столицы;
- признак крупного города;
- примечание.

Для хранения подробного маршрута вылета будет создан справочник участков маршрута, содержащий следующие поля:

- уникальный номер участка маршрута;
- уникальный номер вылета;
- порядковый номер участка маршрута;
- уникальный номер аэропорта отправления;
- уникальный номер аэропорта прибытия;
- расстояние участка маршрута;
- время полёта по участку маршрута;
- стоимость полёта по участку маршрута;
- время стоянки после прибытия.

Для хранения токенов обновления авторизации будет создан справочник токенов, содержащий следующие поля:

- уникальный номер токена;
- уникальный номер пользователя;
- хэш токена;
- дата и время создания токена;
- дата и время истечения токена;
- дата и время отзыва токена;
- уникальный номер токена, которым был заменён текущий токен.

Для своевременного информирования участников процесса в системе предусмотрены уведомления. Уведомления позволяют фиксировать значимые события, возникающие при обработке заявок, сопровождении вылетов и управлении сотрудниками авиакомпании. Пользователь может получать уведомления об изменении статуса вылета, маршрута, даты и времени отправления, а также о приглашении в авиакомпанию или изменении должности. Для хранения пользовательских уведомлений будет создан справочник, содержащий следующие поля:

- уникальный номер уведомления;
- уникальный номер пользователя;
- заголовок уведомления;
- текст уведомления;
- тип действия, связанного с уведомлением;
- уникальный номер авиакомпании;
- уникальный номер роли;
- дата и время создания уведомления;
- дата и время прочтения уведомления.

Также предусмотрены уведомления авиакомпании, предназначенные для фиксации внутренних организационных событий: создания рабочего аккаунта

сотрудника, изменения должности, увольнения сотрудника или выхода сотрудника из авиакомпании. Для хранения уведомлений авиакомпании будет создан справочник, содержащий следующие поля:

- уникальный номер уведомления авиакомпании;
- уникальный номер авиакомпании;
- заголовок уведомления;
- текст уведомления;
- дата и время создания уведомления;
- дата и время прочтения уведомления.

Ограничения на информационной системе:

- если возраст клиента меньше 12 лет, совершить перелёт можно только вместе с совершеннолетним сопровождающим неважно, родственник это или нет;
- каждый клиент должен предоставить всю необходимую информацию.

1.2 Проектирование бизнес-процессов предметной области

Для проектирования бизнес-процессов предметной области используется методология RUP (Rational Unified Process), так как она позволяет описать функциональные требования к системе через прецеденты использования и определить доступные действия для каждой группы пользователей.

В рамках проектирования программного продукта AirCharter выделяются следующие акторы:

- неавторизованный пользователь — посетитель веб-приложения, имеющий доступ к просмотру каталога самолётов, выбору маршрута и ознакомлению с доступными вариантами чартерного рейса;
- клиент — зарегистрированный пользователь, который может оформить заявку на чартерный рейс, указать пассажиров, просматривать свои заявки, отменять заявку и получать сопроводительные документы;

- представитель авиакомпании — сотрудник, рассматривающий заявки клиентов, проверяющий данные рейса и пассажиров, принимающий решение об одобрении или отклонении заявки, а также сопровождающий выполнение рейса;
- администратор авиакомпании — пользователь с расширенными правами, выполняющий управление самолётами, сотрудниками, ролями, профилем авиакомпании и аналитикой.

Для каждого актора определены следующие прецеденты использования:

- неавторизованный пользователь: просмотр каталога самолётов, поиск и подбор самолёта по маршруту, регистрация и авторизация;
- клиент: создание заявки на чартерный рейс, выбор аэропортов вылета и прибытия, выбор самолёта, добавление пассажиров, просмотр статуса заявки, отмена заявки, получение билета или иных документов;
- представитель авиакомпании: просмотр поступивших заявок, проверка информации о рейсе, одобрение или отклонение заявки, изменение статуса вылета, назначение сотрудников на рейс;
- администратор авиакомпании: управление парком самолётов, управление сотрудниками и их ролями, настройка профиля авиакомпании, просмотр аналитической информации.

Диаграмма вариантов использования, отражающая перечисленных акторов и доступные им прецеденты, представлена в приложении А.

1.3 Описание входной информации

Входными данными являются данные о физических лицах, пользователях, авиакомпаниях, самолётах и вылетах. Входными документами будут являться данные об аэропортах. В таблице 1.3.1 представлены описания входных документов.

Таблица 1.3.1 – Описание входных документов

Наименование документа (шифр)	Периодичность поступления документа	Откуда поступает документ
1	2	3
Заявка на чартерный рейс	По запросу	От пользователя

Продолжение таблицы 1.3.1

1	2	3
Данные пассажира	По запросу	От пользователя
Данные авиакомпании	По запросу	От представителя
Данные самолёта	По запросу	От сотрудника
Данные о статусе вылета	По запросу	От сотрудника

1.4 Описание выходной информации

Выходным документом является договор оказания услуг. Описание выходных документов представлено в таблице 1.4.1.

Таблица 1.4.1 – Описание выходных документов

Наименование документа (шифр)	Периодичность выдачи документа	Кол-во экз.	Куда передаются
Договор оказания услуг	Каждый раз при запросе	1	Клиенту, совершившему заказ и авиакомпании
Информирующее сообщение о регистрации рабочего аккаунта с данными для входа	При создании рабочего аккаунта	1	Сотруднику авиакомпании
Сообщение с кодом подтверждения почты	При регистрации пользователя или подтверждении адреса электронной почты	1	Пользователю, подтверждающему электронную почту

Шаблоны выходных документов представлены в приложении Б.

1.5 Общие требования к программному продукту

Разрабатываемый программный продукт должен обеспечивать работу нескольких категорий пользователей: клиентов, представителей авиакомпаний и администраторов системы. Каждая категория пользователей должна иметь доступ только к тем функциям, которые соответствуют её роли.

К функциональным требованиям относятся:

- регистрация и авторизация пользователей;
- подтверждение электронной почты пользователя;
- просмотр каталога самолётов;
- поиск аэропортов отправления и прибытия;

Изм.	Лист	№ докум.	Подпись	Дата

40.С2454-2026 09.02.07 ДП-ПЗ

Лист

16

- расчёт расстояния, времени полёта, количества пересадок и стоимости рейса;
- создание заявки на чартерный рейс;
- добавление и редактирование данных пассажиров;
- просмотр пользователем собственных заявок и вылетов;
- управление самолётами авиакомпании;
- одобрение или отклонение заявок представителем авиакомпании;
- назначение сотрудников на вылет;
- изменение статуса вылета;
- формирование и хранение документов, связанных с рейсом.
- формирование и отображение уведомлений для пользователей и авиакомпаний о значимых изменениях в системе

К нефункциональным требованиям относятся удобство пользовательского интерфейса, корректная обработка ошибок, защита персональных данных, разграничение прав доступа, целостность данных в базе данных и возможность работы приложения через современный веб-браузер на персональном компьютере или ноутбуке без установки отдельного клиентского программного обеспечения.

Система должна обеспечивать хранение данных в реляционной базе данных MySQL. Взаимодействие клиентской части с серверной частью должно выполняться через программный интерфейс ASP.NET Core Web API.

Надёжность программного продукта обеспечивается проверкой входных данных, разграничением прав доступа, хранением паролей в защищённом виде, использованием токенов авторизации, контролем целостности данных в базе данных и обработкой ошибок при выполнении запросов.

Эффективность работы системы обеспечивается разделением клиентской и серверной частей, использованием API для обмена данными, хранением

постоянных данных в реляционной базе MySQL и выполнением расчётов параметров рейса на серверной стороне.

1.6 Описание структуры базы данных

База данных (БД) – это организованное электронное хранилище информации, структурированное по определенным правилам для удобного хранения, поиска, обработки и управления данными, часто в виде таблиц, связанных между собой. Для разработки системы используется СУБД MySQL и утилита MySQL Workbench 8.0 CE.

Структура базы данных представлена в таблицах 1.6.1 – 1.6.16.

Таблица 1.6.1 – roles (Роли)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор роли	id	INT(4)	Суррогатный первичный ключ
Наименование роли	name	VARCHAR(45)	Уникальное поле

Таблица 1.6.2 – airlines (Авиакомпании)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Идентификатор авиакомпании	id	INT(4)	Суррогатный первичный ключ
Наименование авиакомпании	airline_name	VARCHAR(45)	Уникальное поле
Дата регистрации	creation_date	DATE(3)	
Юридический адрес	legal_address	VARCHAR(255)	
Почтовый адрес	postal_address	VARCHAR(255)	
Номер телефона	phone_number	VARCHAR(20)	
Электронная почта	email	VARCHAR(255)	
Базовая стоимость обслуживания	service_base_cost	DECIMAL(12,2)	

Продолжение таблицы 1.6.2

1	2	3	4
Базовая стоимость одной пересадки	transfer_base_cost	DECIMAL(12,2)	
Название банка	bank_name	VARCHAR(45)	
ИНН	taxpayer_id	VARCHAR(12)	
КПП	tax_registration_reason_code	VARCHAR(9)	
ОПГН	primary_state_registration_number	VARCHAR(15)	
Расчётный счёт	current_account_number	VARCHAR(20)	
Корреспондентский счёт	correspondent_account_number	VARCHAR(20)	
БИК	bank_identifier_code	VARCHAR(9)	
Логотип авиакомпании	image	LOB	Допускается NULL
Город заключения договора	contract_city	VARCHAR(100)	Допускается NULL
Срок оплаты	payment_deadline_days	INT	Допускается NULL
Класс питания	catering_class	VARCHAR(100)	Допускается NULL
Срок действия договора	contract_validity_days	INT	Допускается NULL
Время прибытия пассажиров до вылета	passenger_arrival_minutes_before_flight	INT	Допускается NULL
Статус отображения авиакомпании в каталоге	is_catalog_visible	BOOLEAN	

Продолжение таблицы 1.6.2

1	2	3	4
Организационно-правовая форма	organization_type	VARCHAR(32)	

Таблица 1.6.3 – persons (Физические лица)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Идентификатор пассажира	id	INT(4)	Суррогатный первичный ключ
Имя	first_name	VARCHAR(45)	
Фамилия	last_name	VARCHAR(45)	
Отчество	patronymic	VARCHAR(45)	Допускается NULL
Серия паспорта	passport_series	CHAR(4)	
Номер паспорта	passport_number	CHAR(6)	
Адрес регистрации	registration_address	VARCHAR(255)	Допускается NULL
Фактический адрес	actual_address	VARCHAR(255)	Допускается NULL
Номер телефона	phone_number	VARCHAR(20)	Допускается NULL
ИНН	taxpayer_id	VARCHAR(12)	Допускается NULL
Название банка	bank_name	VARCHAR(100)	Допускается NULL
Расчетный счет	current_account_number	VARCHAR(20)	Допускается NULL
Корреспондентский счет	correspondent_account_number	VARCHAR(20)	Допускается NULL
БИК	bank_identifier_code	VARCHAR(9)	Допускается NULL

Продолжение таблицы 1.6.3

1	2	3	4
Почта для уведомлений	email	VARCHAR(255)	Допускается NULL
Дата рождения	birth_date	DATE(3)	Допускается NULL

Таблица 1.6.4 – users (Пользователи)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор пользователя	id	INT(4)	Суррогатный первичный ключ
Идентификатор пассажира	person_id	INT(4)	Вторичный ключ (Физические лица), допускается NULL
Идентификатор роли	role_id	INT(4)	Вторичный ключ (Роли)
Идентификатор авиакомпании	airline_id	INT(4)	Вторичный ключ (Авиакомпаний), допускается NULL
Электронная почта	email	VARCHAR(255)	Уникальное поле
Пароль	password_hash	VARCHAR(255)	
Хэш кода подтверждения почты	email_confirmation_code_hash	VARCHAR(255)	Допускается NULL
Время истечения кода подтверждения почты	email_confirmation_code_expires_at_utc	DATETIME(8)	Допускается NULL
Статус подтверждения почты	is_email_confirmed	BOOLEAN(1)	
Статус активности пользователя	is_active	BOOLEAN(1)	

Таблица 1.6.5 – planes (Самолёты)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор самолёта	id	INT(4)	Суррогатный первичный ключ
Идентификатор авиакомпании	airline_id	INT(4)	Вторичный ключ (Авиакомпания)
Название модели	model_name	VARCHAR(45)	
Максимальная дальность полёта	max_distance	INT(4)	
Количество мест	passenger_capacity	INT(4)	
Крейсерская скорость	cruising_speed	INT(4)	
Стоимость часа перелёта	flight_hour_cost	DECIMAL(12,2)	
Фотография самолёта	image	LONGBLOB	Допускается NULL

Таблица 1.6.6 – airports (Аэропорты)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор аэропорта	id	INT(4)	Суррогатный первичный ключ
Название	name	VARCHAR(225)	
Город	city	VARCHAR(45)	Допускается NULL
Страна	country	VARCHAR(45)	
IATA	iata	VARCHAR(3)	Допускается NULL
ICAO	icao	VARCHAR(4)	Допускается NULL
Географическая широта	latitude	DECIMAL(9, 6)	
Географическая долгота	longitude	DECIMAL(9, 6)	

Таблица 1.6.7 – departures (Вылеты)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Идентификатор вылета	id	INT(4)	Суррогатный первичный ключ
Идентификатор заказчика чартерного рейса	charter_requester_id	INT(4)	Вторичный ключ (Пользователи)

Продолжение таблицы 1.6.7

1	2	3	4
Идентификатор самолёта	plane_id	INT(4)	Вторичный ключ (Самолёты)
Аэропорт вылета	take_off_airport_id	INT(4)	Вторичный ключ (Аэропорты)
Аэропорт посадки	landing_airport_id	INT(4)	Вторичный ключ (Аэропорты)
Расстояние полёта	distance	INT(4)	
Время полёта	flight_time	TIME(3)	
Цена	price	DECIMAL(12,2)	
Количество пересадок	transfers	INT(4)	
Запрошенное число и время вылета	requested_take_off_date_time	DATETIME(8)	
Документ договора	contract_document	LOB	Допускается NULL
Имя файла договора	contract_document_file_name	VARCHAR(255)	Допускается NULL
Формат файла договора	contract_document_content_type	VARCHAR(100)	Допускается NULL
Дата и время загрузки договора	contract_document_uploaded_at	DATETIME	Допускается NULL
Пользователь, загрузивший договор	contract_document_uploaded_by_user_id	INT	Допускается NULL

Таблица 1.6.8 – statuses (Статусы)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор статуса	id	INT(4)	Суррогатный первичный ключ
Наименование статуса	status	VARCHAR(45)	

Таблица 1.6.9 – departure_statuses (Статусы вылетов)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор записи	id	INT(4)	Суррогатный первичный ключ
Идентификатор вылета	departure_id	INT(4)	Вторичный ключ (Вылеты)
Идентификатор статуса	status_id	INT(4)	Вторичный ключ (Статусы)
Дата и время установки статуса	status_setting_date_time	DATETIME(8)	

Таблица 1.6.10 – passenger_departure (Пассажиры вылетов)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор вылета	departure_id	INT(4)	Составной первичный ключ, вторичный ключ (Вылеты)
Идентификатор пассажира	person_id	INT(4)	Составной первичный ключ, вторичный ключ (Пассажиры)

Таблица 1.6.11 – departure_employees (Сотрудники вылетов)

Содержание поля	Имя поля	Тип, длина	Примечание
Идентификатор вылета	departure_id	INT(4)	Составной первичный ключ, вторичный ключ (Вылеты)
Идентификатор сотрудника	employee_id	INT(4)	Составной первичный ключ, вторичный ключ (Пользователи)

Таблица 1.6.12 – airport_route_priorities (Приоритеты аэропортов)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Уникальный номер аэропорта	airport_id	INT	Первичный ключ, вторичный ключ (Аэропорты)

Продолжение таблицы 1.6.12

1	2	3	4
Приоритет аэропорта при построении маршрута	priority_score	INT	
Признак столицы	is_capital	BOOLEAN	
Признак крупного города	is_large_city	BOOLEAN	
Примечание	note	VARCHAR(255)	Допускается NULL

Таблица 1.6.13 – departure_route_legs (Участки маршрута вылета)

Содержание поля	Имя поля	Тип, длина	Примечание
Уникальный номер участка маршрута	id	INT	Первичный ключ
Уникальный номер вылета	departure_id	INT	Вторичный ключ (Вылеты)
Порядковый номер участка маршрута	sequence_number	INT	
Аэропорт отправления	from_airport_id	INT	Вторичный ключ (Аэропорты)
Аэропорт прибытия	to_airport_id	INT	Вторичный ключ (Аэропорты)
Расстояние участка маршрута	distance	INT	
Время полета по участку маршрута	flight_time	TIME	
Стоимость полета по участку маршрута	flight_cost	DECIMAL(12,2)	
Время стоянки после прибытия	ground_time_after_arrival	TIME	Допускается NULL

Таблица 1.6.14 – refresh_tokens (Токены обновления)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Уникальный номер токена	id	INT	Первичный ключ
Уникальный номер пользователя	user_id	INT	Вторичный ключ (Пользователи)
Хэш токена	token_hash	VARCHAR(128)	Уникальное поле

Продолжение таблицы 1.6.14

1	2	3	4
Дата и время создания токена	created_at_utc	DATETIME	
Дата и время истечения токена	expires_at_utc	DATETIME	
Дата и время отзыва токена	revoked_at_utc	DATETIME	Допускается NULL
Токен, которым был заменен текущий токен	replaced_by_token_id	INT	Вторичный ключ (Токены обновления), допускается NULL

Таблица 1.6.15 – notifications (Уведомления пользователей)

Содержание поля	Имя поля	Тип, длина	Примечание
Уникальный номер уведомления	id	INT	
Уникальный номер пользователя	user_id	INT	
Заголовок уведомления	title	VARCHAR(255)	
Текст уведомления	message	VARCHAR(1000)	
Тип действия	action_type	VARCHAR(100)	Допускается NULL
Уникальный номер авиакомпании	airline_id	INT	Допускается NULL
Уникальный номер роли	role_id	INT	Допускается NULL
Дата и время создания уведомления	created_at_utc	DATETIME	
Дата и время прочтения уведомления	read_at_utc	DATETIME	Допускается NULL

Таблица 1.6.16 – airline_notifications (Уведомления авиакомпании)

Содержание поля	Имя поля	Тип, длина	Примечание
1	2	3	4
Уникальный номер уведомления авиакомпании	id	INT	

Продолжение таблицы 1.6.16

1	2	3	4
Уникальный номер авиакомпании	airline_id	INT	
Заголовок уведомления	title	VARCHAR(255)	
Текст уведомления	message	VARCHAR(1000)	
Дата и время создания уведомления	created_at_utc	DATETIME	
Дата и время прочтения уведомления	read_at_utc	DATETIME	Допускается NULL

ERD – диаграмма БД представлена в приложении В на рисунке В.1.

1.7 Контрольный пример

Контрольный пример предназначен для проверки корректности работы разработанного программного продукта на заранее подготовленном наборе данных. В качестве контрольного примера используются записи справочников и основных таблиц базы данных: роли пользователей, физические лица, пользователи, авиакомпании, самолёты, аэропорты, заявки, вылеты, пассажиры, сотрудники вылетов, пользовательские уведомления и уведомления авиакомпании.

Данные контрольного примера приведены в приложении Г. Они позволяют проверить корректность связей между таблицами, работу пользовательских сценариев и соответствие результатов работы программы ожидаемым значениям.

2 Экспериментальный раздел

2.1 Описание программы

Для автоматизации учёта чартерных авиаперевозок был разработан программный продукт, обеспечивающий хранение, обработку и отображение информации об авиакомпаниях, пользователях, самолётах, аэропортах, вылетах, пассажирах и статусах вылетов. Система предназначена для поддержки основных бизнес-процессов предметной области: ведения авиапарка, расчёта параметров перелёта, оформления заявок на чартерный рейс, сопровождения вылетов и контроля их статусов. Требуемые функции соответствуют описанной предметной области: клиент должен иметь возможность просматривать самолёты и создавать заявки на рейс, а представитель авиакомпании — управлять авиапарком, заявками и вылетами.

Для решения поставленной задачи выбрана клиент-серверная архитектура. Такое решение подходит лучше настольного приложения, так как система предполагает работу нескольких категорий пользователей: клиентов, оформляющих заявки на рейс, и представителей авиакомпаний, управляющих авиапарком, заявками и вылетами. Веб-приложение позволяет обеспечить доступ к системе через браузер без установки отдельного клиентского программного обеспечения на каждое устройство.

Клиентская часть отвечает за отображение интерфейса, ввод данных и отправку запросов на сервер. Серверная часть обрабатывает запросы, выполняет бизнес-логику, проверяет корректность данных, рассчитывает параметры перелёта и взаимодействует с базой данных. Такое разделение упрощает сопровождение программы, делает структуру решения более понятной и позволяет независимо развивать клиентскую и серверную части.

В качестве системы управления базой данных выбрана MySQL. Этот выбор обоснован тем, что предметная область содержит набор взаимосвязанных сущностей, естественно представимых в виде реляционной модели. В системе

используются таблицы авиакомпаний, ролей, персон, пользователей, самолётов, аэропортов, вылетов, статусов, истории статусов, пассажиров вылетов и сотрудников вылетов. Такая структура позволяет обеспечить целостность данных и удобную обработку информации.

Для разработки программного продукта использовались языки C#, TypeScript и SQL. Исходный код представлен в приложении Е.

Язык C# использовался для реализации серверной части приложения. Его выбор обусловлен поддержкой объектно-ориентированного подхода, строгой типизацией, удобством построения многоуровневых приложений и наличием развитых средств для создания веб-API.

Язык TypeScript использовался для реализации клиентской части приложения на основе библиотеки React. Выбор TypeScript обусловлен строгой типизацией клиентского кода, а выбор React — возможностью создавать интерфейс в виде независимых переиспользуемых компонентов.

Язык SQL использовался для проектирования базы данных, создания таблиц, определения ограничений целостности, а также для заполнения и изменения данных. Структура базы данных реализована в MySQL и включает таблицы airlines, roles, persons, users, planes, airports, departures, statuses, departure_statuses, passenger_departure, departure_employees, airport_route_priorities, departure_route_legs, refresh_tokens.

В качестве средств разработки использовались Visual Studio 2022 и Visual Studio Code. Visual Studio 2022 применялась для разработки и отладки серверной части на C#. Visual Studio Code использовалась для разработки клиентской части на React и TypeScript, а также для редактирования конфигурационных и вспомогательных файлов проекта.

Разработанный программный продукт состоит из клиентской части, серверной части и базы данных. Клиентская часть реализует пользовательский интерфейс и обеспечивает взаимодействие пользователя с системой. Через неё пользователь может просматривать авиакомпании и самолёты, выбирать

маршрут, получать примерную информацию о перелёте, создавать заявку и отслеживать вылеты. Серверная часть реализует обработку запросов, выполнение основной логики приложения, авторизацию, работу с ролями пользователей, расчёт параметров перелёта, управление сущностями системы и доступ к базе данных. База данных хранит всю постоянную информацию системы.

В программном продукте реализована подсистема уведомлений. Она предназначена для информирования пользователей о важных изменениях, связанных с их заявками и вылетами. Пользователь получает уведомления при изменении статуса вылета, маршрута, даты и времени отправления, а также при получении приглашения в авиакомпанию. В уведомлении отображаются заголовок, текст сообщения, дата создания и признак прочтения. Пользователь может отметить отдельное уведомление как прочитанное или отметить прочитанными все уведомления.

Для представителей авиакомпании предусмотрен отдельный журнал уведомлений авиакомпании. В нём фиксируются события, связанные с управлением сотрудниками: создание рабочего аккаунта, изменение должности, увольнение сотрудника или самостоятельный выход сотрудника из авиакомпании. Это позволяет сотрудникам авиакомпании отслеживать важные организационные изменения без необходимости вручную проверять разные разделы системы.

После выбора технологий была определена общая структура приложения. Для этого была разработана модульная схема, которая показывает, из каких функциональных частей состоит система и как они связаны между собой. Такой подход позволяет лучше организовать внутреннюю архитектуру программы, упростить её разработку и сделать дальнейшее сопровождение более удобным.

Модульная схема представляет собой разделение системы на отдельные функциональные части, каждая из которых выполняет свою роль. Такое построение позволяет выделить независимые компоненты, определить способы

их взаимодействия и скрыть внутреннюю реализацию каждого блока. Благодаря этому система становится более понятной, гибкой и удобной для расширения.

Модульная схема веб-приложения представлена в приложении Д на рисунке Д.1.

Описание модулей представлено в таблице 2.1.1.

Таблица 2.1.1 – Описание модулей

Модуль	Описание модуля
1	2
main.tsx	Точка входа React-приложения. Подключает главный компонент приложения и выполняет рендеринг интерфейса в браузере.
App.tsx	Главный компонент приложения. Настраивает маршруты страниц и общую структуру клиентской части.
LoginPage.tsx	Страница авторизации пользователя. Отвечает за ввод почты и пароля, отправку запроса на сервер и обработку результата входа.
RegisterPage.tsx	Страница регистрации пользователя. Позволяет создать учетную запись и передать регистрационные данные на сервер.
ConfirmEmailPage.tsx	Страница подтверждения электронной почты. Используется для ввода кода подтверждения и активации учетной записи.
CatalogPage.tsx	Страница каталога. Отображает доступные авиакомпании и самолеты для просмотра клиентом.
OrderPage.tsx	Страница оформления заявки на чартерный рейс. Позволяет выбрать маршрут, самолет, дату вылета и рассчитать параметры рейса.
CabinetPage.tsx	Страница личного кабинета пользователя. Отображает информацию о заказанных вылетах.
CabinetDeparturePage.tsx	Страница подробной информации о вылете. Используется для просмотра пассажиров, маршрута, статуса и документов по рейсу.
ProfilePage.tsx	Страница профиля пользователя. Позволяет просматривать и редактировать персональные данные.
AirlineProfilePage.tsx	Страница профиля авиакомпании. Используется для просмотра и редактирования данных авиакомпании.

Продолжение таблицы 2.1.1

1	2
RegisterAirlinePage.tsx	Страница регистрации авиакомпании. Позволяет внести данные организации и создать профиль авиакомпании в системе.
ManagementPage.tsx	Главная страница управления для сотрудников авиакомпании. Отображает основные разделы работы с авиапарком и вылетами.
ManagementPlanesPage.tsx	Страница управления самолетами. Позволяет просматривать, добавлять, редактировать и удалять самолеты авиакомпании.
ManagementPlaneFormPage.tsx	Форма добавления и редактирования самолета. Используется для ввода характеристик воздушного судна.
ManagementOrderRoutePage.tsx	Страница управления маршрутом заявки. Позволяет сотруднику просматривать и корректировать маршрут вылета.
Header.tsx	Компонент шапки сайта. Содержит навигацию, элементы авторизации и переключение основных разделов.
AirportSearch.tsx	Компонент поиска аэропортов. Используется при выборе аэропорта вылета и посадки.
CatalogPlaneCard.tsx	Компонент карточки самолета в каталоге. Отображает краткую информацию о модели, вместимости и стоимости.
UserDepartureCard.tsx	Компонент карточки вылета пользователя. Показывает основные данные заявки и текущий статус рейса.
RouteModal.tsx	Компонент модального окна маршрута. Отображает подробную информацию о маршруте и пересадках.
ProtectedRoute.tsx	Компонент защиты маршрутов. Ограничивает доступ к страницам для неавторизованных пользователей.
StaffRoute.tsx	Компонент защиты маршрутов сотрудников. Ограничивает доступ к управленческим страницам по роли пользователя.
apiClient.ts	Модуль настройки HTTP-клиента для взаимодействия с серверной частью.
authService.ts	Сервис работы с авторизацией. Выполняет запросы входа, регистрации, подтверждения почты и обновления токена.
orderService.ts	Сервис работы с заявками на вылет. Отправляет данные заказа и получает расчет параметров рейса.

Продолжение таблицы 2.1.1

1	2
managementService.ts	Сервис управленческих функций. Используется для работы с вылетами, пассажирами, статусами и сотрудниками.
planesService.ts	Сервис работы с самолетами. Получает данные каталога и выполняет операции управления авиапарком.
airlineService.ts	Сервис работы с авиакомпаниями. Используется для получения и изменения данных авиакомпании.
airportsService.ts	Сервис работы с аэропортами. Выполняет поиск аэропортов и передачу данных маршрута.
personService.ts	Сервис работы с персональными данными пользователя.
userService.ts	Сервис работы с текущим пользователем, его ролью и данными учетной записи.
Program.cs	Точка входа серверного приложения. Настраивает сервисы, подключение к базе данных, авторизацию, CORS и маршрутизацию API.
AirCharterExtendedContext.cs	Контекст базы данных Entity Framework Core. Описывает таблицы БД, связи между ними и параметры сопоставления моделей.
AuthenticationController.cs	Контроллер авторизации. Обрабатывает регистрацию, вход, подтверждение почты, повторную отправку кода и обновление токенов.
UsersController.cs	Контроллер пользователей. Предоставляет данные о текущем пользователе, его роли и связанных персональных данных.
PersonsController.cs	Контроллер физических лиц. Используется для просмотра и обновления персональной информации пользователя.
AirlinesController.cs	Контроллер авиакомпаний. Обрабатывает регистрацию авиакомпании, изменение профиля, логотипа и договорных настроек.
PlanesController.cs	Контроллер самолетов. Реализует получение, добавление, редактирование и удаление самолетов авиакомпании.
AirportsController.cs	Контроллер аэропортов. Предоставляет поиск аэропортов для выбора маршрута рейса.
FlightsController.cs	Контроллер расчетов рейса. Используется для получения примерных параметров перелета и доступных самолетов.

Продолжение таблицы 2.1.1

1	2
DeparturesController.cs	Контроллер вылетов. Обрабатывает создание заявок, управление пассажирами, сотрудниками, маршрутом, статусами и документами вылета.
JwtService.cs	Сервис работы с JWT-токенами. Создает токены доступа и получает данные пользователя из токена.
EmailService.cs	Сервис отправки электронных писем. Используется для отправки кода подтверждения почты.
RoutePlanningService.cs	Сервис построения маршрута. Рассчитывает маршрут между аэропортами с учетом дальности самолета и возможных пересадок.
FlightLegCalculationService.cs	Сервис расчета участков маршрута. Определяет расстояние, время и стоимость отдельных частей перелета.
AirportSearchService.cs	Сервис поиска аэропортов. Выполняет фильтрацию аэропортов по названию, городу и кодам IATA/ICAO.
GeoDistanceCalculator.cs	Сервис географического расчета расстояния между аэропортами по координатам.
AirportGraphCache.cs	Сервис кэширования графа аэропортов. Ускоряет построение маршрутов между аэропортами.
AirportGraph.cs	Модуль графа аэропортов. Хранит связи между аэропортами, используемые при поиске маршрута.
AirportSpatialIndex.cs	Модуль пространственного индекса аэропортов. Помогает быстрее находить ближайшие аэропорты при построении маршрута.
ImageAspectRatioValidator.cs	Сервис проверки изображений. Контролирует допустимое соотношение сторон загружаемых фотографий и логотипов.
ContractPdfService.cs	Сервис формирования PDF-договора по данным вылета и авиакомпании.
TicketPdfService.cs	Сервис формирования PDF-билетов для пассажиров вылета.
DeparturePdfDataFactory.cs	Модуль подготовки данных вылета для формирования документов.
ContractPdfDataFactory.cs	Модуль подготовки данных договора для генерации PDF-файла.

Продолжение таблицы 2.1.1

1	2
DatabaseCompatibilityService.cs	Сервис проверки совместимости структуры базы данных с ожидаемой структурой приложения.
AirlineProfileCompleteness.cs	Сервис проверки заполненности профиля авиакомпании. Используется для определения готовности авиакомпании к работе в системе.
Program.cs	Точка входа серверного приложения. Настраивает сервисы, подключение к базе данных, авторизацию, CORS и маршрутизацию API.
AirCharterExtendedContext.cs	Контекст базы данных Entity Framework Core. Описывает таблицы БД, связи между ними и параметры сопоставления моделей.
AuthenticationController.cs	Контроллер авторизации. Обрабатывает регистрацию, вход, подтверждение почты, повторную отправку кода и обновление токенов.

Описание процедур представлено в таблице 2.1.2.

Таблица 2.1.2 – Описание процедур

Процедуры	Описание процедур
1	2
App.tsx «Главный модуль приложения»	
App()	Настраивает маршруты приложения и связывает страницы между собой.
main.tsx «Точка входа приложения»	
createRoot(...).render(...)	Выполняет запуск приложения.
apiClient.ts «Базовый клиент HTTP-запросов»	
sendRequest(path, init)	Универсальная процедура отправки HTTP-запросов. Формирует адрес запроса, добавляет заголовки, проверяет успешность ответа и возвращает полученные данные.
sendRequest.ts «Универсальный модуль отправки запросов»	
sendRequest(path, method, body, signal)	Отправляет HTTP-запрос на сервер. При наличии токена добавляет авторизацию, обрабатывает ошибки и возвращает ответ сервера.
createApiError(status, responseText)	Формирует объект ошибки API на основе кода ответа и текста ошибки.
airportsService.ts «Модуль работы с аэропортами»	

Продолжение таблицы 2.1.2

1	2
searchAirports(query, limit, signal)	Выполняет запрос к серверу для поиска аэропортов по введённой строке.
authService.ts «Сервис авторизации и регистрации»	
register(request)	Отправляет на сервер данные для регистрации пользователя.
login(request)	Отправляет данные для входа пользователя и получает токен доступа.
confirmEmail(request)	Отправляет код подтверждения электронной почты.
resendEmailConfirmationCode(request)	Отправляет повторный запрос на получение нового кода подтверждения.
personService.ts «Сервис работы с профилем пользователя»	
getMyPerson()	Получает с сервера персональные данные текущего пользователя.
updateMyPerson(data)	Отправляет изменённые персональные данные пользователя на сервер для сохранения.
planesService.ts «Сервис работы с самолётами»	
getPlanes()	Получает общий список самолётов для отображения в каталоге.
getCatalogPlanes(takeOffAirportId, landingAirportId)	Получает список самолётов с расчётом параметров для выбранного маршрута.
userService.ts «Модуль работы с пользователем»	
getCurrentUser()	Получает данные текущего авторизованного пользователя.
getUserDepartures()	Получает список вылетов текущего пользователя.
authErrorMessage.ts «Обработка сообщений ошибок»	
getRegisterErrorMessage(error)	Преобразует ошибку регистрации в понятное текстовое сообщение.
getLoginErrorMessage(error)	Преобразует ошибку входа в понятное текстовое сообщение.
getConfirmEmailErrorMessage(error)	Преобразует ошибку подтверждения почты в понятное текстовое сообщение.
getResendCodeErrorMessage(error)	Преобразует ошибку повторной отправки кода в понятное текстовое сообщение.
getApiError(error)	Извлекает из объекта ошибки сведения о статусе и тексте ошибки.
getRegisterBadRequestMessage(message)	Возвращает подходящее сообщение при ошибке регистрации.
getLoginBadRequestMessage(message)	Возвращает подходящее сообщение при ошибке входа.
getConfirmEmailBadRequestMessage(message)	Возвращает подходящее сообщение при ошибке подтверждения почты.

Продолжение таблицы 2.1.2

1	2
getResendCodeBadRequestMessage(message)	Возвращает подходящее сообщение при ошибке повторной отправки кода.
ProtectedRoute.tsx «Проверка авторизации»	
ProtectedRoute({ children })	Проверяет, авторизован ли пользователь. Если пользователь не авторизован, выполняет перенаправление на страницу входа.
AirportSearch.tsx «Поиск аэропортов»	
AirportSearch({ label, onSelect, value })	Выполняет поиск при вводе текста
handleSelect(airport)	Обрабатывает выбор аэропорта из выпадающего списка. Формирует отображаемый текст, передаёт выбранный идентификатор наружу и закрывает список.
CatalogPlaneCard.tsx «Карточка самолёта»	
CatalogPlaneCard(props)	Отображает сведения о самолёте. Позволяет перейти к оформлению заказа.
UserDepartureCard.tsx «Карточка вылета пользователя»	
UserDepartureCard({ departure })	Отображает данные о вылете пользователя.
getStatusClass	Определяет CSS-класс для визуального отображения статуса вылета.
Header.tsx «Заголовок»	
Header({ searchValue, onSearchChange, showSearch, children })	Формирует верхнюю часть интерфейса. Отображает название приложения, навигацию, строку поиска и управляющие элементы.
handleLogout()	Выполняет выход пользователя из аккаунта и перенаправляет его на главную страницу.
InputField.tsx «Поле ввода»	
InputField(props)	Универсальный компонент поля ввода. Отображает подпись, поле ввода, значение и дополнительные элементы интерфейса.
ToggleThemeButton.tsx «Переключатель темы»	
ThemeToggle()	Позволяет переключать тему оформления между светлой и тёмной.
ThemeContext.tsx «Контекст темы оформления»	
ThemeProvider({ children })	Хранит текущее состояние темы оформления приложения и передаёт его во все дочерние компоненты.
toggleTheme()	Переключает тему оформления.
useTheme()	Предоставляет доступ к контексту темы в компонентах приложения.

Продолжение таблицы 2.1.2

1	2
UserContext.tsx «Контекст пользователя»	
UserProvider({ children })	Хранит данные текущего пользователя и состояние загрузки. При инициализации приложения запрашивает данные пользователя с сервера.
fetchUser()	Выполняет загрузку данных текущего пользователя. При ошибке очищает пользовательские данные.
logout()	Удаляет токен доступа и очищает данные пользователя в контексте.
useUser()	Предоставляет доступ к пользовательскому контексту в компонентах приложения.
CabinetPage.tsx «Личный кабинет пользователя»	
CabinetPage()	Формирует страницу личного кабинета. Отображает сведения о пользователе и список его вылетов.
fetchOrders()	Загружает с сервера список вылетов текущего пользователя и сохраняет их в состоянии страницы.
ProfilePage.tsx «Профиль пользователя»	
ProfilePage()	Формирует страницу профиля пользователя. Загружает текущие данные, отображает форму редактирования и выводит сообщения о результате сохранения.
loadPersonData()	Загружает персональные данные пользователя с сервера и подготавливает их для отображения в форме.
updateField(name, value)	Изменяет значение одного поля формы профиля.
validateAge(birthDate)	Проверяет возраст пользователя по дате рождения.
handleSubmit(e)	Проверяет корректность введенных данных и отправляет форму профиля на сервер.
ConfirmEmailPage.tsx «Подтверждение электронной почты»	
ConfirmEmailPage()	Формирует страницу подтверждения электронной почты. Управляет вводом почты и кода, а также состоянием отправки формы.
handleSubmit(event)	Проверяет введенные данные и отправляет код подтверждения на сервер.
handleResendCodeClick()	Выполняет повторную отправку кода подтверждения.

Продолжение таблицы 2.1.2

1	2
handleBackClick()	Возвращает пользователя на страницу регистрации.
LoginPage.tsx «Вход в систему»	
LoginPage()	Формирует страницу входа пользователя. Отображает форму входа и управляет её состоянием.
handleSubmit(event)	Проверяет поля формы, отправляет запрос на вход, сохраняет токен и обновляет данные пользователя.
handleRegisterClick()	Открывает страницу регистрации.
handleCatalogClick()	Выполняет переход на страницу каталога.
RegisterPage.tsx «Регистрация пользователя»	
RegisterPage()	Формирует страницу регистрации пользователя. Управляет вводом данных и выводом ошибок.
handleSubmit(event)	Проверяет корректность введённых данных и отправляет запрос на регистрацию.
handleLoginClick()	Открывает страницу входа.
handleCatalogClick()	Выполняет переход на страницу каталога.
CatalogPage.tsx «Каталог самолётов»	
formatFlightTime(timeStr)	Преобразует строковое значение времени полёта в удобный для отображения вид.
CatalogPage()	Формирует страницу каталога самолётов. Загружает самолёты, хранит фильтры, выбранный маршрут и состояние боковой панели.
fetchPlanes()	Загружает общий каталог самолётов или каталог с расчётом для выбранного маршрута.
filteredCatalogPlanes	Выполняет фильтрацию списка самолётов по модели, вместимости, дальности полёта, числу пересадок и строке поиска.
handleOrder(plane)	Выполняет переход к оформлению заказа и передаёт данные выбранного самолёта.
clearFilters()	Сбрасывает фильтры и выбранный маршрут.
AirportsController.cs «Контроллер поиска аэропортов»	
SearchAirports(query, limit, cancellationToken)	Обрабатывает запрос поиска аэропортов. Возвращает найденный список аэропортов клиенту.
AuthenticationController.cs «Контроллер аутентификации и регистрации»	

Продолжение таблицы 2.1.2

1	2
Register(request, cancellationToken)	Выполняет регистрацию пользователя. Генерирует код подтверждения и отправляет его на электронную почту.
ConfirmEmail(request, cancellationToken)	Подтверждает электронную почту пользователя.
ResendEmailConfirmationCode(request, cancellationToken)	Выполняет повторную отправку кода подтверждения.
Login(request, cancellationToken)	Выполняет вход пользователя в систему.
SetEmailConfirmationCode(user)	Формирует новый код подтверждения, сохраняет его хэш и срок действия в записи пользователя.
IsConfirmationCodeValid(user, confirmationCode)	Проверяет соответствие введенного кода его сохранённому хэшу.
GenerateEmailConfirmationCode()	Генерирует шестизначный код подтверждения электронной почты.
FlightsController.cs «Контроллер расчёта параметров рейса»	
GetCatalogPlanes(request, cancellationToken)	Рассчитывает параметры перелёта для каждого самолёта.
PersonsController.cs «Контроллер персональных данных пользователя»	
GetMyPerson(cancellationToken)	Получает персональные данные текущего авторизованного пользователя
UpdateMyPerson(request, cancellationToken)	Обновляет персональные данные текущего пользователя.
ValidateRequest(request)	Проверяет корректность данных формы.
GetOrCreatePerson(user)	Возвращает существующую запись персональных данных пользователя либо создаёт новую, если она отсутствует.
PlanesController.cs «Контроллер каталога самолётов»	
GetPlanes(cancellationToken)	Возвращает список самолётов для общего каталога без расчёта маршрута.
UsersController.cs «Контроллер пользователя»	
GetCurrentUser(cancellationToken)	Получает данные текущего пользователя.
GetMyDepartures(cancellationToken)	Получает список вылетов текущего пользователя.

2.2 Протокол тестирования программного продукта

Тестирование программного продукта является одним из ключевых этапов разработки, поскольку именно на этом этапе проверяется, насколько фактическая работа приложения соответствует установленным требованиям и ожидаемым результатам. Для этого используются заранее подготовленные

тестовые сценарии, позволяющие оценить различные стороны работы системы, включая её функциональные возможности, устойчивость, безопасность и удобство взаимодействия с пользователем.

Главная задача тестирования состоит в обнаружении ошибок, недочётов и возможных уязвимостей, которые могли появиться в процессе проектирования или реализации программного решения. Кроме того, результаты тестирования позволяют зафиксировать текущее состояние программы, что в дальнейшем облегчает её сопровождение, доработку и развитие.

В процессе проверки программного решения с использованием как корректных, так и некорректных входных данных критических ошибок, влияющих на работоспособность приложения или нарушающих целостность системы, обнаружено не было.

В таблицах 2.2.1 – 2.2.5 приведены протоколы тестирования: авторизация пользователя с корректными данными, авторизация пользователя с некорректными данными, выход из системы, переход на окно заказов и выход из заказов в каталог.

Таблица 2.2.1 – Протокол тестирования «Авторизация пользователя с корректными данными»

Наименование	Описание
1	2
Даты тестирования	13.04.2026
Test Case #	ТС_UI_1
Приоритет тестирования (Низкий/ Средний/ Высокий)	Высокий
Название тестирования/ Имя	Проверка авторизации при вводе корректных данных
Резюме испытания	Необходимо проверить корректность поведения программы при входе
Шаги тестирования	Нажатие на кнопку Вход Ввод данных в поля авторизации Нажатие кнопки «Войти»
Данные тестирования	Логин «PrivatAir@gmail.com»; Пароль «zxсzxc».

Продолжение таблицы 2.2.1

1	2
Ожидаемый результат	Открытие каталога
Фактический результат	Открылся каталог

На рисунках 2.2.1 – 2.2.2 изображены результаты тестирования «Авторизация пользователя с корректными данными».

Вход

Почта

PrivatAir@gmail.com

Пароль

.....

Регистрация Войти

Рисунок 2.2.1 – Ввод корректных данных

AirCharter Поиск по модели... Выход Личный кабинет

Маршрут

Вылет
Город, страна, или код...

Прибытие
Город, страна, или код...

Сортировка

Сначала популярные

Цена

Авиакомпании

Параметры

Сбросить всё

Авиационный транспорт	Мест	Дальность	Скорость	Пересадки	Вылеты	Цена	Действие
	19	13890 км	904 км/ч	0	4	975 000 Р/ч	Заказать
	9	3441 км	815 км/ч	0	3	322 500 Р/ч	Заказать
	30	2907 км	504 км/ч	0	2	206 250 Р/ч	Заказать

Рисунок 2.2.2 – Открытие каталога

Таблица 2.2.2 – Протокол тестирования «Авторизация пользователя с некорректными данными»

Наименование	Описание
Даты тестирования	13.04.2026
Test Case #	ТС_UI_2
Приоритет тестирования (Низкий/ Средний/ Высокий)	Средний
Название тестирования/ Имя	Проверка авторизации при вводе некорректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при входе с некорректными данными
Шаги тестирования	Нажатие на кнопку Вход Ввод некорректных данных в поля авторизации Нажатие кнопки «Войти»
Данные тестирования	Логин «PrivatAir@gmail.com»; Пароль «qstbna».
Ожидаемый результат	Появление сообщения «Неверная почта или пароль.»
Фактический результат	Появилось сообщение «Неверная почта или пароль.»

На рисунках 2.2.3 – 2.2.4 изображены результаты тестирования «Авторизация пользователя с некорректными данными».

Рисунок 2.2.3 – Ввод некорректных данных для авторизации

Рисунок 2.2.4 – Вывод сообщения: «Неверная почта или пароль.»

Таблица 2.2.3 – Протокол тестирования «Выход из системы»

Наименование	Описание
Даты тестирования	13.04.2026
Test Case #	ТС_UI_3
Приоритет тестирования (Низкий/ Средний/ Высокий)	Низкий
Название тестирования/ Имя	Проверка выхода из системы
Резюме испытания	Необходимо проверить работоспособность выхода из системы
Шаги тестирования	Нажатие на кнопку Вход Ввод корректных данных в поля авторизации Нажать на кнопку «Войти» Нажать кнопку «Выход»
Данные тестирования	Логин «PrivatAir@gmail.com»; Пароль «zxсzxc».
Ожидаемый результат	Выход из аккаунта и появление кнопок «Вход» и «Регистрация»
Фактический результат	Произошёл выход из аккаунта и появились кнопки «Вход» и «Регистрация»

На рисунках 2.2.5 – 2.2.6 изображены результаты тестирования «Выход из системы».

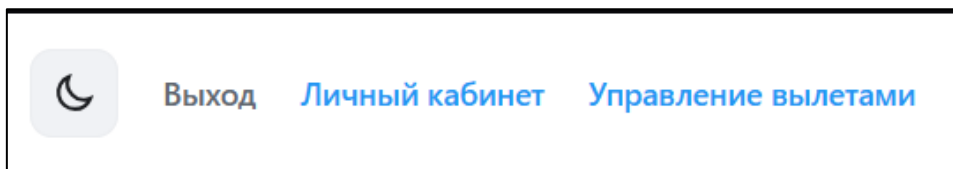


Рисунок 2.2.5 – Нажатие кнопки «Выход»

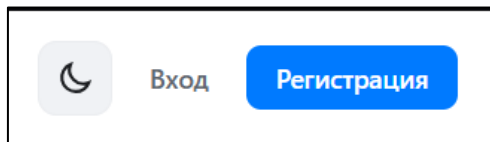


Рисунок 2.2.6 – Выход из аккаунта и появление кнопок «Вход» и «Регистрация»

Таблица 2.2.4 – Протокол тестирования «Переход на окно заказов»

Наименование	Описание
1	2
Даты тестирования	13.04.2026
Test Case #	ТС_UI_4
Приоритет тестирования (Низкий/ Средний/ Высокий)	Высокий
Название тестирования/ Имя	Проверка работы перехода на окно заказов
Резюме испытания	Необходимо проверить переход на окно заказов
Шаги тестирования	Нажатие на кнопку Вход Ввод корректных данных в поля авторизации Нажать на кнопку «Войти» Нажать кнопку «Личный кабинет»
Данные тестирования	Логин «PrivatAir@gmail.com»; Пароль «zxсzxc».
Ожидаемый результат	Открытие окна заказов
Фактический результат	Открылось окно заказов

На рисунках 2.2.7 – 2.2.8 изображены результаты тестирования «Переход на окно заказов».

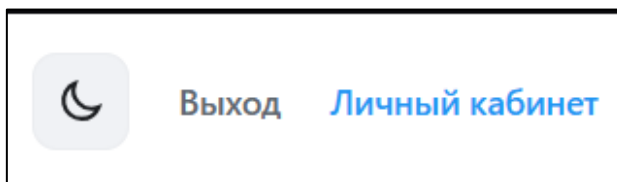


Рисунок 2.2.7 – Нажатие кнопки «Личный кабинет»

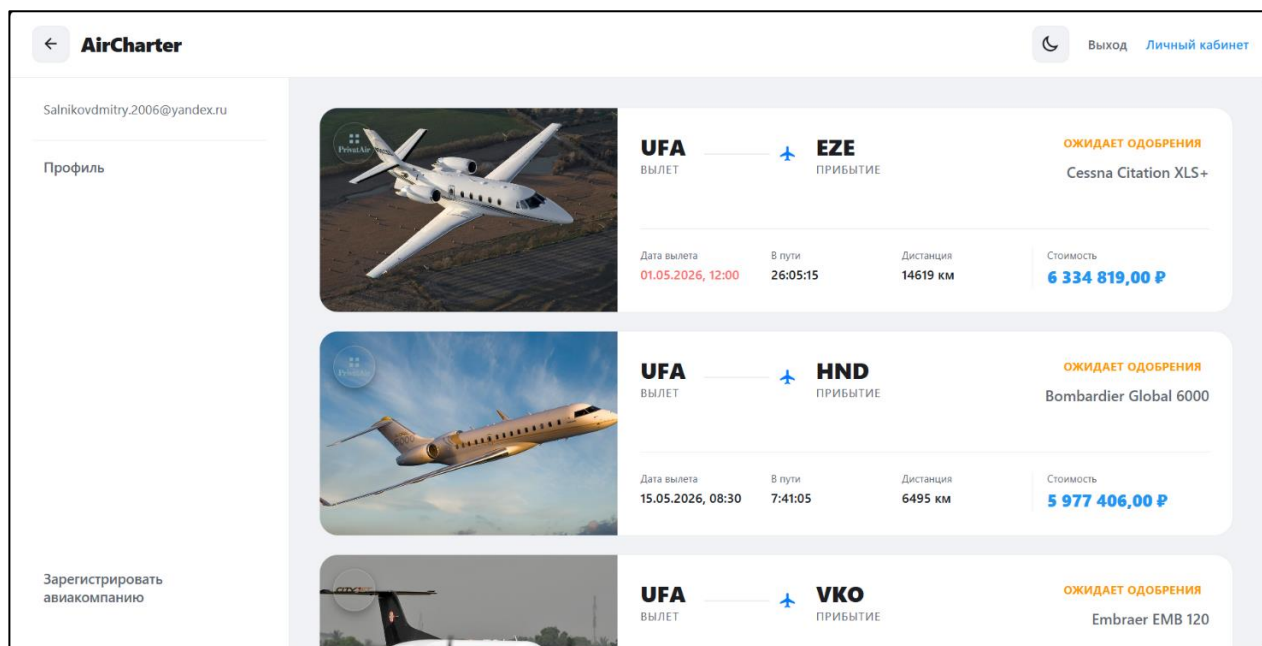


Рисунок 2.2.8 – Переход на окно заказов

Таблица 2.2.5 – Протокол тестирования «Выход из заказов в каталог»

Наименование	Описание
Даты тестирования	13.04.2026
Test Case #	ТС_UI_5
Приоритет тестирования (Низкий/ Средний/ Высокий)	Средний
Название тестирования/ Имя	Проверка выхода из заказов в каталог
Резюме испытания	Необходимо проверить систему на корректность выхода из окна заказов в окно каталога
Шаги тестирования	Нажатие на кнопку Вход Ввод корректных данных в поля авторизации Нажать на кнопку «Войти» Нажать кнопку «Личный кабинет» Нажать на кнопку «←»
Данные тестирования	Логин «PrivatAir@gmail.com»; Пароль «zxсzxс».
Ожидаемый результат	Открытие каталога
Фактический результат	Открылся каталог

На рисунках 2.2.9 – 2.2.10 изображены результаты тестирования «Выход из заказов в каталог».

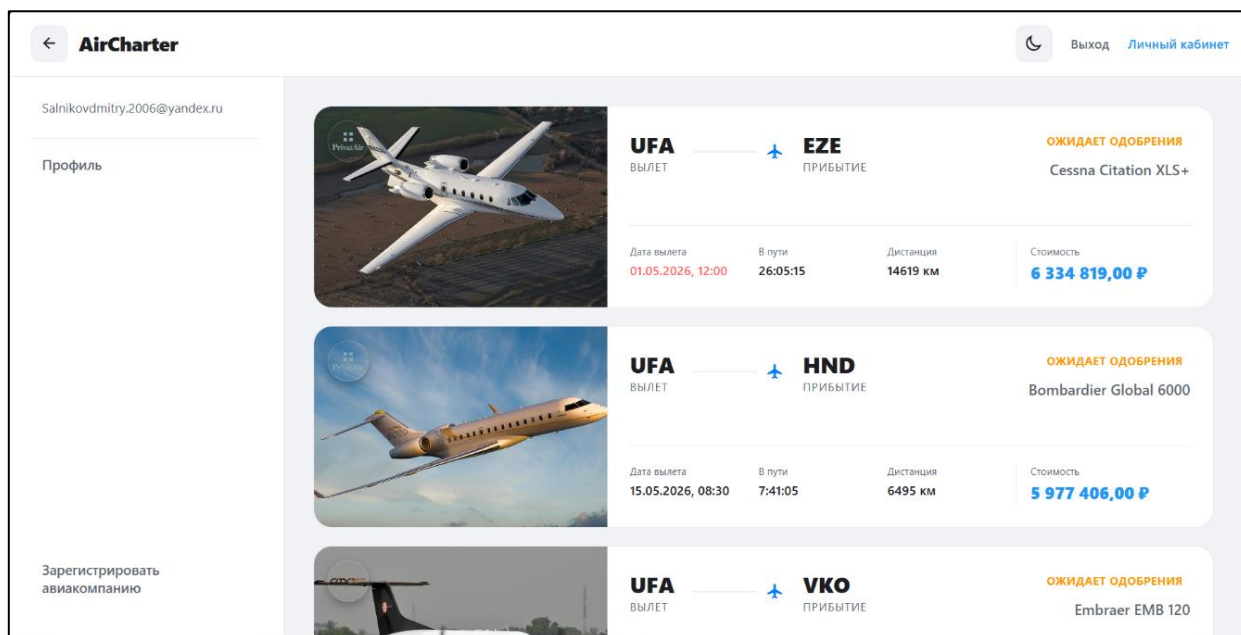


Рисунок 2.2.9 – Окно заказов

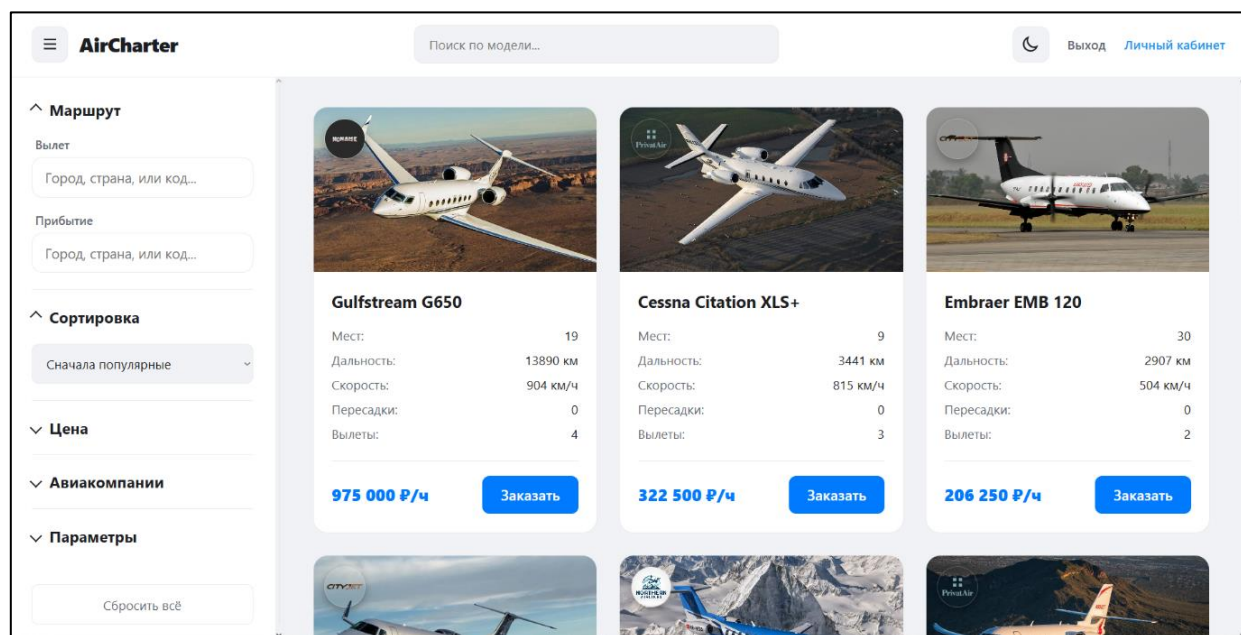


Рисунок 2.2.10 – Возврат в каталог

2.3 Руководство пользователя

Разработанное веб-приложение предназначено для просмотра каталога самолётов, расчёта параметров чартерного рейса, создания заявок, работы с документами и сопровождения вылетов.

Основной целью данной информационной системы является управление авиапарком, а также создание заявок для упрощения организации чартерных рейсов.

Для работы с клиентской частью программного продукта пользователю необходим персональный компьютер, ноутбук или другое устройство с современным веб-браузером и доступом к сети, в которой размещено приложение.

Минимальные требования к клиентскому устройству:

- операционная система Windows 10/11, Linux или macOS;
- веб-браузер Google Chrome, Microsoft Edge, Mozilla Firefox, Safari или аналогичный;
- стабильное подключение к сети;
- разрешение экрана монитора не менее 1920 × 1080;
- принтер;
- сканер или многофункциональное устройство.

Веб-приложение предназначено для пользователей, имеющих базовые навыки работы с персональным компьютером, графическим интерфейсом операционной системы и веб-браузером. Пользователь должен уметь вводить данные в формы, выбирать значения из списков, загружать и скачивать файлы, а также переходить между разделами веб-приложения.

Подготовка системы к работе

Для запуска клиентской части программы необходимо открыть веб-приложение в браузере по адресу <http://localhost:5173>. После загрузки приложения пользователю отображается страница каталога самолётов (рисунок 2.3.1).

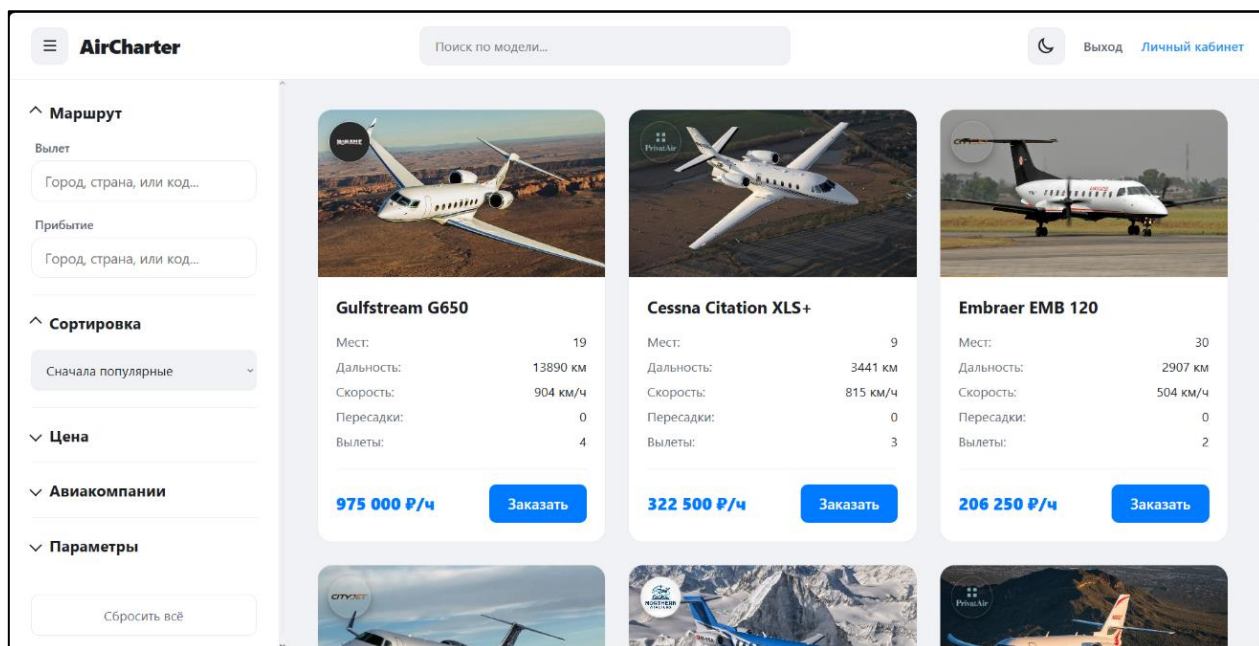


Рисунок 2.3.1 – Каталог самолётов

Для доступа к полному функционалу пользователю нужно войти в аккаунт. Для перехода на окно авторизации нажмите на кнопку «Вход». Введите свои данные и нажмите «Войти» для входа в аккаунт (рисунок 2.3.2).

Вход

Почта

Пароль

[Регистрация](#)
[Войти](#)

Рисунок 2.3.2 – Окно авторизации для входа в программу

Для регистрации нового пользователя необходимо перейти на страницу регистрации, для этого нажмите на кнопку «Регистрация». В открывшейся

форме укажите электронную почту и пароль. После заполнения необходимо нажать кнопку регистрации (рисунок 2.3.3).

The screenshot shows a registration form with the title "Регистрация" in bold. It contains three input fields: "Почта" (Email), "Пароль" (Password), and "Подтвердите пароль" (Confirm password). Below the fields are two buttons: "Вход" (Login) in blue text and "Зарегистрироваться" (Register) in white text on a blue background.

Рисунок 2.3.3 – Регистрация пользователя

После регистрации на указанную электронную почту отправляется код подтверждения. Для завершения регистрации необходимо ввести полученный код в поле подтверждения и нажать кнопку подтверждения почты (рисунок 2.3.4).

The screenshot shows an email confirmation form with the title "Подтверждение почты" in bold. It contains two input fields: "Почта" (Email) with the value "Salnikovdmitry.2006@yandex.ru" and "Код" (Code). Below the fields are three buttons: "Назад" (Back) in blue text, "Отправить код снова" (Resend code) in blue text, and "Подтвердить" (Confirm) in white text on a blue background.

Рисунок 2.3.4 – Подтверждение электронной почты

После ввода правильного логина и пароля осуществляется вход в приложение. Если в систему вошел пользователь, то будут доступны вкладки,

необходимые для осуществления заказа и просмотра имеющихся заказов (рисунок 2.3.5).

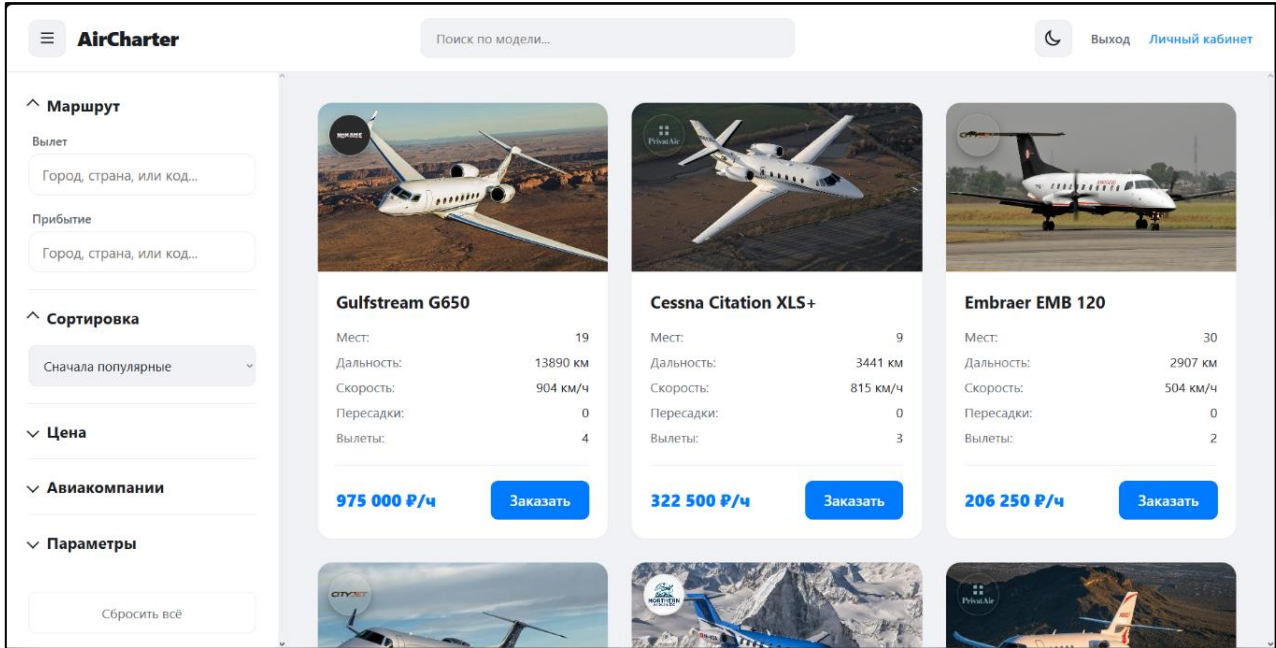


Рисунок 2.3.5 – Интерфейс приложения после входа в аккаунт

При входе в приложение отображается список самолётов. Для того, чтобы сравнить время перелёта на разных самолётах необходимо выбрать аэропорт взлёта и прибытия. (рисунок 2.3.6)

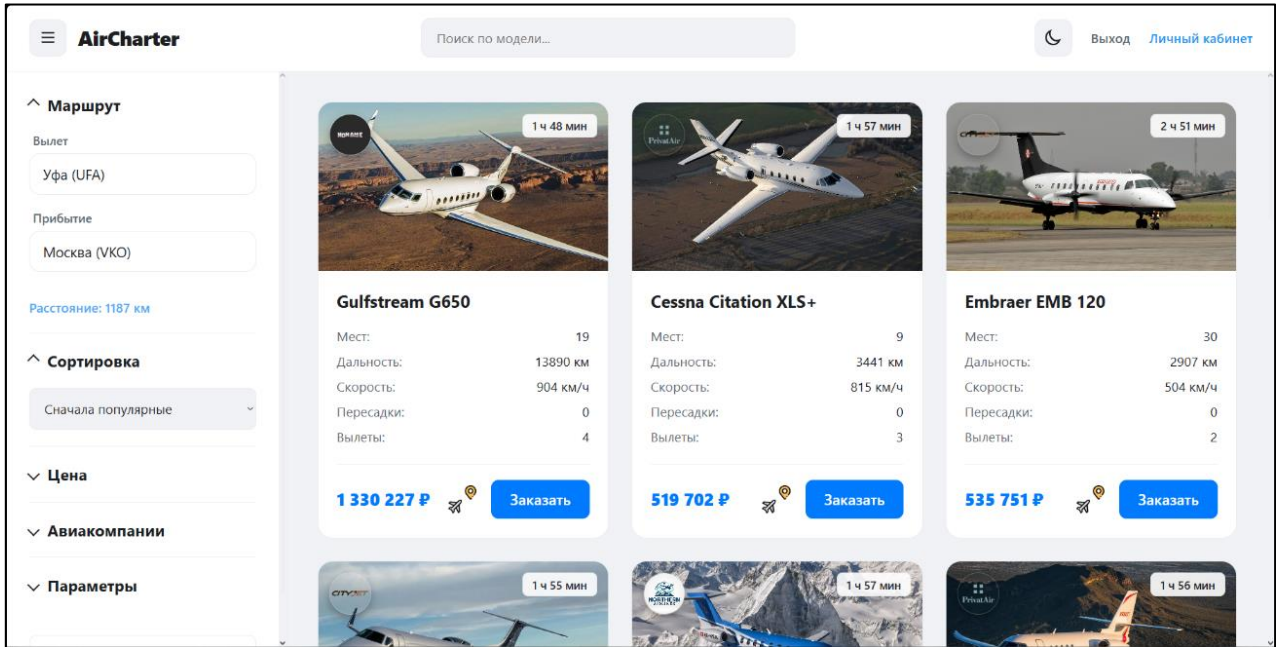


Рисунок 2.3.6 – Интерфейс приложения, в случае указания аэропорта взлёта и прибытия

В случае дальних полётов (если максимальной дальности самолёта недостаточно), указывается количество технических посадок. (рисунок 2.3.7)

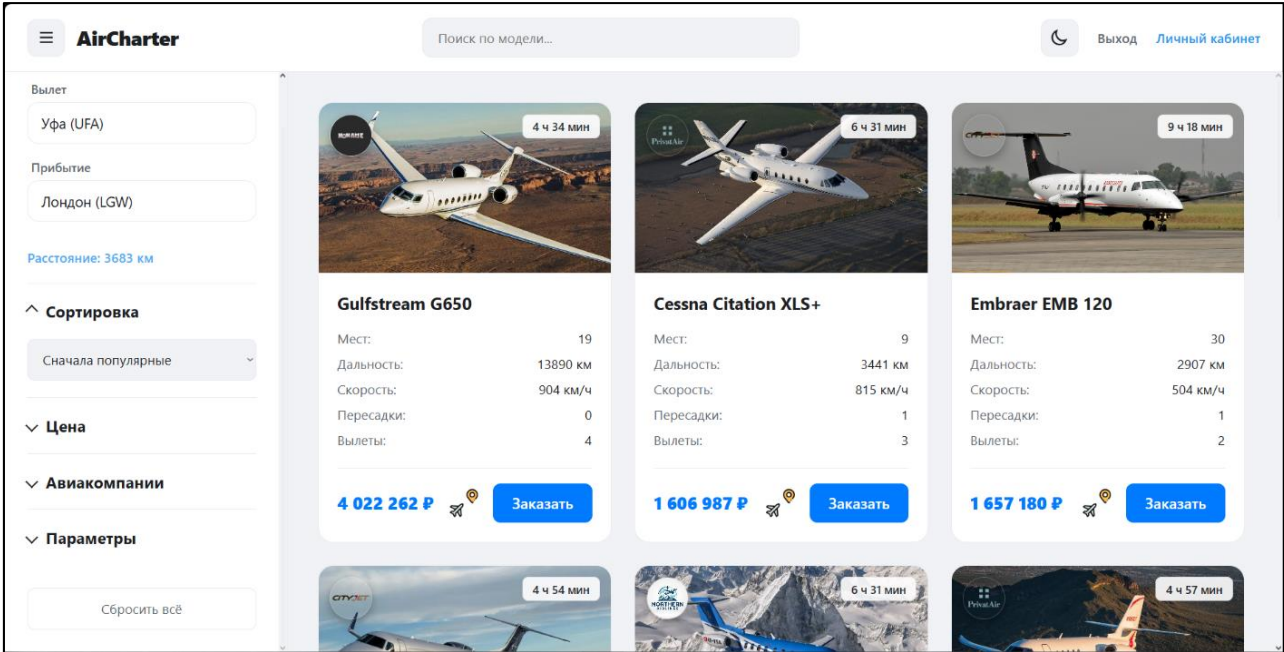


Рисунок 2.3.7 – Интерфейс приложения, в случае указания дальних аэропортов

Для фильтрации списка самолётов можно воспользоваться панелью в правой части экрана (рисунок 2.3.8)

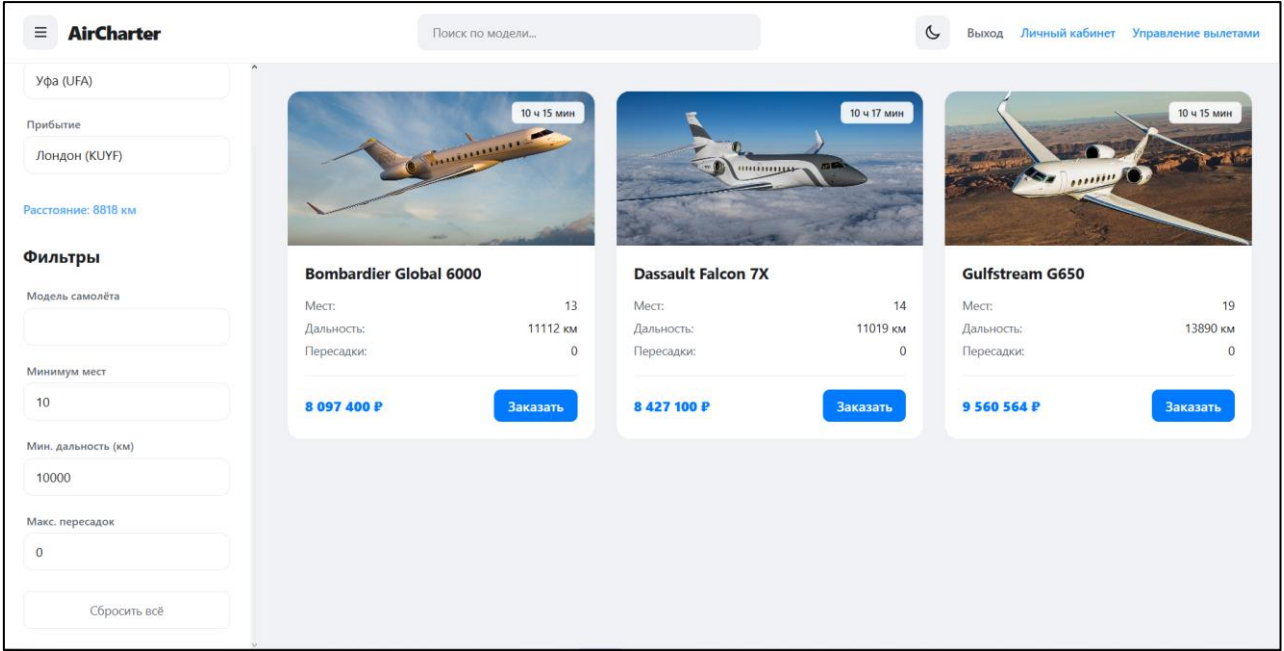


Рисунок 2.3.8 – Фильтрация каталога

Для просмотра своих, ранее созданных, заказов нажмите на кнопку «Личный кабинет» (Рисунки 2.3.9 – 2.3.10). В личном кабинете пользователь

также может перейти к разделу уведомлений. Если имеются непрочитанные уведомления, рядом с кнопкой отображается числовой индикатор.

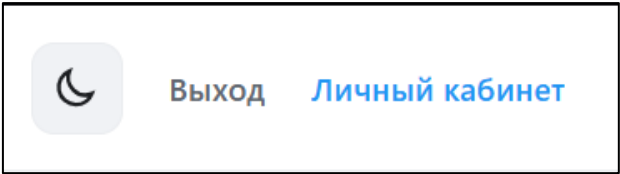


Рисунок 2.3.9 – Кнопка «Личный кабинет»

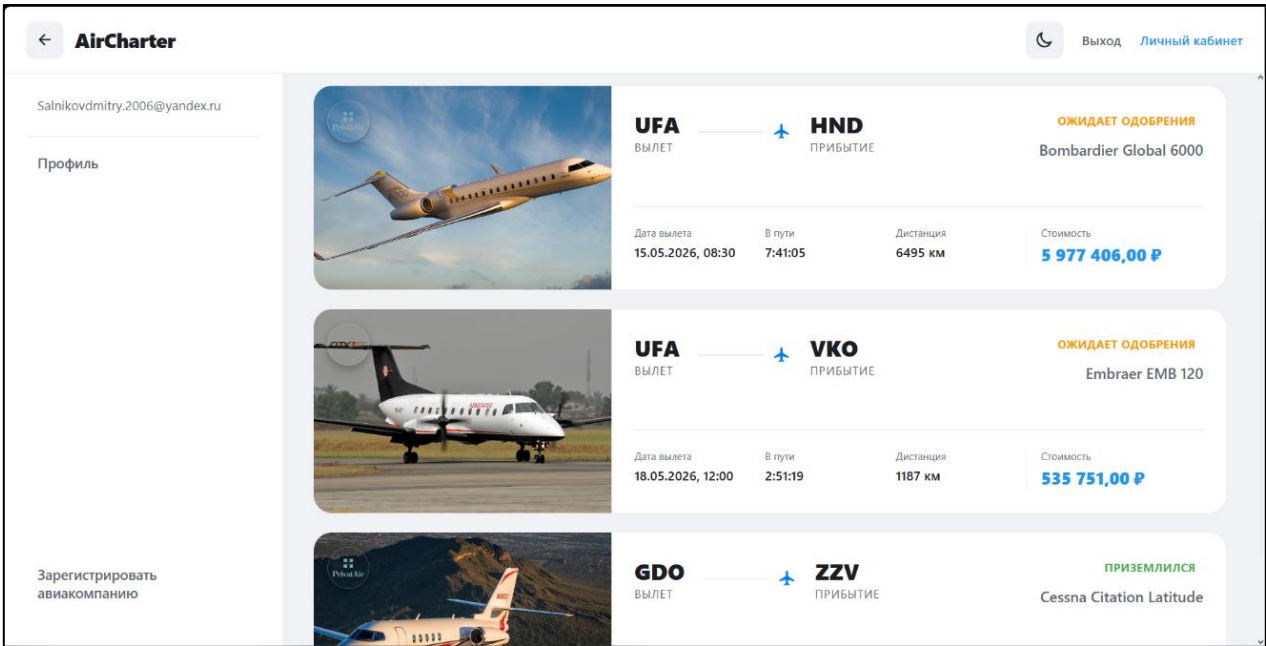


Рисунок 2.3.10 – Окно заказов

Для редактирования личных данных необходимо перейти в профиль. Для этого нажмите кнопку «Профиль» в правом верхнем углу (рисунок 2.3.11).

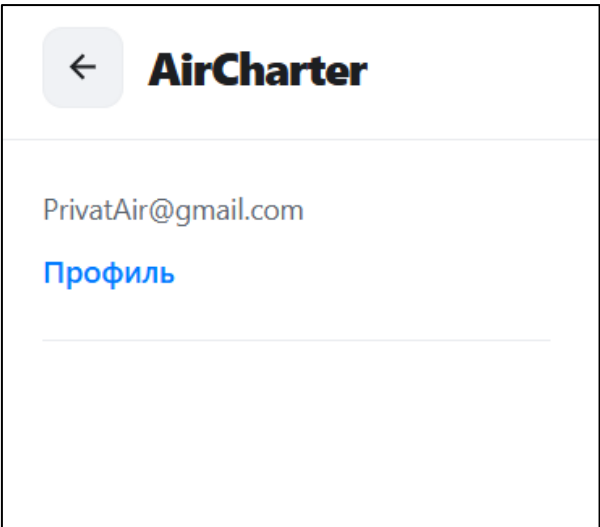


Рисунок 2.3.11 – Кнопка «Профиль»

Для редактирования данных пользователя введите нужные данные и нажмите на кнопку сохранить. В случае некорректных данных, появится сообщение с подсказкой (рисунок 2.3.12).

Профиль

Фамилия

Кузцов

Имя

Ибрагим

Отчество

Андреевич

Серия

8020

Номер

614352

Дата рождения

19.04.2003

Email для связи

ibragim.kuzcenov@example.com

Изменения сохранены

Сохранить

Рисунок 2.3.12 – Профиль пользователя

Для создания заявки на чартерный рейс необходимо в каталоге выбрать подходящий самолёт и нажать кнопку «Заказать» на карточке самолёта (рисунок 2.3.13).



Gulfstream G650

Мест: 19

Дальность: 13890 км

Скорость: 904 км/ч

Пересадки: 0

Вылеты: 4

975 000 ₽/ч

Заказать

Рисунок 2.3.13 – Кнопка создания заявки на рейс

После перехода на страницу создания заявки необходимо проверить выбранный маршрут, указать дату и время вылета. Система автоматически рассчитывает расстояние, время полёта, стоимость рейса и количество пересадок. Для сохранения заявки необходимо нажать кнопку «Создать черновик заявки» (рисунок 2.3.14).

←

Маршрут: Уфа (UFA) → Лондон (LGW)
Gulfstream G650

Откуда
Уфа (UFA)

Куда
Лондон (LGW)

Дата вылета
ДД.ММ.ГГГГ

Время
12:00

Итоговая стоимость: **4 022 262 ₽**

Расстояние: 3 683 км

Время в пути: 4 ч 34 мин

Пересадки: 0

Детали маршрута

Уфа (UFA) → Лондон (LGW)
3 683 км • 4 ч 4 мин • 4 022 262 ₽

Создать черновик заявки

Рисунок 2.3.14 – Создание черновика заявки на чартерный рейс

После успешного создания черновика заявки пользователь может перейти к просмотру созданного вылета в личном кабинете. Для этого необходимо нажать кнопку перехода к заявке (рисунок 2.3.15).

Черновик заявки создан

Чтобы отправить заявку менеджеру, откройте её в личном кабинете, проверьте пассажиров и нажмите «Отправить заявку».

Перейти в личный кабинет

Рисунок 2.3.15 – Сообщение об успешном создании черновика заявки

Для просмотра подробной информации о вылете необходимо открыть нужную заявку из списка заказов в личном кабинете. На странице вылета отображаются данные маршрута, самолёта, стоимости, времени полёта, текущего статуса и пассажиров (рисунок 2.3.16).

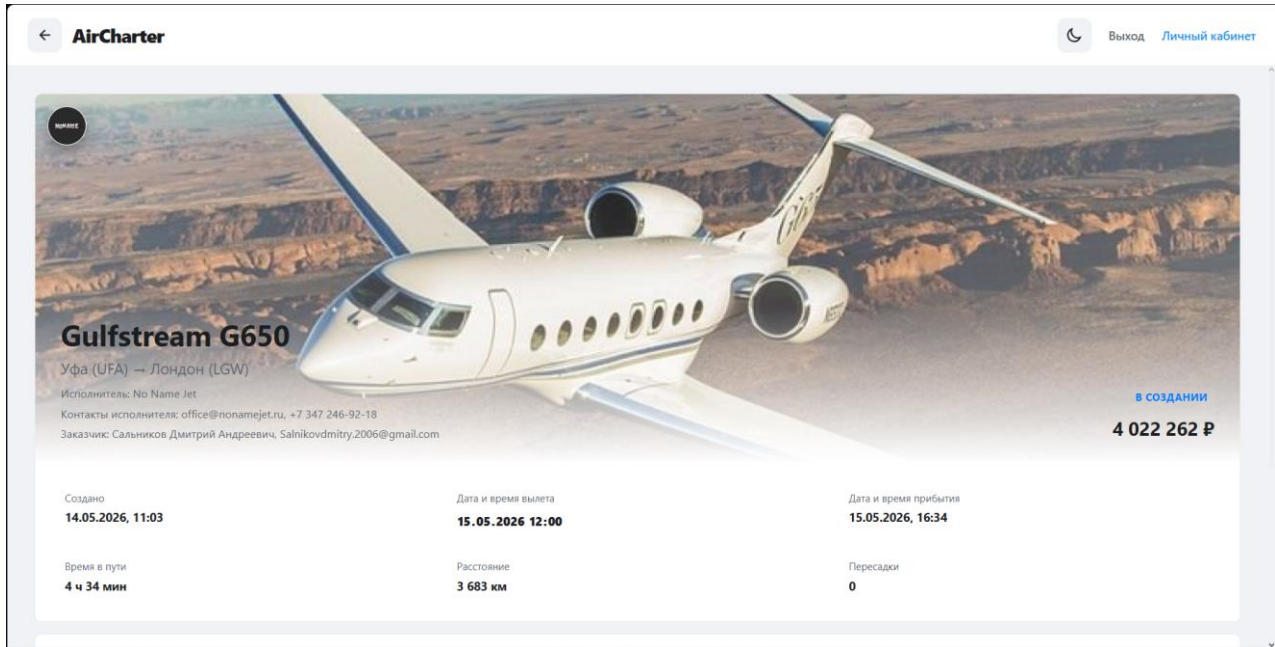


Рисунок 2.3.16 – Подробная информация о заказанном вылете

Для добавления пассажира к вылету необходимо в карточке вылета открыть раздел пассажиров и нажать кнопку добавления пассажира. В открывшейся форме требуется указать фамилию, имя, отчество, паспортные данные, почту для уведомлений и дату рождения, после чего нажать кнопку сохранения (рисунок 2.3.17).

Новый пассажир

Фамилия

Имя

Отчество

Серия паспорта

Номер паспорта

Почта для уведомлений

Дата рождения

Отмена

Добавить

Рисунок 2.3.17 – Добавление пассажира к вылету

При необходимости пользователь может изменить данные пассажира или удалить пассажира из списка вылета. Для этого нужно выбрать нужного пассажира в списке и выполнить требуемое действие в открывшейся форме (рисунок 2.3.18).

Рисунок 2.3.18 – Редактирование данных пассажира

Если заявка ещё доступна для изменения, пользователь может скорректировать маршрут и дату вылета. Для изменения маршрута необходимо выбрать аэропорты в блоке маршрута, при необходимости добавить промежуточный аэропорт, после чего нажать кнопку «Сохранить» (рисунок 2.3.19).

Рисунок 2.3.19 – Редактирование маршрута вылета

После одобрения заявки пользователь может скачать маршрутную квитанцию и шаблон договора. Для этого на странице вылета используются

кнопки «Сохранить маршрутную квитанцию» и «Сохранить шаблон договора» (рисунок 2.3.20).

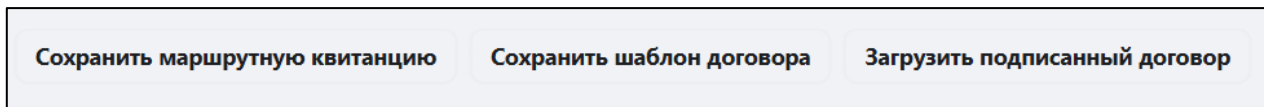


Рисунок 2.3.20 – Скачивание маршрутной квитанции и договора

Для передачи подписанного договора необходимо нажать кнопку «Загрузить подписанный договор» и выбрать файл документа на компьютере. После загрузки файл будет прикреплен к вылету.

Для оплаты вылета необходимо нажать кнопку оплаты. В открывшемся окне требуется указать номер карты, срок действия и CVC-код, после чего подтвердить оплату (рисунок 2.3.21).

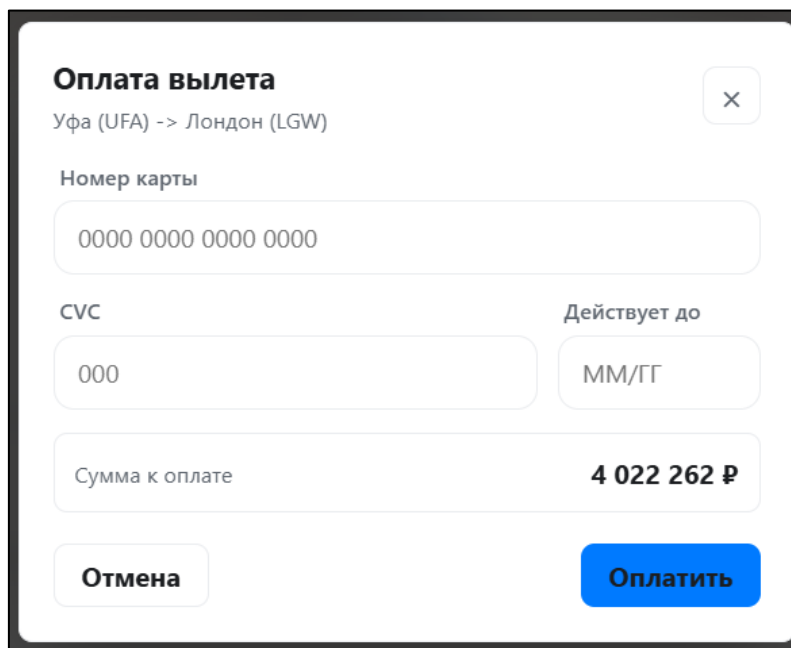


Рисунок 2.3.21 – Окно оплаты вылета

Если пользователь является представителем авиакомпании, ему доступен раздел управления вылетами. Для перехода в него необходимо нажать кнопку «Управление вылетами» в верхней части интерфейса (рисунок 2.3.22).

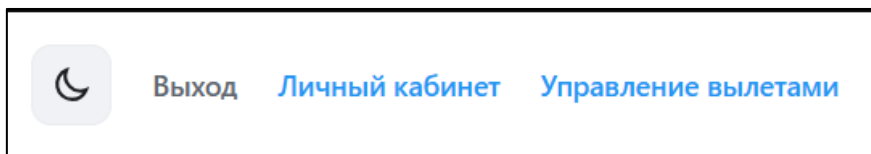


Рисунок 2.3.22 – Переход в раздел управления вылетами

В разделе управления отображаются заявки и вылеты авиакомпании. Сотрудник может просматривать карточки заявок, открывать подробную информацию, одобрять или отклонять заявки, а также переходить к управлению конкретным вылетом (рисунок 2.3.23).

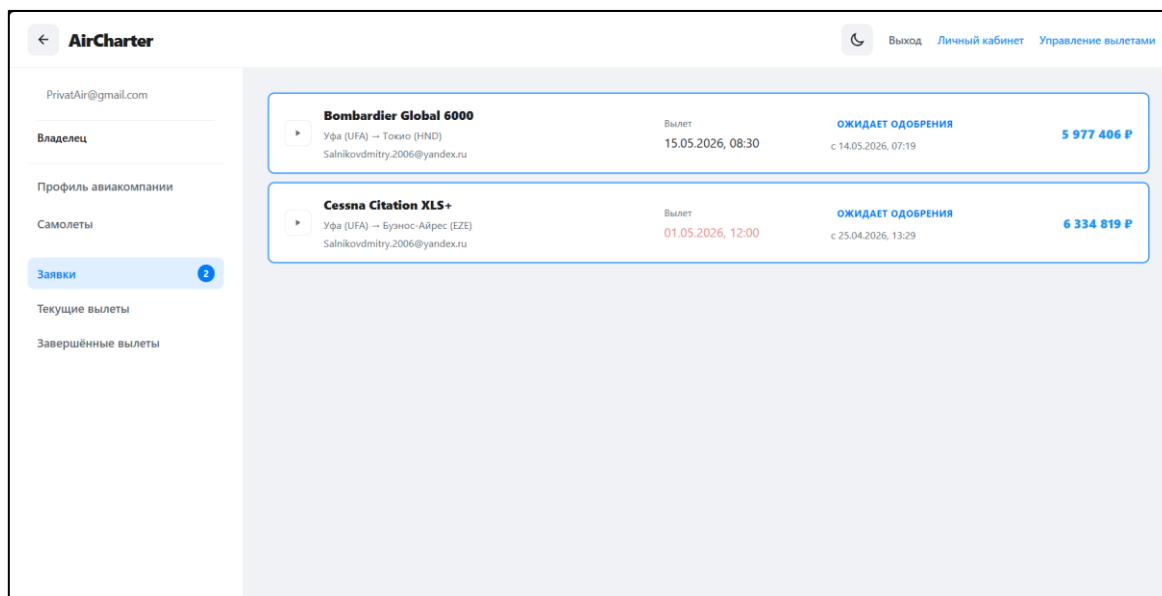


Рисунок 2.3.23 – Раздел управления вылетами

Для обработки новой заявки сотрудник открывает карточку заявки и проверяет маршрут, самолёт, стоимость и данные заказчика. Если заявка корректна, необходимо нажать кнопку «Одобрить». Если выполнение заявки невозможно, необходимо нажать кнопку «Отклонить» (рисунок 2.3.24).

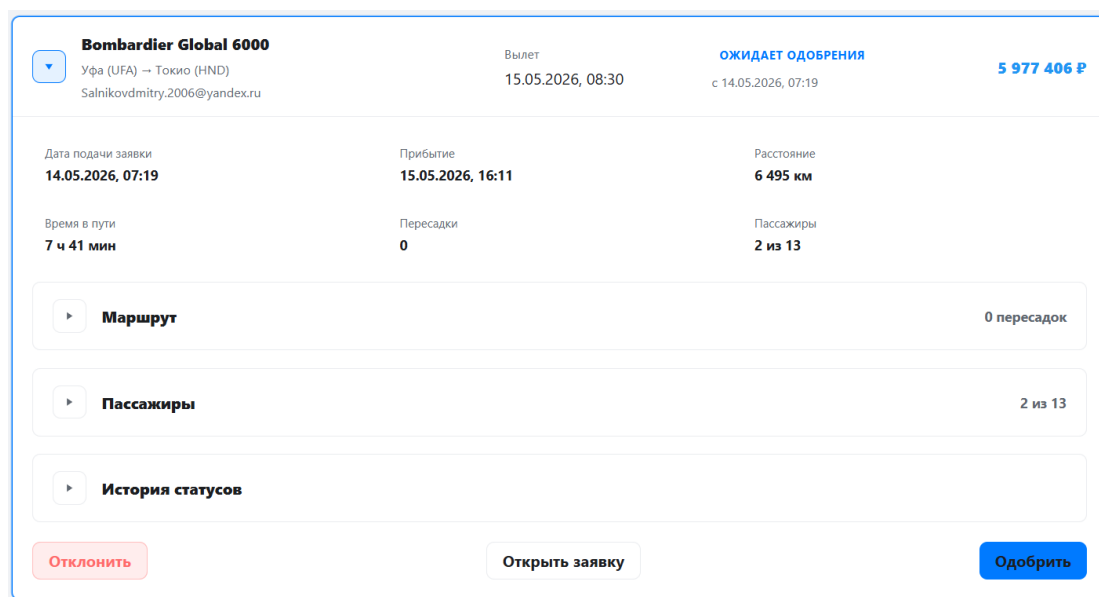


Рисунок 2.3.24 – Одобрение или отклонение заявки

Для сопровождения вылета сотрудник открывает страницу управления вылетом. На данной странице можно изменить маршрут, дату и время вылета, управлять пассажирами, назначать сотрудников и изменять статус вылета (рисунок 2.3.25).

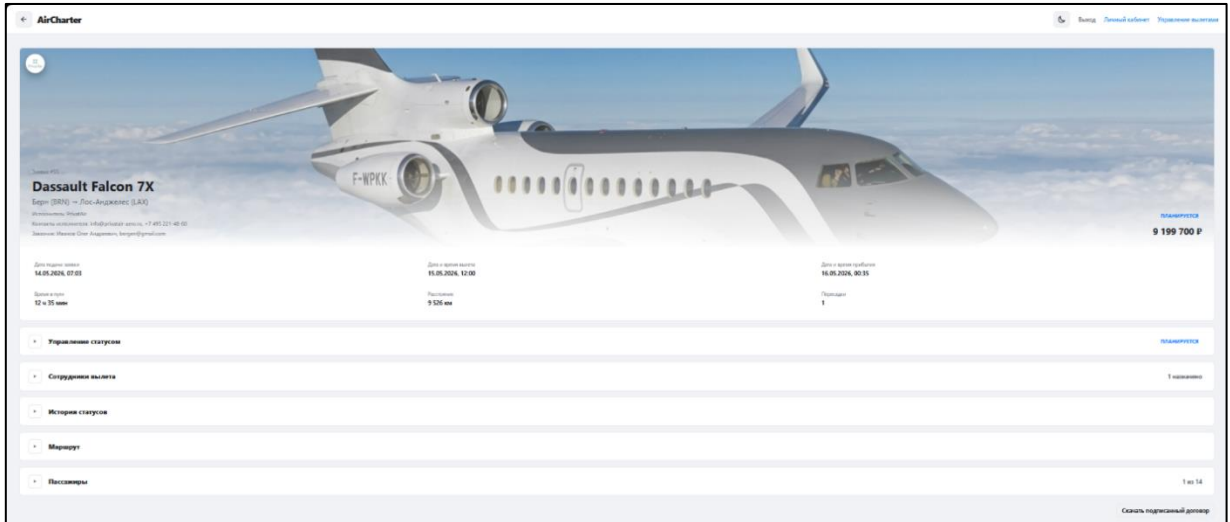


Рисунок 2.3.25 – Страница управления вылетом

Для назначения сотрудников на вылет необходимо открыть блок сотрудников, выбрать ответственных пользователей и сохранить изменения. После этого выбранные сотрудники будут связаны с данным вылетом (рисунок 2.3.26).

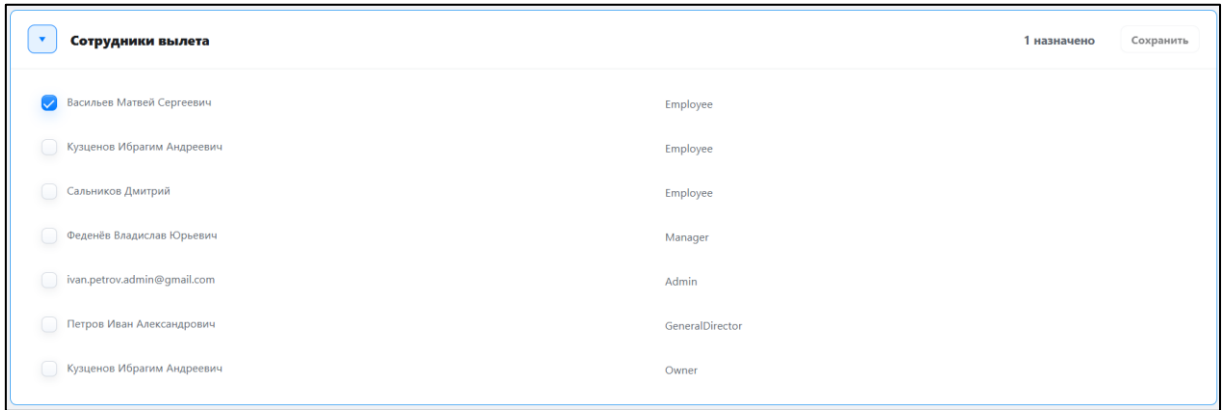


Рисунок 2.3.26 – Назначение сотрудников на вылет

Для изменения статуса вылета необходимо выбрать следующий статус в блоке управления статусами. После подтверждения новый статус будет сохранён и станет доступен клиенту в личном кабинете (рисунок 2.3.27).

Управление статусом

ПЛАНИРУЕТСЯ

Расчётный статус: Ожидает вылета

Берн (BRN)

Вылет запланирован на 15.05.2026, 12:00.

Фактическое состояние: Планируется

Берн (BRN)

Установлено 14.05.2026, 07:12.

Следующий статус

Регистрация открыта

Предложение построено по маршруту и пересадкам.

15.05.2026, 12:00	Вылет из Берн (BRN)	В пути 43 мин
15.05.2026, 12:43	Прибытие в Лидд (LYX)	Промежуточная посадка, стоянка 1 ч 30 мин
15.05.2026, 14:13	Вылет из Лидд (LYX)	В пути 9 ч 51 мин
16.05.2026, 00:04	Прибытие в Лос-Анджелес (LAX)	Финальная точка маршрута

Удалить текущий статус

Установить расчётный статус

Установить следующий статус

Рисунок 2.3.27 – Изменение статуса вылета

Для завершения вылета необходимо нажать кнопку завершения и подтвердить действие в открывшемся окне. После завершения вылет будет перенесён в список завершённых вылетов (рисунок 2.3.28).

Завершить вылет?

Берн (BRN) -> Лос-Анджелес (LAX)

Будет установлен финальный статус «Приземлился», а вылет перейдёт из текущих в завершённые. Если самолёт ещё не приземлился, лучше отменить действие и дождаться фактической посадки.

Текущий статус

В пути

Расчётное прибытие

16.05.2026, 00:35

Отмена

Завершить вылет

Рисунок 2.3.28 – Завершение вылета

Для управления авиапарком сотрудник авиакомпании переходит в раздел «Самолёты». В этом разделе отображается список самолётов авиакомпании с их основными характеристиками (рисунок 2.3.29).

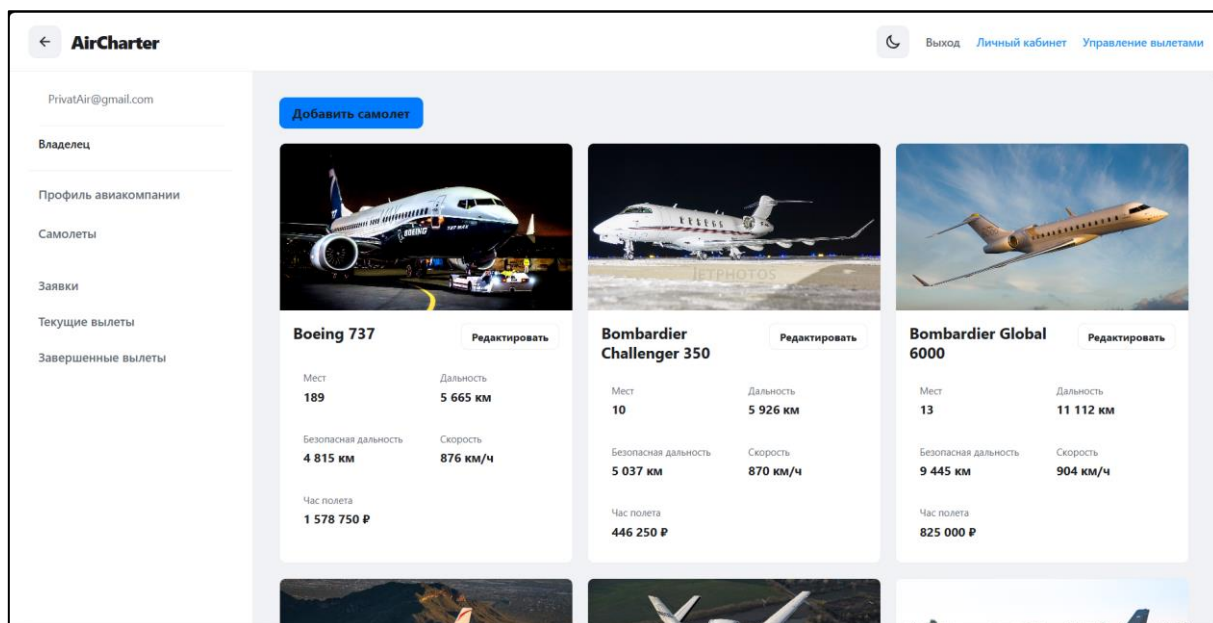


Рисунок 2.3.29 – Раздел управления самолётами

Для добавления нового самолёта необходимо нажать кнопку добавления самолёта, заполнить модель, дальность полёта, пассажировместимость, крейсерскую скорость, стоимость часа полёта и загрузить фотографию. После заполнения формы необходимо нажать кнопку «Сохранить» (рисунок 2.3.30).

Рисунок 2.3.30 – Добавление самолёта

Для изменения данных самолёта необходимо открыть карточку нужного самолёта, внести изменения в форму и нажать кнопку «Сохранить». Изменённые данные будут использоваться в каталоге и при расчёте заявок.

Для редактирования данных авиакомпании необходимо перейти в профиль авиакомпании. На странице профиля можно изменить основные сведения, адреса, реквизиты, условия договора, логотип и статус отображения авиакомпании в каталоге (рисунки 2.3.31-2.3.32).

Основное

Название

PrivatAir

Тип организации

ООО - Общество с ограниченной ответственнос

Email

info@privatair-aero.ru

Телефон

+7 495 221-48-60

Базовая стоимость обслуживания

50000

Базовая стоимость пересадки

100000

Адреса и реквизиты

Юридический адрес

119019, г. Москва, ул. Новый Арбат, д. 21, офис €

Почтовый адрес

119019, г. Москва, ул. Новый Арбат, д. 21, офис €

ИНН

7702457812

КПП

770201001

ОГРН

1027700457812

Банк

АО «Альфа-Банк»

Расчётный счёт

40702810401470002136

Корреспондентский счёт

30101810200000000593

БИК

044525593

Рисунок 2.3.31 – Профиль авиакомпании

Условия договора

Город договора

Уфа

Срок действия договора, дней

30

Срок оплаты, дней

7

Прибытие пассажиров до вылета, минут

60

Класс бортипитания

Бизнес

Скрыть из каталога

Сохранить

Рисунок 2.3.32 – Профиль авиакомпании

Таблица 2.3.1 – Сообщение пользователю

Сообщение	Причина	Что делать
1	2	3
Укажите почту.	Пользователь не ввёл адрес электронной почты	Ввести почту и повторить действие
Укажите пароль.	Пользователь не ввёл пароль	Ввести пароль и повторить действие
Неверная почта или пароль.	Введены неправильные данные для входа	Проверить почту и пароль, затем повторить вход

Продолжение таблицы 2.3.1

1	2	3
Почта не подтверждена.	Пользователь пытается войти без подтверждения почты	Подтвердить почту с помощью кода подтверждения
Пользователь с такой почтой уже существует.	При регистрации указана уже занятая почта	Использовать другую почту или выполнить вход
Не удалось зарегистрироваться.	Произошла ошибка при регистрации	Проверить введённые данные и повторить попытку
Укажите код подтверждения.	Не введён код подтверждения почты	Ввести код подтверждения
Неверный код подтверждения.	Введён неправильный код подтверждения	Проверить код и ввести его повторно
Срок действия кода истёк.	Код подтверждения больше не действителен	Запросить новый код подтверждения
Изменения сохранены	Данные профиля успешно сохранены	Продолжить работу с программой
Ошибка загрузки профиля	Не удалось загрузить данные профиля	Повторить попытку позже
Дата рождения не может быть в будущем	Введена некорректная дата рождения	Указать корректную дату рождения
Не удалось сохранить данные.	Произошла ошибка при сохранении профиля	Проверить данные и повторить сохранение
Сессия истекла. Пожалуйста, войдите снова.	Истёк срок действия авторизации	Повторно войти в систему
Пароль изменён.	Пароль пользователя успешно изменён	Продолжить работу с программой
Укажите текущий пароль.	Не введён текущий пароль	Ввести текущий пароль
Новый пароль должен быть не короче 6 символов.	Новый пароль слишком короткий	Ввести пароль длиной не менее 6 символов
Новые пароли не совпадают.	Подтверждение нового пароля не совпадает	Повторно ввести новый пароль и подтверждение
Текущий пароль указан неверно.	Введён неправильный текущий пароль	Проверить текущий пароль и повторить попытку
Не удалось изменить пароль.	Произошла ошибка при смене пароля	Повторить попытку или обратиться к администратору

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана информационная система, предназначенная для автоматизации процессов подбора воздушных судов, оформления заявок на чартерные авиаперевозки и сопровождения вылетов. В процессе работы были изучены особенности предметной области, определены требования к программному продукту, описаны входные и выходные данные, спроектированы бизнес-процессы и структура базы данных.

Практическим результатом стало создание web-приложения, включающего клиентскую и серверную части. Клиентская часть разработана с использованием React, TypeScript и Vite, серверная часть реализована на C# с применением ASP.NET Core и Entity Framework Core, а для хранения данных используется MySQL. В системе реализованы регистрация и авторизация пользователей, работа с личным кабинетом, каталог воздушных судов, создание заявок, поиск аэропортов, расчёт параметров маршрута, управление вылетами, пассажирами, сотрудниками авиакомпании, самолётами и уведомлениями.

Разработанная база данных обеспечивает хранение и обработку сведений об авиакомпаниях, аэропортах, воздушных судах, пользователях, пассажирах, маршрутах, статусах вылетов и связанных документах. Реализованное решение позволяет упорядочить процесс обработки заявок, сократить количество ручных операций и предоставить пользователям единый интерфейс для работы с данными.

В ходе выполнения экспериментального раздела было проведено тестирование основных функций программного продукта: регистрации и входа в систему, создания заявки, выбора маршрута, расчёта параметров рейса, изменения статусов вылетов, работы с каталогом и административными разделами. По результатам тестирования были устранены выявленные

недостатки, повышена корректность обработки входных данных и проверена стабильность работы основных сценариев.

Таким образом, цель выпускной квалификационной работы достигнута, а поставленные задачи выполнены в полном объёме. Разработанная информационная система имеет практическую значимость, так как может применяться для автоматизации учёта и сопровождения чартерных авиаперевозок. Перспективами дальнейшего развития являются расширение аналитических возможностей, совершенствование алгоритмов расчёта стоимости и маршрута, развитие средств формирования документов, интеграция с внешними сервисами и добавление более гибких инструментов управления рейсами.

Приложение А

Диаграмма вариантов использования

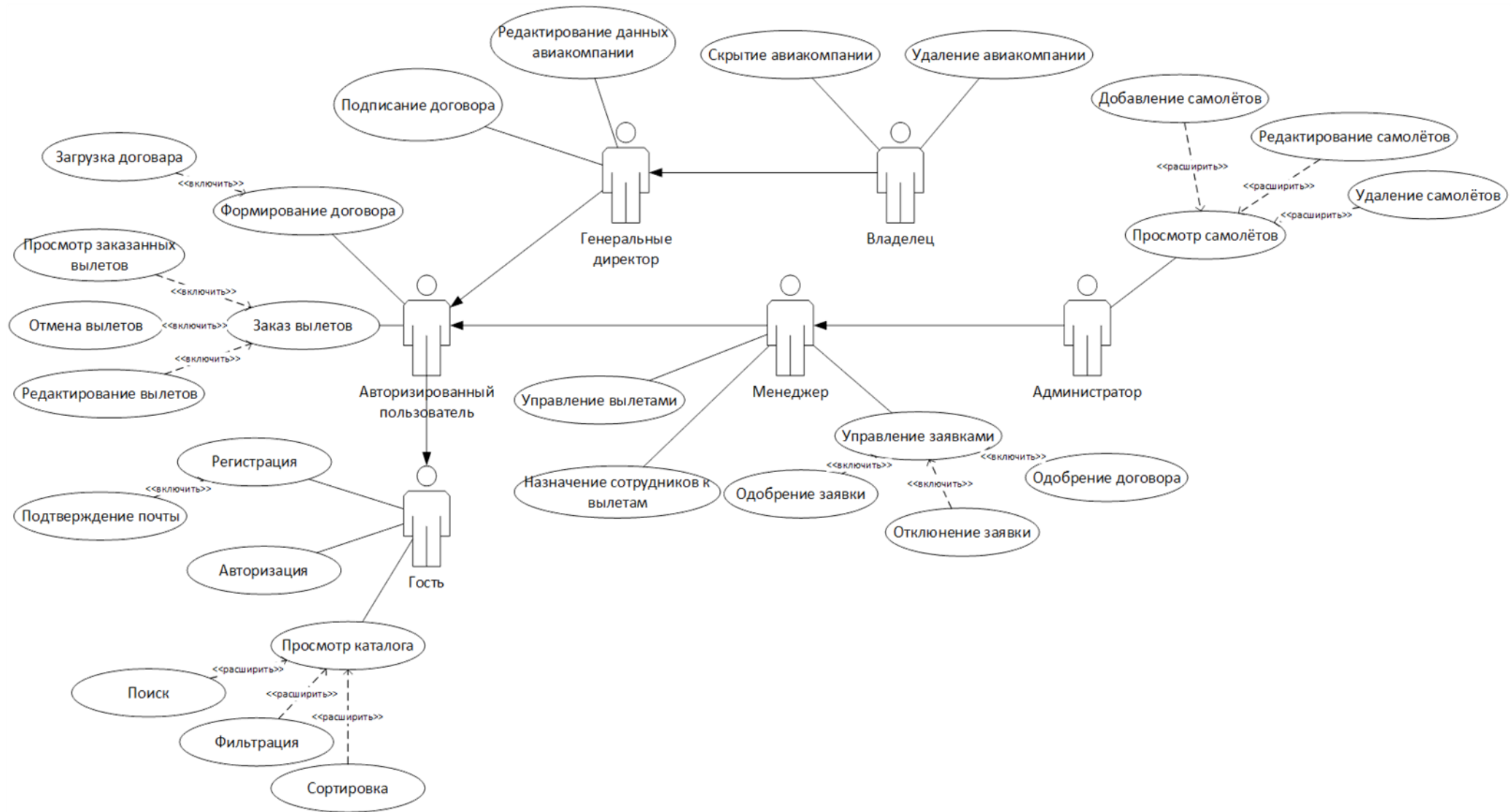


Рисунок А.1 – Диаграмма вариантов использования

Приложение Б

Шаблон выходных документов

На рисунках Б.1 – Б.4 представлен шаблон договора на предоставление услуг.

Является публичной офертой	
ДОГОВОР на предоставление услуг по организации чартерной воздушной перевозки № _____	
г. Москва	« ____ » _____ 202_ года
{ Полное наименование организации }, в дальнейшем именуемое "Исполнитель", в лице генерального директора { ф.и.о. ген. дир. }, действующего на основании Устава, с одной стороны и { ф.и.о. } далее именуемый «Заказчик», с другой стороны, вместе именуемые "Стороны", заключили настоящий Договор о нижеследующем.	
1. ПРЕДМЕТ ДОГОВОРА	
1.1.	Исполнитель предоставляет Заказчику услуги по организации чартерных воздушных перевозок, а также дополнительные услуги, связанные с их организацией, по заявкам Заказчика.
1.2.	Вся деятельность Исполнителя, связанная с организацией выполнения чартерной воздушной перевозки, ее маршрута, графика движения, типом и компоновкой воздушного судна, а также с иными услугами и уточнениями, осуществляется на основании Заказа Заказчика, являющегося неотъемлемой частью настоящего Договора. Правила оформления Заказа указаны в ст. 7 настоящего Договора.
2. ПРАВА И ОБЯЗАННОСТИ СТОРОН	
2.1. Обязанности и права Исполнителя	
2.1.1.	На основании Заказа Заказчика Исполнитель, по мере необходимости, привлекая третьи лица, организует услуги: Фрахтование воздушного судна соответствующего типа и компоновки, в надлежащем состоянии для осуществления полета, с экипажем, состав и квалификация которого отвечает всем необходимым правилам и требованиям, согласно дате выполнения перевозки, маршрутом, графиком движения; Дополнительные услуги.
2.1.2.	При невозможности предоставить указанное в Заявке воздушное судно, предварительно уведомив Заказчика, Исполнитель имеет право произвести замену воздушного судна на аналогичное ему по комфорту, без изменения условий настоящего Договора.
2.1.3.	Исполнитель имеет право отказать в перевозке: В целях обеспечения безопасности полета, безопасности пассажиров; В целях предотвращения нарушений соответствующих законов, постановлений, правил и предписаний государственных органов; В случае отсутствия и/или неправильно оформленных документов у пассажиров на въезд в страну по маршруту выполнения перевозки; Вследствие отказа пассажира выполнять правила и инструкции Исполнителя и Командира воздушного судна; Вследствие существенных изменений Заказчиком параметров его Заявки, если внесенные изменения, не позволяют обеспечить организацию рейса, в срок до даты выполнения перевозки.
2.1.4.	Командир воздушного судна осуществляет полный контроль над выполнением перевозки и управлением воздушным судном. Его решения по вопросам обеспечения безопасности полета являются окончательным и обязательным для Заказчика.
2.2. Обязанности и права Заказчика	
2.2.1.	Заказчик оплачивает услуги Исполнителя полностью и точно в указанные в Договоре сроки.
2.2.2.	Заказчик не будет передавать свои права и обязанности по настоящему Договору третьим лицам.
2.2.3.	Заказчик обязуется выполнять все требования и распоряжения командира воздушного судна, а так же правила и инструкции Исполнителя.
2.2.4.	Заказчик обязуется аккуратно обращаться с предоставленным ему имуществом воздушного судна. В случае порчи данного имущества Заказчиком, Заказчик возместит Исполнителю причиненные убытки.
2.2.5.	Заказчик обеспечит прибытие пассажиров в аэропорт отправления не позднее, чем за 2 часа до запланированного времени вылета, а так же и обеспечит наличие у пассажиров всех необходимых документов для выполнения перевозки.
2.2.6.	Заказчик информирует пассажиров об условиях перевозки, правилах пребывания и видах услуг на борту, а так же требованиями законодательства РФ, распространяющихся на виды обслуживания согласно настоящему Договору.
2.2.7.	Заказчик предоставляет Исполнителю не позднее, чем за 2 рабочих дня до вылета рейса данные о пассажирах.
3. РАСЧЕТЫ	
3.1.	Стоимость услуг Исполнителя и сроки платежей указаны в Приложениях / Заказах к настоящему Договору.
3.2.	Стоимость услуг Исполнителя относительно каждой заявки Заказчика рассчитывается в зависимости от сроков подачи заявки, маршрута, типа воздушного судна, категорий дополнительных услуг.
3.3.	В случае увеличения фактической стоимости услуг (увеличение цен на авиатопливо, борт. питание, сборов за аэропортовые услуги, обработка антиобледенительной жидкостью, аэронавигационных сборов, налогов и сборов), Заказчик оплачивает дополнительные расходы не позднее 6 (шести) банковских дней с момента выставления счета Исполнителем.
3.4.	Все дополнительные расходы Исполнителя, не относящиеся к организации чартерной воздушной перевозки и дополнительным услугам в соответствии с Заказом, которые могут возникнуть по вине или по инициативе Заказчика, несет Заказчик.
Исполнитель _____	Заказчик _____
	Образец

Рисунок Б.1 – Шаблон договора на предоставление услуг (страница 1)

Продолжение приложения Б

Является публичной офертой

Дополнительно, в случае просрочки платежа, согласно соответствующему Приложению, Заказчик уплачивает пени в размере 0,1% за каждый день просрочки от не перечисленной суммы, но не более 10% от первоначальной общей стоимости услуг по организации воздушной перевозки и дополнительных услуг.

4. ОТВЕТСТВЕННОСТЬ СТОРОН

- 4.1. Исполнитель и Заказчик не несут друг перед другом ответственности, если они не смогут выполнить взятые по Договору обязательства в результате действия непреодолимой силы (форс-мажор), как-то: метеоусловия, правительственные акты, забастовки или какие-либо другие факты, выходящие из-под контроля Исполнителя и Заказчика.
- 4.2. В случае наступления форс-мажорных обстоятельств, расходы и издержки Стороны несут самостоятельно.
- 4.3. Заказчик несет ответственность за все последствия, связанные с неправильным оформлением документов пассажиров и возмещит Исполнителю понесенные затраты, причиненные данными последствиями.
- 4.4. Стороны настоящим подтверждают, что Исполнитель не несет ответственности за любые задержки оказания услуг, вызванные действиями/бездействиями оператора, перевозчика, поставщиков услуг, составляющих услуги по настоящему Договору, и/или компетентных органов, при условии что такая ситуация находится вне разумного контроля Исполнителя и исключает его вину.
- 4.5. В случае нарушения условий Заказа непосредственно третьими лицами (Оператором, фактическим Перевозчиком, поставщиками дополнительных услуг по заявке Заказчика) и не по вине Исполнителя, Исполнитель приложит все разумные усилия, в меру возможности, для урегулирования в интересах Заказчика вопросов возмещения причиненных Заказчику убытков (при их наличии) непосредственно от фактического перевозчика и/или поставщиков дополнительных услуг, тем не менее все необходимые связанные с таким урегулированием расходы, в том числе на юридические услуги (при необходимости), несет Заказчик.
- 4.6. Исполнитель ни при каких обстоятельствах не несет ответственность за любые косвенные, не прямые, случайные убытки, ущерб, затраты или расходы, а также упущенную выгоду (включая, но не ограничиваясь, убытки производства, потери или искажение данных, потерю прибыли, контрактов или бизнеса, потерю деловой или обычной репутации).
- 4.7. При отказе Заказчика от услуг по Заявке Заказчик возмещает Исполнителю все фактически понесенные им расходы в связи с исполнением обязательств по Договору (соответствующей Заявке) до момента отказа Заказчика, в том числе (при наличии) фактически предъявленные Исполнителю в связи с отказом Заказчика от услуг по Заявке суммы неустоек, пени, штрафов от поставщиков соответствующих услуг, необходимых для выполнения Исполнителем своих обязательств по Договору.
- 4.8. В случае изменения условий выполнения рейса не по вине Исполнителя (в силу метеоусловий, неисправности судна, непредоставления/несвоевременного предоставления разрешений авиационными властями/структурами и т.п.), в частности, изменение/увеличение количества посадок воздушного судна, и/или длительности рейса более чем на 1 час, стоимость услуг по соответствующей Заявке может быть увеличена по требованию Исполнителя на сумму соответствующих фактических расходов, связанных с такими изменениями рейса, подлежащих возмещению Заказчиком.
- 4.9. Ответственность Сторон за неисполнение своих обязательств по настоящему Договору, в том числе и за отказ от перевозки в соответствии с Заказом, устанавливается индивидуально и закрепляется отдельным пунктом в конкретном Заказе.

5. СРОК И ПОРЯДОК ДЕЙСТВИЯ ДОГОВОРА

- 5.1. Настоящий Договор вступает в силу с момента подписания его обеими Сторонами и действует по «{ число }» { месяц } { год} года либо до полного исполнения Сторонами принятых на себя обязательств по настоящему Договору.
- 5.2. Если за 30 дней до истечения срока действия Договора ни одна из Сторон не заявит о его прекращении, Договор считается возобновленным на тех же условиях, на тот же срок. В дальнейшем действие настоящего договора автоматически продлевается на каждый последующий год, если ни одна из Сторон не заявит о его расторжении за 30 (тридцать) календарных дней до окончания срока действия.
- 5.3. В случае, если один или несколько пунктов Договора окажутся незаконными или неосуществимыми, остальные пункты сохраняют законную силу и обязательность их выполнения.
- 5.4. Настоящий Договор составлен в двух идентичных экземплярах, имеющих одинаковую юридическую силу, по одному экземпляру для каждой из Сторон.
- 5.5. Любые изменения и дополнения к настоящему Договору действительны при условии, если составлены в письменной форме и подписаны уполномоченными представителями Сторон.
- 5.6. Стороны признают, что любая без исключения переписка, корреспонденция по настоящему Договору, отправленные по факсу, почте или электронным средствам связи: по адресам электронной почты, указанным в настоящем Договоре, с использованием смс сообщений по номерам мобильных телефонов, указанным в настоящем Договоре, с использованием мессенджера WhatsApp (мобильного приложения для обмена сообщениями и аудио-видеофайлами) с аккаунтов, которые созданы по номерам телефонов, указанным в настоящем Договоре, является исходящей от надлежащим образом уполномоченных представителей сторон и имеет обязательную для обеих сторон юридическую силу. Стороны обязуются незамедлительно сообщать друг другу обо всех случаях несанкционированного доступа к их электронным ящикам. Исполнение, произведенное стороной договора в отсутствие у нее такого уведомления, признается надлежащим и лишает вторую сторону права ссылаться на указанные обстоятельства.
- 5.7. Во всем остальном, что не предусмотрено настоящим Договором, Стороны руководствуются действующим законодательством Российской Федерации, международными Конвенциями и Соглашениями, участником которых является РФ. При коллизии права, закрепляется приоритет международных норм.

6. ПОРЯДОК РАСТОРЖЕНИЯ ДОГОВОРА

Исполнитель _____

Заказчик _____

Образец

Рисунок Б.2 – Шаблон договора на предоставление услуг (страница 2)

Продолжение приложения Б

Является публичной офертой

- 6.1. Настоящий Договор может быть расторгнут одной Стороной в одностороннем порядке при условии письменного уведомления другой Стороны не менее чем за 30 (тридцать) дней.
- 6.2. При расторжении настоящего Договора Стороны составляют акт о взаиморасчетах. Настоящий Договор считается расторгнутым после полного исполнения всех обязательств по Договору между Сторонами.

7. ЗАКАЗ и ЗАЯВКА

- 7.1. Заказчик направляет Исполнителю предварительную Заявку в произвольной форме на организацию чартерной воздушной перевозки и дополнительные услуги посредством телефонной, факсимильной или электронным средствам связи (п. 5.6. Договора).
- 7.2. Исполнитель принимает предварительную Заявку к обработке. Исполнитель направляет Заказчику посредством телефонной, факсимильной или электронной связи Предложение с указанием стоимости и условий организации перевозки и дополнительных услуг.
- 7.3. При достижении соглашения по стоимости и условиям, Стороны подписывают документ, являющийся Заказом на организацию чартерной воздушной перевозки и дополнительные услуги. Условия настоящего Договора влекут за собой обязательства Сторон, только при условии документального оформления и подписания Сторонами формы конкретного Заказа.

8. РАССМОТРЕНИЕ СПОРОВ

- 8.1. Стороны будут стремиться решать споры и разногласия, возникшие в процессе исполнения настоящего Договора, путем переговоров.
- 8.2. Споры и разногласия, не разрешенные в результате переговоров, подлежат рассмотрению в Арбитражном суде города Москвы.

9. АДРЕСА, БАНКОВСКИЕ РЕКВИЗИТЫ, ПОДПИСИ СТОРОН

ИСПОЛНИТЕЛЬ { Полное наименование организации }	ЗАКАЗЧИК { фιο }
Юридический адрес: { Юридический адрес } Почтовый адрес: { Почтовый адрес } ИНН: { ИНН } КПП: { КПП } ОГРН: { ОГРН } р/с { р/с } в { Название банка } к/с { к/с } БИК { БИК } Адрес эл. почты: { email } Номер/ра телефона/ов для смс сообщений: { Номер телефона }	Адрес прописки: { Адрес прописки } Фактический адрес: { Фактический адрес } ИНН Банковские реквизиты: Банк { Название банка } БИК { БИК } Адрес эл. почты: { email } Номер/ра телефона/ов для смс сообщений: { Номер телефона }

Исполнитель
{ Полное наименование организации }

Заказчик
{ фιο }

_____/ { фιο ген. дир. }/
М.П.

_____/ { Инициалы }/
М.П.

ПРИЛОЖЕНИЕ к ДОГОВОРУ на предоставление услуг
по организации чартерной воздушной перевозки № { Номер приложения к договору } от «{ день }» { месяц } { год }г.

Исполнитель _____

Заказчик _____

Образец

Рисунок Б.3 – Шаблон договора на предоставление услуг (страница 3)

Продолжение приложения Б

Является публичной офертой

ЗАКАЗ № { Номер заказа }
на предоставление услуг
по организации чартерной воздушной перевозки

г. Москва
«{ день }» { месяц } { год } года

{ Полное наименование организации }, в дальнейшем именуемое "Исполнитель", в лице генерального директора { фио }, действующего на основании Устава, с одной стороны и { фио } далее именуемое «Заказчик», с другой стороны, вместе именуемые "Стороны", заключили настоящий Договор о нижеследующем.

Воздушное судно: { Номер воздушного судна }

1. **Маршрут и график движения** (время местное)

Кол-во Пассажиров	A/n отправления / A/n назначения	Дата вылета	Время вылета	Время прилета	Время в полете

Заказчик обеспечит прибытие пассажиров, груза и багажа в аэропорт отправления за 1 часа до вылета рейса.

2. **Бортпитание пассажиров – VIP**
3. **Выполнение перевозки зависит от разрешений на пролет территории и посадку по маршруту полета от местных властей, слотов в а/п, и метеоусловий.**
4. **Курение на борту воздушного судна: запрещается**
5. **Штраф за простой ВС: 1% от стоимости рейса, при задержке свыше двух часов по вине Заказчика**
6. **Ответственность сторон (Исполнителя и Заказчика) (размер и порядок выплаты неустойки) согласно п. 4.5. Договора:**
 - 25% от общей стоимости услуг при отмене с момента подписания настоящего Приложения, но более чем за три дня до даты вылета (совершения рейса);
 - 50% от общей стоимости услуг при отмене за 3 дня до даты вылета (совершения рейса);
 - 100% от общей стоимости услуг при отмене за сутки до вылета воздушного судна из аэропорта отправления.

Стоимость услуг и условия платежа
 Стоимость составляет – { стоимость услуг } ({ стоимость услуг текстом }), Стоимость услуг не облагается НДС на основании ст.346.11 главы 26.2 НК РФ.
 Оплата производится в рублях Российской Федерации, по курсу ЦБ на день оплаты, путём их перечисления на расчетный счет Исполнителя до «{ дата }» { месяц } { год }г.

В стоимость услуг включены: аренда самолета с экипажем, сборы за взлет-посадку, сборы за аэропортовые услуги, АНО Обслуживание на маршруте и в районе аэродрома, расходы по заправке ВС топливом на полет, сборы за обслуживание пассажиров в залах прилета и вылета, борт. Питание класса { класс питания }, обязательное страхование пассажиров, сборы за получение разрешений на выполнение рейса.

В стоимость услуг не включено:

- а) обработка самолета антиобледенительной жидкостью.
- б) любые другие дополнительные расходы за услуги, не указанные в Заказе.
- в) изменение даты вылета.
- г) изменение количества пассажиров.

изменение маршрута, любые дополнительные посадки в том числе, связанные с уходом самолета на запасной аэродром по причине плохой погоды.

- г) продление регламента аэропортов.

Исполнитель

{ Полное наименование организации }

_____ / { фио ген. дир. } /

М.П.

Исполнитель _____

Заказчик

{ фио }

_____ / { фио } /

М.П.

Заказчик _____

Образец

Рисунок Б.4 – Шаблон договора на предоставление услуг (страница 4)

На рисунке Б.5 представлено сообщение с кодом подтверждения почты.

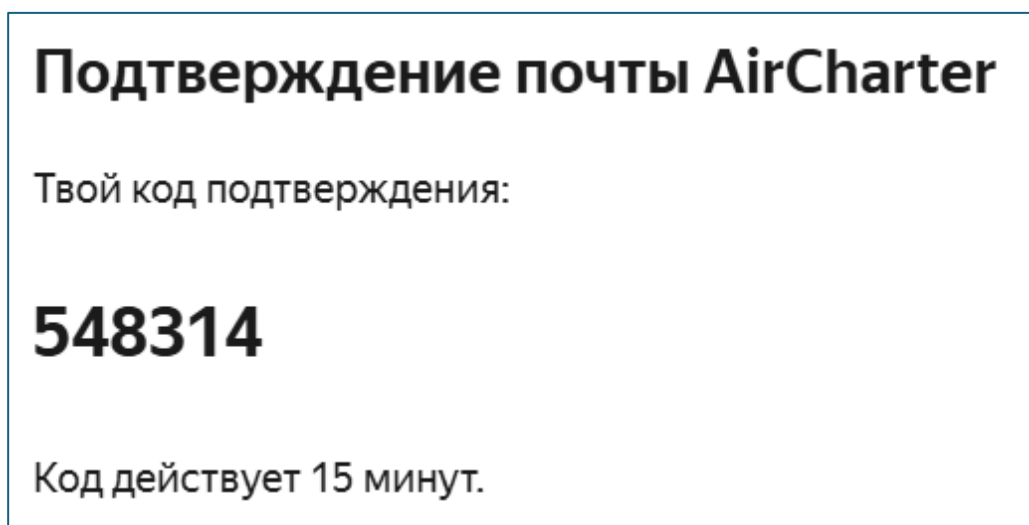


Рисунок Б.5 – Сообщение с кодом подтверждения почты

На рисунке Б.6 представлено информирующее сообщение о регистрации рабочего аккаунта с данными для входа.

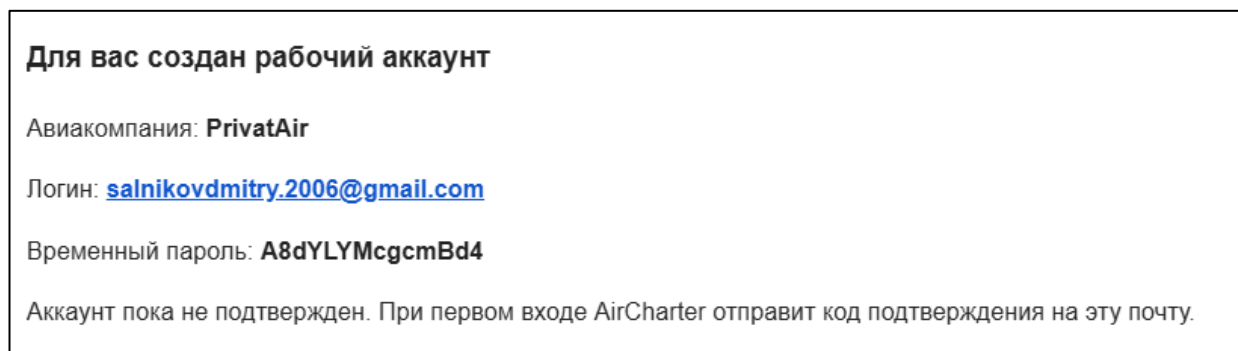


Рисунок Б.6 – Информационное сообщение о регистрации рабочего аккаунта с данными для входа

Приложение В

ER-диаграмма

На рисунке В.1 представлена ER-диаграмма.

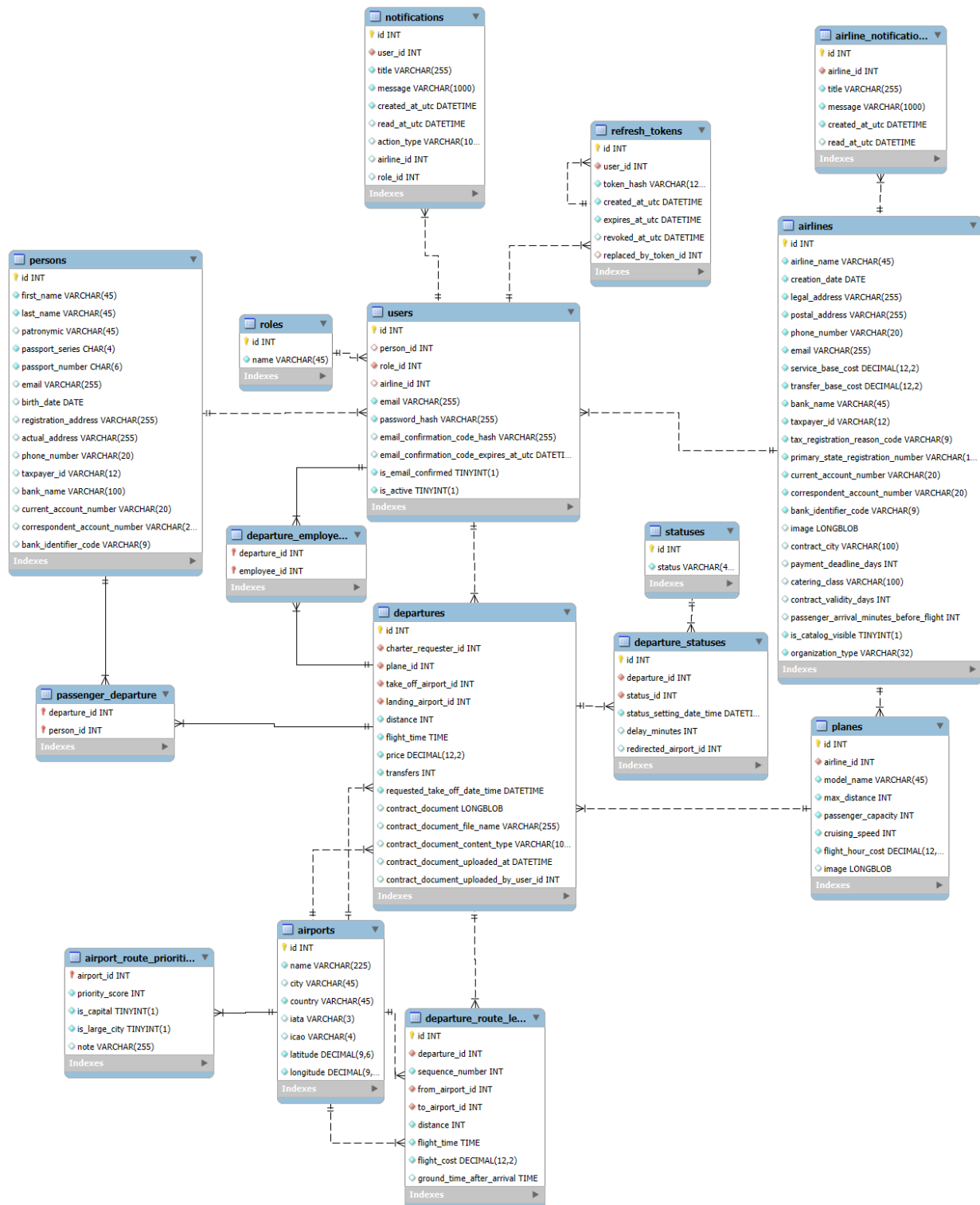


Рисунок В.1 – ER-диаграмма БД

Приложение Г

Контрольный пример

Таблица Г.1 – Контрольный пример «Роли»

Идентификатор роли	Название роли
1	Client
2	Owner
3	Manager
4	Admin
5	GeneralDirector

Таблица Г.2 – Контрольный пример «Физические лица»

Идентификатор пассажира	Фамилия	Имя	Отчество	Серия паспорта	Номер паспорта	Почта для уведомлений	Дата рождения
1	Алексей	Иванов	Дмитриевич	7485	123123	aleksey.ivanov@example.com	1990-04-12
2	Мария	Смирнова	Валерьевна	8596	456456	maria.smirnova@example.com	1993-09-25
3	Олег	Козлов	Игоревич	4152	789789	oleg.kozlov@example.com	1988-02-14
4	Елена	Фролова	Александровна	5263	963741	elena.frolova@example.com	1992-07-08

Таблица Г.3 – Контрольный пример «Пользователи»

Идентификатор пользователя	Идентификатор пассажира	Идентификатор роли	Идентификатор авиакомпании	Электронная почта	Пароль	Код подтверждения почты	Время истечения кода подтверждения почты	Статус подтверждения почты	Статус активности пользователя
1	16	1		bergen@gmail.com				1	1
2	21	2	2	PrivatAir@gmail.com				1	1
3	1	2	5	CityJet@yandex.ru				1	1
5	13	1		Salnikovdmitry.2006@yandex.ru				1	1
6	14	1		erik.fayzullin.2007@mail.ru				1	1

Продолжение приложения Г

Таблица Г.4 – Контрольный пример «Самолёты»

Идентификатор самолёта	Идентификатор авиакомпании	Название модели	Максимальная дальность полёта	Количество мест	Крейсерская скорость	Стоимость часа перелёта	Фотография самолёта
1	5	Embraer EMB 120	2907	30	504	206250.00	
2	2	Cessna Citation XLS+	3441	9	815	322500.00	
3	2	Cessna Citation Latitude	5000	9	826	435000.00	
4	2	Bombardier Challenger 350	5926	10	870	446250.00	
5	2	Bombardier Global 6000	11112	13	904	825000.00	

Таблица Г.5 – Контрольный пример «Аэропорты»

Идентификатор аэропорта	Название	Город	Страна	IATA	ICAO	Географическая широта	Географическая долгота
1	Аэропорт Горока	Горока	Папуа Новая Гвинея	GKA	AYGA	-6.081690	145.391998
2	Маданский аэропорт	Мадан	Папуа Новая Гвинея	MAG	AYMD	-5.207080	145.789001
3	Маунт Хаген Кагамуга Аэропорт	Гора Хаген	Папуа Новая Гвинея	HGU	AYMH	-5.826790	144.296005
4	Надзаб аэропорт	Надзаб	Папуа Новая Гвинея	LAE	AYNZ	-6.569803	146.725977
5	Порт -Морсби Джексонс международный аэропорт	Порт Морсби	Папуа Новая Гвинея	POM	AYPY	-9.443380	147.220001

Продолжение приложения Г

Таблица Г.6 – Контрольный пример «Вылеты»

Идентификатор вылета	Идентификатор заказчика чартерного рейса	Идентификатор самолёта	Аэропорт вылета	Аэропорт посадки	Расстояние полёта	Время полёта	Цена	Количество пересадок	Запрошенное число и время вылета	Документ договора	Имя файла договора	Формат файла договора	Дата и время загрузки договора	Пользователь, загрузивший договор
17	9	1	2835	2258	6495	12:53:13	2907924.11	2	2025-06-25 14:00:56	byte[]	dogovor_17.pdf	application/pdf	2025-06-24 12:30:00	2
18	2	7	2835	784	10103	11:10:33	10946487.83	0	2025-06-25 03:13:57	byte[]	dogovor_18.pdf	application/pdf	2025-06-24 13:10:00	2
19	6	7	2258	2835	6495	07:11:05	7055116.15	0	2025-07-22 06:00:48	byte[]	dogovor_19.pdf	application/pdf	2026-04-13 16:45:00	2

Таблица Г.7 – Контрольный пример «Статусы»

Идентификатор статуса	Наименование статуса
1	В создании
2	Ожидает одобрения
3	Планируется
4	Запланирован

Продолжение приложения Г

Таблица Г.8 – Контрольный пример «Статусы вылетов»

Идентификатор записи	Идентификатор вылета	Идентификатор статуса	Дата и время установки статуса
38	17	1	2025-06-24 01:56:18
39	17	2	2025-06-24 02:12:55
40	18	1	2025-06-24 03:15:34
41	19	1	2025-06-24 04:44:48
42	19	2	2025-06-24 05:04:40

Таблица Г.9 – Контрольный пример «Пассажиры вылетов»

Идентификатор вылета	Идентификатор пассажира
17	14
19	14
19	15
17	19
22	19

Таблица Г.10 – Контрольный пример «Сотрудники вылетов»

Идентификатор вылета	Идентификатор сотрудника
22	11
22	12
22	13
22	14
22	15

Продолжение приложения Г

Таблица Г.11 – Контрольный пример «Авиакомпаний»

Но мер а ви а ко м п а н и и	Наим ено ва ние а ви а к о м п а н и и	Дат а ре ги с т ра ц и и	Полное наимено вание о рг а н и з а ц и и	Сокращ ённое наимено вание о рг а н и з а ц и и	Юр. адрес	Почтов ый адрес	Ном ер те ле фо на	Эл. поч та	Стои мость обслу жива ния	Стои мость перес адки	Назва ние банка	И Н Н	К П П	О Р Г Н	Расчёт ный счёт	Кор рес пон ден тск ий счёт	БИ К	Лог оти п а ви а ко м п а н и и
1	2	3	4	5	6	7	8	9			10	11	12	13	14	15	16	17
17	No Name Jet	202 5- 06- 24	Общест во с огранич енной ответств енность ю «No Name Jet»	ООО «No Name Jet»	450077, Республ ика Башкорт остан, г. Уфа, ул. Ленина, д. 70, офис 15	450077, Респуб лика Башкор тостан, г. Уфа, ул. Ленина , д. 70, офис 15	+7 347 246- 92- 18	offic e@n ona mej et.ru	5000 0.00	1000 00.00	ПАО «Сбе рбанк »	02 76 98 34 12	02 76 01 00 1	12 50 20 00 34 12 7	407028 100620 000145 28	301 018 103 000 000 006 01	048 073 601	byt e[]
17	No Nam e Jet	202 5- 06- 24	Общест во с огранич енной ответств енность ю «No Name Jet»	ООО «No Name Jet»	450077, Республ ика Башкорт остан, г. Уфа, ул. Ленина, д. 70, офис 15	450077, Респуб лика Башкор тостан, г. Уфа, ул. Ленина , д. 70, офис 15	+7 347 246- 92- 18	offic e@n ona mej et.ru	5000 0.00	1000 00.00	ПАО «Сбе рбанк »	02 76 98 34 12	02 76 01 00 1	12 50 20 00 34 12 7	407028 100620 000145 28	301 018 103 000 000 006 01	048 073 601	byt e[]

Продолжение приложения Г

Продолжение таблицы Г.11

1	2	3	4	5	6	7	8	9			10	11	12	13	14	15	16	17
17	No Name Jet	202 5- 06- 24	Общес во с огранич енной ответств енность ю «No Name Jet»	ООО «No Name Jet»	450077, Республ ика Башкор тостан, г. Уфа, ул. Ленина, д. 70, офис 15	450077 , Респуб лика Башко ртоста н, г. Уфа, ул. Ленин а, д. 70, офис 15	+7 347 246- 92-18	offic e@n ona mej et.ru	5000 0.00	1000 00.00	ПАО «Сбе рбанк »	02 76 98 34 12	02 76 01 00 1	12 50 20 00 34 12 7	40702 81006 20000 14528	301 018 103 000 000 006 01	048 073 601	byt e[]

Таблица Г.12 – Контрольный пример «Приоритеты аэропортов»

Идентификатор аэропорта	Приоритет аэропорта при построении маршрута	Признак столицы	Признак крупного города	Примечание
1	10	0	1	Крупный региональный аэропорт
2	5	0	0	Региональный аэропорт
3	5	0	0	Региональный аэропорт
4	7	0	1	Аэропорт крупного города
5	10	1	1	Столичный аэропорт

Продолжение приложения Г

Таблица Г.13 – Контрольный пример «Участки маршрута вылета»

Идентификатор участка маршрута	Идентификатор вылета	Порядковый номер участка маршрута	Аэропорт отправления	Аэропорт прибытия	Расстояние участка маршрута	Время полёта по участку маршрута	Стоимость полёта по участку маршрута	Время стоянки после прибытия
1	17	1	2835	2828	1176	01:20:33	526080.00	00:50:00
2	17	2	2828	784	2400	04:45:00	1074000.00	01:00:00
3	17	3	784	2258	2919	06:47:40	1306844.11	
4	18	1	2835	784	10103	11:10:33	10946487.83	
5	22	1	2835	2828	1176	01:20:33	2169417.81	

Таблица Г.14 – Контрольный пример «Токены обновления»

Идентификатор токена	Идентификатор пользователя	Хэш токена	Дата и время создания токена	Дата и время истечения токена	Дата и время отзыва токена	Токен, которым был заменен текущий токен
1	1	refresh_token_hash_001	2025-06-24 10:00:00	2025-07-24 10:00:00		
2	2	refresh_token_hash_002	2025-06-24 11:00:00	2025-07-24 11:00:00		
3	3	refresh_token_hash_003	2025-06-24 12:00:00	2025-07-24 12:00:00	2025-06-25 09:30:00	4
4	3	refresh_token_hash_004	2025-06-25 09:30:00	2025-07-25 09:30:00		
5	5	refresh_token_hash_005	2025-06-25 14:15:00	2025-07-25 14:15:00		

Продолжение приложения Г

Таблица Г.15 – Контрольный пример «Уведомления пользователей»

Идентификатор уведомления	Идентификатор пользователя	Заголовок	Текст уведомления	Тип действия	Идентификатор авиакомпании	Идентификатор роли	Дата и время создания	Дата и время прочтения
1	1	Статус вылета изменён	Статус вылета по заявке №1 изменён на «Ожидает подписания договора».	NULL	NULL	NULL	2026-05-20 10:15:00	NULL
2	1	Маршрут вылета изменён	По заявке №1 маршрут изменён: Москва — Санкт-Петербург.	NULL	NULL	NULL	2026-05-20 11:30:00	2026-05-20 12:00:00
3	1	Приглашение в авиакомпанию	Авиакомпания «AirCharter» приглашает вас в свою команду.	AirlineEmploymentInvite	1	4	2026-05-21 09:00:00	NULL
4	2	Приглашение в авиакомпанию	Авиакомпания «AirCharter» приглашает вас в свою команду.	AirlineEmploymentInvite	1	4	2026-05-21 09:30:00	NULL

Продолжение приложения Г

Таблица Г.16 – Контрольный пример «Уведомления авиакомпании»

Идентификатор уведомления	Идентификатор авиакомпании	Заголовок	Текст уведомления	Дата и время создания	Дата и время прочтения
1	2	Приглашение принято	Пользователь anna.smirnova.attendant@gmail.com принял приглашение и стал сотрудником авиакомпании.	2026-05-21 09:05:00	NULL
2	2	Сотрудник зарегистрирован	Создан рабочий аккаунт 03novart@gmail.com с должностью Сотрудник.	2026-05-22 14:20:00	NULL
3	2	Должность сотрудника изменена	Сотруднику ivan.petrov.admin@gmail.com изменили должность: Менеджер -> Администратор.	2026-05-23 16:40:00	NULL
4	2	Сотрудник уволился	Сотрудник dmitriy.kuznetsov.copilot@aircharter.local покинул авиакомпанию.	2026-05-23 17:40:00	NULL
5	2	Приглашение принято	Пользователь aleksey.sokolov.pilot@aircharter.local принял приглашение и стал сотрудником авиакомпании.	2026-05-23 18:15:00	NULL

Приложение Д

Модульная схема приложения

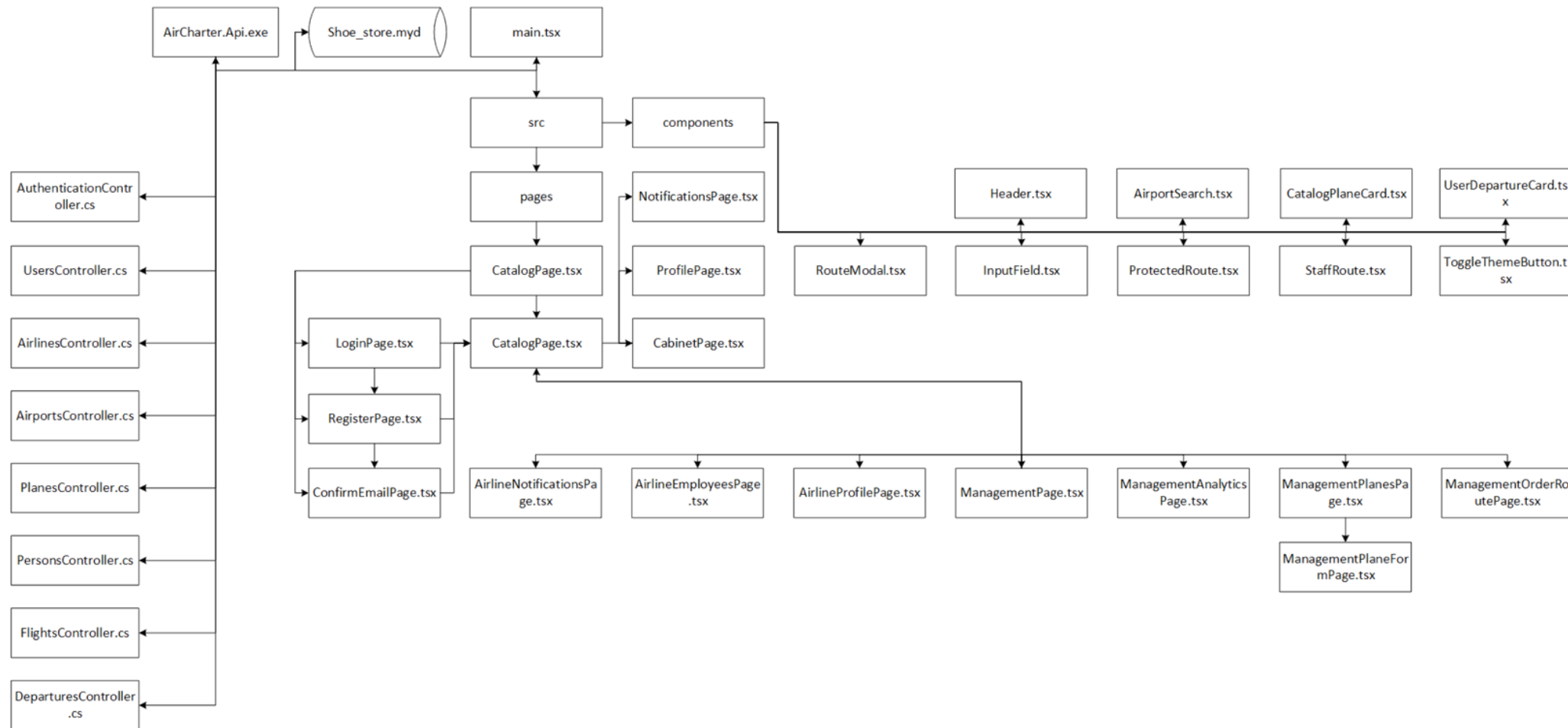


Рисунок Д.1 – Модульная схема приложения

Приложение Е

Код программы

```
Program.cs
// Точка входа backend: здесь подключаются сервисы, база данных, JWT-защита, CORS, Swagger и HTTP-
// конвейер приложения.
using System.Text;
using System.Text.Json.Serialization;
using AirCharter.API.Model;
using AirCharter.API.Services;
using AirCharter.API.Services.Documents;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using MigraDoc;
using MigraDoc.DocumentObjectModel;
using MigraDoc.Rendering;
using PdfSharp.Fonts;
internal class Program
{
    private static void Main(string[] args)
    {
        WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

        // PDF-библиотеки должны знать доступный системный шрифт, иначе русские документы могут собираться
        // с ошибками.
        GlobalFontSettings.UseWindowsFontsUnderWindows = true;
        PredefinedFontsAndChars.ErrorFontName = "Arial";

        builder.Services.AddControllers();
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();
        builder.Services.AddAuthorization();

        string secretKey = builder.Configuration["Jwt:SecretKey"]
            ?? throw new InvalidOperationException("JWT secret key is not configured.");

        string issuer = builder.Configuration["Jwt:Issuer"]
            ?? throw new InvalidOperationException("JWT issuer is not configured.");

        string audience = builder.Configuration["Jwt:Audience"]
            ?? throw new InvalidOperationException("JWT audience is not configured.");

        // Сериализатору разрешено игнорировать циклы EF Core, чтобы связанные сущности не ломали JSON-
        // ответы.
        builder.Services.AddControllers()
            .AddJsonOptions(options =>
            {
                options.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;
            });

        // JWT-настройки говорят API, какой токен считать подлинным и когда считать его просроченным.
        builder.Services
            .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
            .AddJwtBearer(options =>
            {
                options.TokenValidationParameters = new TokenValidationParameters
                {
                    ValidateIssuer = true,
                    ValidIssuer = issuer,
                }
            })
    }
}
```

Продолжение приложения E

```
        ValidateAudience = true,
        ValidAudience = audience,

        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(secretKey)
        ),

        ValidateLifetime = true,
        ClockSkew = TimeSpan.Zero
    };
});

// Swagger показывает схему API и позволяет тестировать защищенные методы с Bearer-токеном.
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new() { Title = "AssetStore API", Version = "v1" });

    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Введите JWT токен"
    });

    c.AddSecurityRequirement(securityRequirement: new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});

// CORS открыт только для локального Vite-фронтенда, который ходит в API с cookie и заголовками
авторизации.
builder.Services.AddCors(options =>
{
    options.AddPolicy("FrontendPolicy", policyBuilder =>
    {
        policyBuilder
            .WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
```

Продолжение приложения Е

```
builder.Services.AddDbContext<AirCharterExtendedContext>(options =>
    options.UseMySQL(connectionString, ServerVersion.AutoDetect(connectionString)));

builder.Services.AddScoped<JwtService>();
builder.Services.AddScoped<EmailService>();

builder.Services.AddScoped<AirportSearchService>();

// Старый расчет плеч оставлен как отдельный сервис, а новый планировщик маршрутов работает через граф
аэропортов.
builder.Services.AddScoped<FlightLegCalculationService>();

builder.Services.AddScoped<RoutePlanningService>();
builder.Services.AddSingleton<AirportGraphCache>();
builder.Services.AddSingleton<AirportTimeZoneService>();

builder.Services.AddScoped<DeparturePdfDataFactory>();
builder.Services.AddScoped<TicketPdfService>();
builder.Services.AddScoped<ContractPdfDataFactory>();
builder.Services.AddScoped<ContractPdfService>();
builder.Services.AddScoped<DatabaseCompatibilityService>();

WebApplication app = builder.Build();

// Перед стартом API проверяет совместимость схемы БД, чтобы старые данные не ломали новые запросы.
using (IServiceScope serviceScope = app.Services.CreateScope())
{
    DatabaseCompatibilityService databaseCompatibilityService =
        serviceScope.ServiceProvider.GetRequiredService<DatabaseCompatibilityService>();

    databaseCompatibilityService.EnsureAsync().GetAwaiter().GetResult();
}

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

// Последний рубеж обработки ошибок возвращает понятный текст вместо технического stack trace.
app.Use(async (context, next) =>
{
    try
    {
        await next();
    }
    catch (Exception exception)
    {
        app.Logger.LogError(exception, "Unhandled request exception.");

        if (context.Response.HasStarted)
            throw;

        context.Response.Clear();
        context.Response.StatusCode = StatusCodes.Status500InternalServerError;
        context.Response.ContentType = "text/plain; charset=utf-8";

        await context.Response.WriteAsync(
            "Что-то пошло не так. Попробуйте ещё раз чуть позже.");
    }
});
```

Продолжение приложения Е

```
});

app.UseHttpsRedirection();
app.UseCors("FrontendPolicy");
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
}
}
```

App.tsx

// Корневой компонент frontend: здесь описаны основные маршруты и страницы, между которыми переходит пользователь.

```
import { useEffect } from "react";
import { BrowserRouter, Route, Routes, useLocation, useNavigate } from "react-router-dom";
import { unauthorizedResponseEventName } from "../api/sendRequest";
import ProtectedRoute from "../components/ProtectedRoute";

import LoginPage from "../pages/auth/LoginPage";
import RegisterPage from "../pages/auth/RegisterPage";
import ConfirmEmailPage from "../pages/auth/ConfirmEmailPage";

import CatalogPage from "../pages/catalog/CatalogPage";
import CabinetPage from "../pages/cabinet/CabinetPage";
import CabinetDeparturePage from "../pages/cabinet/CabinetDeparturePage";
import ProfilePage from "../pages/profilePage/ProfilePage";
import AirlineProfilePage from "../pages/airlineProfile/AirlineProfilePage";
import RegisterAirlinePage from "../pages/airlineProfile/RegisterAirlinePage";
import OrderPage from "../pages/orderPage/OrderPage.tsx";
import StaffRoute from "../components/StaffRoute";
import ManagementPage from "../pages/management/ManagementPage";
import ManagementOrderRoutePage from "../pages/management/ManagementOrderRoutePage";
import ManagementPlanesPage from "../pages/management/ManagementPlanesPage";
import ManagementPlaneFormPage from "../pages/management/ManagementPlaneFormPage";
import ManagementAnalyticsPage from "../pages/management/ManagementAnalyticsPage";
import NotificationsPage from "../pages/notifications/NotificationsPage";
import AirlineNotificationsPage from "../pages/airlineNotifications/AirlineNotificationsPage";
import AirlineEmployeesPage from "../pages/airlineEmployees/AirlineEmployeesPage";
import { useUser } from "../context/UserContext";

export default function App() {
  return (
    <BrowserRouter>
      <UnauthorizedRedirect />
      <Routes>
        { /* Публичные страницы доступны без токена и нужны для входа, регистрации и просмотра каталога. */ }

        <Route path="/" element={<CatalogPage />} />
        <Route path="/catalog" element={<CatalogPage/>} />

        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route path="/confirm-email" element={<ConfirmEmailPage />} />

        { /* Личный кабинет и оформление заявки требуют авторизации обычного пользователя. */ }
        <Route path="/cabinet" element={<ProtectedRoute><CabinetPage/></ProtectedRoute>} />
      </Routes>
    </BrowserRouter>
  );
}
```


Продолжение приложения Е

```

    <Route path="/cabinet/departures/:departureId" element={<ProtectedRoute><CabinetDeparturePage
/></ProtectedRoute>} />
    <Route path="/profile" element={<ProtectedRoute><ProfilePage/></ProtectedRoute>} />
    <Route path="/notifications" element={<ProtectedRoute><NotificationsPage /></ProtectedRoute>} />
    <Route path="/airline-notifications" element={<ProtectedRoute><AirlineNotificationsPage
/></ProtectedRoute>} />
    <Route path="/airline-register" element={<ProtectedRoute><RegisterAirlinePage /></ProtectedRoute>} />
    <Route path="/airline-profile" element={<ProtectedRoute><AirlineProfilePage /></ProtectedRoute>} />
    <Route path="/airline-employees" element={<ProtectedRoute><AirlineEmployeesPage
/></ProtectedRoute>} />
    <Route path="/create-order" element={<ProtectedRoute><OrderPage/></ProtectedRoute>} />
    { /* Управленческий раздел дополнительно проверяет роль сотрудника через StaffRoute. */ }
    <Route element={<StaffRoute />>
      <Route path="/management" element={<ManagementPage />} />
      <Route path="/management/orders" element={<ManagementPage />} />
      <Route path="/management/orders/:departureId" element={<ManagementOrderRoutePage />} />
      <Route path="/management/flights" element={<ManagementPage />} />
      <Route path="/management/flights/:departureId" element={<ManagementOrderRoutePage />} />
      <Route path="/management/completed" element={<ManagementPage />} />
      <Route path="/management/completed/:departureId" element={<ManagementOrderRoutePage />} />
      <Route path="/management/planes" element={<ManagementPlanesPage />} />
      <Route path="/management/planes/new" element={<ManagementPlaneFormPage />} />
      <Route path="/management/planes/:planeId" element={<ManagementPlaneFormPage />} />
      <Route path="/management/analytics" element={<ManagementAnalyticsPage />} />
    </Route>
  </Routes>
</BrowserRouter>
);
}

function UnauthorizedRedirect() {
  const navigate = useNavigate();
  const location = useLocation();
  const { logout } = useUser();

  useEffect(() => {
    // sendRequest сообщает об истекшей сессии через событие, а этот компонент централизованно отправляет
    // пользователя на вход.
    function handleUnauthorizedResponse() {
      logout();

      if (location.pathname !== "/login") {
        navigate("/login", {
          replace: true,
          state: { from: location }
        });
      }
    }

    window.addEventListener(unauthorizedResponseEventName, handleUnauthorizedResponse);

    return () => {
      window.removeEventListener(unauthorizedResponseEventName, handleUnauthorizedResponse);
    };
  }, [location, logout, navigate]);

  return null;
}

```

Продолжение приложения Е

UserContext.tsx

// React-контекст UserContext хранит общее состояние приложения и передает его компонентам без ручной прокладки props.

```
import React, { createContext, useContext, useEffect, useState } from "react";
import { logout as logoutSession } from "../api/authService";
import { getCurrentUser } from "../api/userService";
import type { UserProfileResponse } from "../contracts/responses/users/userPersonResponse";
```

```
interface UserContextType {
  user: UserProfileResponse | null;
  isLoading: boolean;
  logout: () => void;
  refreshUser: () => Promise<void>;
}
```

```
const UserContext = createContext<UserContextType | undefined>(undefined);
```

```
export const UserProvider = ({ children }: { children: React.ReactNode }) => {
  const [user, setUser] = useState<UserProfileResponse | null>(null);
  const [isLoading, setIsLoading] = useState(true);
```

// Профиль запрашивается после загрузки приложения и повторно после входа, чтобы роли сразу обновились в интерфейсе.

```
const fetchUser = async () => {
  setIsLoading(true);
  try {
    const data = await getCurrentUser();
    setUser(data);
  } catch {
    setUser(null);
  } finally {
    setIsLoading(false);
  }
};
```

```
useEffect(() => {
  fetchUser();
}, []);
```

```
const logout = () => {
  // Backend очищает refresh-cookie, а frontend отдельно убирает access-токен из localStorage.
  void logoutSession().catch(() => undefined);
  localStorage.removeItem("accessToken");
  setUser(null);
};
```

```
return (
  <UserContext.Provider value={{ user, isLoading, logout, refreshUser: fetchUser }}>
    {children}
  </UserContext.Provider>
);
```

```
export const useUser = () => {
  const context = useContext(UserContext);
  if (context === undefined) {
    throw new Error("useUser must be used within a UserProvider");
  }
  return context;
};
```

Продолжение приложения Е

```
                                sendRequest.ts
// Общая функция отправки запросов скрывает повторяющуюся работу с fetch, JSON, токеном и ошибками.

import { createApiError } from "../utils/apiError";

const apiBaseUrl = "https://localhost:7219";
export const unauthorizedResponseEventName = "aircharter:unauthorized-response";

type AccessTokenResponse = {
  token: string;
};

let refreshAccessTokenRequest: Promise<string | null> | null = null;

export async function sendRequest<TResponse>(
  path: string,
  method: string,
  body?: unknown,
  signal?: AbortSignal
): Promise<TResponse> {
  const accessToken = localStorage.getItem("accessToken");
  const response = await sendFetchRequest(path, method, body, signal, accessToken);

  if (response.ok) {
    return await parseResponse<TResponse>(response);
  }

  // Если access-токен устарел, frontend пробует обновить его через refresh-cookie и повторить исходный запрос
  // один раз.
  if ((response.status === 401 || response.status === 403) && accessToken && !isAuthEndpoint(path)) {
    const refreshedAccessToken = await refreshAccessToken();

    if (refreshedAccessToken !== null) {
      const retryResponse = await sendFetchRequest(
        path,
        method,
        body,
        signal,
        refreshedAccessToken
      );

      if (retryResponse.ok) {
        return await parseResponse<TResponse>(retryResponse);
      }

      if (retryResponse.status === 401) {
        handleUnauthorizedResponse();
      }

      const retryResponseText = await retryResponse.text();
      throw createApiError(retryResponse.status, retryResponseText);
    }

    handleUnauthorizedResponse();
  }

  const responseText = await response.text();
  throw createApiError(response.status, responseText);
}
```

Продолжение приложения E

```
export async function sendBlobRequest(
  path: string,
  method: string,
  body?: unknown,
  signal?: AbortSignal
): Promise<Blob> {
  const accessToken = localStorage.getItem("accessToken");
  const response = await sendFetchRequest(path, method, body, signal, accessToken);

  if (response.ok) {
    return await response.blob();
  }

  if ((response.status === 401 || response.status === 403) && accessToken && !isAuthEndpoint(path)) {
    const refreshedAccessToken = await refreshAccessToken();

    if (refreshedAccessToken !== null) {
      const retryResponse = await sendFetchRequest(
        path,
        method,
        body,
        signal,
        refreshedAccessToken
      );

      if (retryResponse.ok) {
        return await retryResponse.blob();
      }

      if (retryResponse.status === 401) {
        handleUnauthorizedResponse();
      }

      const retryResponseText = await retryResponse.text();
      throw createApiError(retryResponse.status, retryResponseText);
    }

    handleUnauthorizedResponse();
  }

  const responseText = await response.text();
  throw createApiError(response.status, responseText);
}

export async function sendFormDataRequest<TResponse>(
  path: string,
  method: string,
  body: FormData,
  signal?: AbortSignal
): Promise<TResponse> {
  const accessToken = localStorage.getItem("accessToken");
  const response = await sendFetchFormDataRequest(path, method, body, signal, accessToken);

  if (response.ok) {
    return await parseResponse<TResponse>(response);
  }

  const responseText = await response.text();
  throw createApiError(response.status, responseText);
}
```

```

async function sendFetchRequest(
  path: string,
  method: string,
  body: unknown,
  signal: AbortSignal | undefined,
  accessToken: string | null
): Promise<Response> {
  const headers: Record<string, string> = {
    "Content-Type": "application/json"
  };

  if (accessToken && !isAuthEndpoint(path)) {
    headers["Authorization"] = `Bearer ${accessToken}`;
  }

  try {
    return await fetch(`${apiBaseUrl}${path}`, {
      method: method,
      headers: headers,
      body: body === undefined ? undefined : JSON.stringify(body),
      signal: signal,
      credentials: "include"
    });
  } catch {
    throw createApiError(0);
  }
}

async function sendFetchFormDataRequest(
  path: string,
  method: string,
  body: FormData,
  signal: AbortSignal | undefined,
  accessToken: string | null
): Promise<Response> {
  const headers: Record<string, string> = {};

  if (accessToken && !isAuthEndpoint(path)) {
    headers["Authorization"] = `Bearer ${accessToken}`;
  }

  try {
    return await fetch(`${apiBaseUrl}${path}`, {
      method: method,
      headers: headers,
      body: body,
      signal: signal,
      credentials: "include"
    });
  } catch {
    throw createApiError(0);
  }
}

async function parseResponse<TResponse>(response: Response): Promise<TResponse> {
  if (response.status === 204) {
    return undefined as TResponse;
  }
}

```

Продолжение приложения E

```
    try {
      return await response.json() as TResponse;
    } catch {
      throw createApiError(response.status);
    }
  }
}

async function refreshAccessToken(): Promise<string | null> {
  // Одна общая Promise защищает от пачки одновременных запросов /auth/refresh при массовом 401.
  refreshAccessTokenRequest ??= requestFreshAccessToken()
    .finally(() => {
      refreshAccessTokenRequest = null;
    });

  return await refreshAccessTokenRequest;
}

async function requestFreshAccessToken(): Promise<string | null> {
  try {
    const response = await fetch(`${apiBaseUrl}/auth/refresh`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      credentials: "include"
    });

    if (!response.ok) {
      return null;
    }

    const data = await response.json() as AccessTokenResponse;

    localStorage.setItem("accessToken", data.token);

    return data.token;
  } catch {
    return null;
  }
}

function handleUnauthorizedResponse() {
  // Событие нужно, чтобы слой API не зависел напрямую от React Router и контекстов.
  localStorage.removeItem("accessToken");
  window.dispatchEvent(new CustomEvent(unauthorizedResponseEventName));
}

function isAuthEndpoint(path: string): boolean {
  return path.startsWith("/auth/");
}

managementService.ts

// Сервис frontend management содержит функции, через которые React-страницы обращаются к backend API.

import { sendRequest } from "../sendRequest";
import type { ManagementDepartureResponse } from
  "../contracts/responses/departures/managementDepartureResponse";
import type { AirportSearchResponse } from "../contracts/responses/airports/airportSearchResponse";

export type ManagementSection = "orders" | "flights" | "completed";
export type ManagementDepartureQuerySection = ManagementSection | "analytics";
```

Продолжение приложения E

```
export interface UpdateDepartureRouteRequest {
  airportIds: number[];
  groundTimesAfterArrival: Array<string | null>;
}

export interface UpdateDepartureStatusDetails {
  delayMinutes?: number | null;
  redirectedAirportId?: number | null;
}

export interface ManagementRoutePreviewResponse {
  distance: number;
  flightTime: string;
  price: number;
  transfers: number;
  canFly: boolean;
  routeAirports: AirportSearchResponse[];
  routeLegs: ManagementRoutePreviewLegResponse[];
}

export interface ManagementRoutePreviewLegResponse {
  fromAirportId: number;
  toAirportId: number;
  distanceKm: number;
  flightTime: string;
  flightCost: number;
  groundTimeAfterArrival?: string | null;
  canFly: boolean;
  maximumLegDistanceKm: number;
}

export interface ManagementRouteCandidateResponse extends AirportSearchResponse {
  distanceFromCurrentKm: number;
  distanceToDestinationKm: number;
  priorityScore: number;
  isCapital: boolean;
  isLargeCity: boolean;
  isBestSystemChoice: boolean;
}

export async function getManagementDepartures(
  section: ManagementDepartureQuerySection,
  signal?: AbortSignal
): Promise<ManagementDepartureResponse[]> {
  const searchParameters = new URLSearchParams({
    section
  });

  return await sendRequest<ManagementDepartureResponse[]>(
    `/departures/management?${searchParameters.toString()}`,
    "GET",
    undefined,
    signal
  );
}

export async function getManagementDeparture(
  departureId: number,
  signal?: AbortSignal
```

Продолжение приложения Е

```
); Promise<ManagementDepartureResponse> {
  return await sendRequest<ManagementDepartureResponse>(
    `/departures/management/${departureId}`,
    "GET",
    undefined,
    signal
  );
}

export async function approveManagementDeparture(departureId: number): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/approve`,
    "POST"
  );
}

export async function approveManagementDepartureRoute(
  departureId: number,
  request: UpdateDepartureRouteRequest
): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/approve-route`,
    "POST",
    request
  );
}

export async function confirmManagementDepartureContractDocument(departureId: number): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/confirm-contract-document`,
    "POST"
  );
}

export async function updateManagementDepartureEmployees(
  departureId: number,
  employeeIds: number[]
): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/employees`,
    "PUT",
    { employeeIds }
  );
}

export async function updateManagementDepartureStatus(
  departureId: number,
  statusId: number,
  includePreviousStatuses = false,
  targetLegIndex?: number | null,
  details: UpdateDepartureStatusDetails = {}
): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/status`,
    "POST",
    {
      statusId,
      includePreviousStatuses,
      targetLegIndex,
      ...details
    }
  );
}
```


Продолжение приложения E

```
    }  
  );  
}  
  
export async function deleteLatestManagementDepartureStatus(departureId: number): Promise<void> {  
  await sendRequest<void>(  
    `/departures/management/${departureId}/status/latest`,  
    "DELETE"  
  );  
}  
  
export async function saveManagementDepartureRoute(  
  departureId: number,  
  request: UpdateDepartureRouteRequest  
): Promise<void> {  
  await sendRequest<void>(  
    `/departures/management/${departureId}/route`,  
    "POST",  
    request  
  );  
}  
  
export async function updateManagementDepartureTakeOffDateTime(  
  departureId: number,  
  requestedTakeOffDateTime: string  
): Promise<void> {  
  await sendRequest<void>(  
    `/departures/management/${departureId}/take-off-date-time`,  
    "POST",  
    { requestedTakeOffDateTime }  
  );  
}  
  
export async function previewManagementDepartureRoute(  
  departureId: number,  
  request: UpdateDepartureRouteRequest,  
  signal?: AbortSignal  
): Promise<ManagementRoutePreviewResponse> {  
  return await sendRequest<ManagementRoutePreviewResponse>(  
    `/departures/management/${departureId}/route-preview`,  
    "POST",  
    request,  
    signal  
  );  
}  
  
export async function getManagementRouteCandidates(  
  departureId: number,  
  fromAirportId: number,  
  toAirportId: number | null,  
  signal?: AbortSignal  
): Promise<ManagementRouteCandidateResponse[]> {  
  const searchParameters = new URLSearchParams({  
    fromAirportId: fromAirportId.toString(),  
    limit: "30"  
  });  
  
  if (toAirportId !== null) {  
    searchParameters.set("toAirportId", toAirportId.toString());  
  }  
}
```

Продолжение приложения Е

```
return await sendRequest<ManagementRouteCandidateResponse[]>(
  `/departures/management/${departureId}/route-candidates?${searchParameters.toString()}`,
  "GET",
  undefined,
  signal
);
}

export async function rejectManagementDeparture(departureId: number): Promise<void> {
  await sendRequest<void>(
    `/departures/management/${departureId}/reject`,
    "POST"
  );
}

airlineService.ts

// Сервис frontend airline содержит функции, через которые React-страницы обращаются к backend API.

import { sendRequest } from "./sendRequest";

export interface AirlineContractSettingsResponse {
  id: number;
  airlineName: string;
  organizationType: string;
  legalAddress: string;
  postalAddress: string;
  phoneNumber: string;
  email: string;
  serviceBaseCost: number;
  transferBaseCost: number;
  bankName: string;
  taxpayerId: string;
  taxRegistrationReasonCode: string;
  primaryStateRegistrationNumber: string;
  currentAccountNumber: string;
  correspondentAccountNumber: string;
  bankIdentifierCode: string;
  contractCity?: string | null;
  contractValidityDays?: number | null;
  paymentDeadlineDays?: number | null;
  cateringClass?: string | null;
  passengerArrivalMinutesBeforeFlight?: number | null;
  isCatalogVisible: boolean;
  hasDepartures: boolean;
  imageBase64?: string | null;
}

export interface UpdateAirlineContractSettingsRequest {
  airlineName: string;
  organizationType: string;
  legalAddress: string;
  postalAddress: string;
  phoneNumber: string;
  email: string;
  serviceBaseCost: number | null;
  transferBaseCost: number | null;
  bankName: string;
  taxpayerId: string;
  taxRegistrationReasonCode: string;
  primaryStateRegistrationNumber: string;
```

Продолжение приложения E

```
currentAccountNumber: string;
correspondentAccountNumber: string;
bankIdentifierCode: string;
contractCity: string | null;
contractValidityDays: number | null;
paymentDeadlineDays: number | null;
cateringClass: string | null;
passengerArrivalMinutesBeforeFlight: number | null;
}

export interface AirlineEmployeeResponse {
  id: number;
  email: string;
  roleName: string;
  fullName?: string | null;
  isEmailConfirmed: boolean;
  isActive: boolean;
}

export interface CreateAirlineEmployeeRequest {
  email: string;
  roleName: string;
}

export interface UpdateAirlineEmployeeRoleRequest {
  roleName: string;
}

export interface CreateAirlineEmployeeResponse {
  employee?: AirlineEmployeeResponse | null;
  notificationCreated: boolean;
  message: string;
}

export interface AirlineNotificationResponse {
  id: number;
  title: string;
  message: string;
  createdAtUtc: string;
  readAtUtc?: string | null;
}

export interface AccessTokenResponse {
  token: string;
}

export type RegisterAirlineRequest = UpdateAirlineContractSettingsRequest;

export async function registerAirline(data: RegisterAirlineRequest): Promise<AccessTokenResponse> {
  return await sendRequest<AccessTokenResponse>(
    "/airlines/register",
    "POST",
    data
  );
}

export async function getMyAirlineEmployees(
  availableForDepartureId?: number,
  includeInactiveOrSignal: boolean | AbortSignal = false,
  signal?: AbortSignal
```

Продолжение приложения E

```
); Promise<AirlineEmployeeResponse[]> {
  const includeInactive = typeof includeInactiveOrSignal === "boolean"
    ? includeInactiveOrSignal
    : false;
  const requestSignal = typeof includeInactiveOrSignal === "boolean"
    ? signal
    : includeInactiveOrSignal;
  const searchParameters = new URLSearchParams();

  if (availableForDepartureId !== undefined) {
    searchParameters.set("availableForDepartureId", availableForDepartureId.toString());
  }

  if (includeInactive) {
    searchParameters.set("includeInactive", "true");
  }

  const query = searchParameters.toString() === ""
    ? ""
    : `?${searchParameters.toString()}`;

  return await sendRequest<AirlineEmployeeResponse[]>(
    `/airlines/my/employees${query}`,
    "GET",
    undefined,
    requestSignal
  );
}

export async function getMyAirlineContractSettings(): Promise<AirlineContractSettingsResponse> {
  return await sendRequest<AirlineContractSettingsResponse>(
    "/airlines/my/contract-settings",
    "GET"
  );
}

export async function updateMyAirlineContractSettings(
  data: UpdateAirlineContractSettingsRequest
): Promise<AirlineContractSettingsResponse> {
  return await sendRequest<AirlineContractSettingsResponse>(
    "/airlines/my/contract-settings",
    "PUT",
    data
  );
}

export async function updateMyAirlineImage(imageBase64: string | null): Promise<void> {
  await sendRequest<void>(
    "/airlines/my/image",
    "PUT",
    { imageBase64 }
  );
}

export async function createMyAirlineEmployee(
  data: CreateAirlineEmployeeRequest
): Promise<CreateAirlineEmployeeResponse> {
  return await sendRequest<CreateAirlineEmployeeResponse>(
    "/airlines/my/employees",
    "POST",
  );
}
```

Продолжение приложения E

```
        data
    );
}

export async function updateMyAirlineEmployeeRole(
    employeeId: number,
    data: UpdateAirlineEmployeeRoleRequest
): Promise<AirlineEmployeeResponse> {
    return await sendRequest<AirlineEmployeeResponse>(
        `/airlines/my/employees/${employeeId}/role`,
        "PUT",
        data
    );
}

export async function dismissMyAirlineEmployee(employeeId: number): Promise<void> {
    await sendRequest<void>(`/airlines/my/employees/${employeeId}`, "DELETE");
}

export async function resignFromMyAirline(): Promise<void> {
    await sendRequest<void>("/airlines/my/employment", "DELETE");
}

export async function getMyAirlineNotifications(signal?: AbortSignal): Promise<AirlineNotificationResponse[]> {
    return await sendRequest<AirlineNotificationResponse[]>(
        "/airlines/my/notifications",
        "GET",
        undefined,
        signal
    );
}

export async function deleteMyAirline(): Promise<void> {
    await sendRequest<void>("/airlines/my", "DELETE");
}

export async function updateMyAirlineCatalogVisibility(
    isCatalogVisible: boolean
): Promise<AirlineContractSettingsResponse> {
    return await sendRequest<AirlineContractSettingsResponse>(
        "/airlines/my/catalog-visibility",
        "PUT",
        { isCatalogVisible }
    );
}

AuthenticationController.cs
// Контроллер Authentication принимает HTTP-запросы, проверяет входные данные и возвращает ответы
// клиентскому приложению.

using AirCharter.API.Model;
using AirCharter.API.Requests.Authentication;
using AirCharter.API.Services;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.WebUtilities;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Text;
using System.Security.Cryptography;

namespace AirCharter.API.Controllers;
```

Продолжение приложения E

```
[ApiController]
[Route("auth")]
public sealed class AuthController(AirCharterExtendedContext context, JwtService jwtService, EmailService emailService) : ControllerBase
{
    private const int ClientRoleId = 1;
    private const int RefreshTokenLifetimeDays = 30;
    private const string RefreshTokenCookieName = "refreshToken";

    private readonly AirCharterExtendedContext _context = context;
    private readonly JwtService _jwtService = jwtService;
    private readonly EmailService _emailService = emailService;
    private readonly PasswordHasher<User> _passwordHasher = new();

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegisterRequest request, CancellationToken cancellationToken)
    {
        if (string.IsNullOrEmpty(request.Email))
            return BadRequest("Укажите почту.");

        if (string.IsNullOrEmpty(request.Password))
            return BadRequest("Укажите пароль.");

        string email = request.Email.Trim();

        User? existingUser = await _context.Users.FirstOrDefaultAsync(user => user.Email == email, cancellationToken);

        if (existingUser != null && (existingUser.IsEmailConfirmed || existingUser.RoleId != ClientRoleId))
            return Conflict("Пользователь с такой почтой уже существует.");

        // Транзакция не дает сохранить пользователя без письма: если отправка кода упадет, изменения откатываются.
        await using Microsoft.EntityFrameworkCore.Storage.IDbContextTransaction transaction =
            await _context.Database.BeginTransactionAsync(cancellationToken);

        User user;

        if (existingUser == null)
        {
            user = new()
            {
                RoleId = ClientRoleId,
                Email = email,
                IsEmailConfirmed = false,
                IsActive = true
            };
        }
        _context.Users.Add(user);
    }
    else
    {
        user = existingUser;
        user.Email = email;
        user.IsActive = true;
    }

    user.PasswordHash = _passwordHasher.HashPassword(user, request.Password);
    string confirmationCode = SetEmailConfirmationCode(user);
}
```

Продолжение приложения Е

```
await _context.SaveChangesAsync(cancellationToken);

try
{
    await _emailService.SendHtmlMessageAsync(user.Email, "Подтверждение почты",
        $"<h3>Код подтверждения</h3><p>Ваш код: <b>{confirmationCode}</b></p><p>Код действует 10 минут.</p>",
        cancellationToken);
}
catch (Exception exception) when (exception is not OperationCanceledException)
{
    await transaction.RollbackAsync(cancellationToken);

    return StatusCode(
        StatusCodes.Status503ServiceUnavailable,
        "Не удалось отправить код подтверждения. Проверьте настройки почты и попробуйте ещё раз.");
}

await transaction.CommitAsync(cancellationToken);

return NoContent();
}

[HttpPost("confirm-email")]
public async Task<IActionResult> ConfirmEmail([FromBody] ConfirmEmailRequest request, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(request.Email))
        return BadRequest("Укажите почту.");

    if (string.IsNullOrEmpty(request.Code))
        return BadRequest("Укажите код подтверждения.");

    User? user = await _context.Users
        .Include(currentUser => currentUser.Role)
        .FirstOrDefaultAsync(currentUser => currentUser.Email == request.Email, cancellationToken);

    if (user == null)
        return NotFound("Пользователь не найден.");

    if (user.IsEmailConfirmed)
        return BadRequest("Почта уже подтверждена.");

    if (string.IsNullOrEmpty(user.EmailConfirmationCodeHash))
        return BadRequest("Код подтверждения не найден. Запросите новый код.");

    if (user.EmailConfirmationCodeExpiresAtUtc == null)
        return BadRequest("Не удалось проверить срок действия кода. Запросите новый код.");

    if (user.EmailConfirmationCodeExpiresAtUtc.Value < DateTime.UtcNow)
        return BadRequest("Срок действия кода истёк. Запросите новый код.");

    bool isConfirmationCodeValid = IsConfirmationCodeValid(user, request.Code);

    if (!isConfirmationCodeValid)
        return BadRequest("Неверный код подтверждения.");

    user.IsEmailConfirmed = true;
    user.EmailConfirmationCodeHash = null;
```

Продолжение приложения Е

```
user.EmailConfirmationCodeExpiresAtUtc = null;

await _context.SaveChangesAsync(cancellationToken);

await IssueRefreshTokenAsync(user, cancellationToken);

string token = _jwtService.GenerateAccessToken(user.Id, user.Role.Name);

return Ok(new AccessTokenResponse
{
    Token = token
});
}

[HttpPost("resend-email-confirmation-code")]
public async Task<IActionResult> ResendEmailConfirmationCode([FromBody]
ResendEmailConfirmationCodeRequest request, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(request.Email))
        return BadRequest("Укажите почту.");

    User? user = await _context.Users.FirstOrDefaultAsync(currentUser => currentUser.Email == request.Email,
cancellationToken);

    if (user == null)
        return NotFound("Пользователь не найден.");

    if (user.IsEmailConfirmed)
        return BadRequest("Почта уже подтверждена.");

    string confirmationCode = SetEmailConfirmationCode(user);

    await _context.SaveChangesAsync(cancellationToken);

    await _emailService.SendHtmlMessageAsync(user.Email, "Новый код подтверждения",
        $"<h3>Код подтверждения</h3><p>Ваш код: <b>{confirmationCode}</b></p><p>Код действует 10 минут.</p>",
        cancellationToken);

    return NoContent();
}

[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest request, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(request.Email))
        return BadRequest("Укажите почту.");

    if (string.IsNullOrEmpty(request.Password))
        return BadRequest("Укажите пароль.");

    User? user = await _context.Users
        .Include(currentUser => currentUser.Role)
        .FirstOrDefaultAsync(currentUser => currentUser.Email == request.Email, cancellationToken);

    if (user == null)
        return Unauthorized();

    if (!user.IsActive)
        return Forbid();
}
```


Продолжение приложения E

```
PasswordVerificationResult passwordVerificationResult =
    _passwordHasher.VerifyHashedPassword(user, user.PasswordHash, request.Password);

if (passwordVerificationResult == PasswordVerificationResult.Failed)
    return Unauthorized();

if (!user.IsEmailConfirmed)
    return BadRequest("Почта не подтверждена.");

// Access-токен живет коротко, а refresh-токен хранится в защищенной cookie и позволяет незаметно
продлить сессию.
await IssueRefreshTokenAsync(user, cancellationToken);

string token = _jwtService.GenerateAccessToken(user.Id, user.Role.Name);

return Ok(new AccessTokenResponse
{
    Token = token
});
}

[HttpPost("refresh")]
public async Task<IActionResult> Refresh(Cancellation token cancellationToken)
{
    string? refreshTokenValue = Request.Cookies[RefreshTokenCookieName];

    if (string.IsNullOrEmpty(refreshTokenValue))
        return Unauthorized();

    string refreshTokenHash = HashRefreshToken(refreshTokenValue);

    RefreshToken? refreshToken = await _context.RefreshTokens
        .Include(currentRefreshToken => currentRefreshToken.User)
        .ThenInclude(user => user.Role)
        .FirstOrDefaultAsync(currentRefreshToken => currentRefreshToken.TokenHash == refreshTokenHash,
cancellationToken);

    if (refreshToken == null)
    {
        ClearRefreshTokenCookie();
        return Unauthorized();
    }

    if (refreshToken.RevokedAtUtc != null || refreshToken.ExpiresAtUtc <= DateTime.UtcNow)
    {
        ClearRefreshTokenCookie();
        return Unauthorized();
    }

    if (!refreshToken.User.IsActive || !refreshToken.User.IsEmailConfirmed)
    {
        ClearRefreshTokenCookie();
        return Unauthorized();
    }

    string nextRefreshTokenValue = GenerateRefreshTokenValue();
    RefreshToken nextRefreshToken = CreateRefreshToken(refreshToken.UserId, nextRefreshTokenValue);

    await using Microsoft.EntityFrameworkCore.Storage.IDbContextTransaction transaction =
```

Продолжение приложения E

```
await _context.Database.BeginTransactionAsync(cancellationTokens);

_context.RefreshTokens.Add(nextRefreshToken);
await _context.SaveChangesAsync(cancellationTokens);

refreshToken.RevokedAtUtc = DateTime.UtcNow;
refreshToken.ReplacedByTokenId = nextRefreshToken.Id;

await _context.SaveChangesAsync(cancellationTokens);
await transaction.CommitAsync(cancellationTokens);

SetRefreshTokenCookie(nextRefreshToken.Value, nextRefreshToken.ExpiresAtUtc);

string accessToken = _jwtService.GenerateAccessToken(refreshToken.User.Id, refreshToken.User.Role.Name);

return Ok(new AccessTokenResponse
{
    Token = accessToken
});
}

[HttpPost("logout")]
public async Task<IActionResult> Logout(CancellationToken cancellationToken)
{
    string? refreshTokenValue = Request.Cookies[RefreshTokenCookieName];

    if (!string.IsNullOrEmpty(refreshTokenValue))
    {
        string refreshTokenHash = HashRefreshToken(refreshTokenValue);

        RefreshToken? refreshToken = await _context.RefreshTokens
            .FirstOrDefaultAsync(currentRefreshToken =>
                currentRefreshToken.TokenHash == refreshTokenHash &&
                currentRefreshToken.RevokedAtUtc == null,
                cancellationToken);

        if (refreshToken != null)
        {
            refreshToken.RevokedAtUtc = DateTime.UtcNow;
            await _context.SaveChangesAsync(cancellationTokens);
        }
    }

    ClearRefreshTokenCookie();

    return NoContent();
}

private string SetEmailConfirmationCode(User user)
{
    string confirmationCode = GenerateEmailConfirmationCode();

    user.EmailConfirmationCodeHash = _passwordHasher.HashPassword(user, confirmationCode);
    user.EmailConfirmationCodeExpiresAtUtc = DateTime.UtcNow.AddMinutes(10);

    return confirmationCode;
}

private bool IsConfirmationCodeValid(User user, string confirmationCode) =>
    _passwordHasher.VerifyHashedPassword(user, user.EmailConfirmationCodeHash!, confirmationCode)
```

Продолжение приложения E

```
!= PasswordVerificationResult.Failed;

private async Task IssueRefreshTokenAsync(User user, CancellationToken cancellationToken)
{
    string refreshTokenValue = GenerateRefreshTokenValue();
    RefreshToken refreshToken = CreateRefreshToken(user.Id, refreshTokenValue);

    _context.RefreshTokens.Add(refreshToken);
    await _context.SaveChangesAsync(cancellationToken);

    SetRefreshTokenCookie(refreshTokenValue, refreshToken.ExpiresAtUtc);
}

private static RefreshToken CreateRefreshToken(int userId, string refreshTokenValue)
{
    return new RefreshToken
    {
        UserId = userId,
        TokenHash = HashRefreshToken(refreshTokenValue),
        CreatedAtUtc = DateTime.UtcNow,
        ExpiresAtUtc = DateTime.UtcNow.AddDays(RefreshTokenLifetimeDays)
    };
}

private void SetRefreshTokenCookie(string refreshTokenValue, DateTime expiresAtUtc)
{
    Response.Cookies.Append(RefreshTokenCookieName, refreshTokenValue, new CookieOptions
    {
        HttpOnly = true,
        Secure = true,
        SameSite = SameSiteMode.None,
        Expires = expiresAtUtc,
        Path = "/auth"
    });
}

private void ClearRefreshTokenCookie()
{
    Response.Cookies.Delete(RefreshTokenCookieName, new CookieOptions
    {
        HttpOnly = true,
        Secure = true,
        SameSite = SameSiteMode.None,
        Path = "/auth"
    });
}

private static string GenerateRefreshTokenValue()
{
    return WebEncoders.Base64UrlEncode(RandomNumberGenerator.GetBytes(64));
}

private static string HashRefreshToken(string refreshTokenValue)
{
    byte[] tokenBytes = Encoding.UTF8.GetBytes(refreshTokenValue);
    byte[] tokenHash = SHA256.HashData(tokenBytes);

    return Convert.ToBase64String(tokenHash);
}
```

Продолжение приложения Е

```
private static string GenerateEmailConfirmationCode() => RandomNumberGenerator.GetInt32(100000, 1000000).ToString();
}
```

RegisterPage.tsx

// Страница RegisterPage собирает данные, обработчики и компоненты для одного пользовательского сценария.

```
import { useRef, useState } from "react";
import type { FormEvent } from "react";
import { useNavigate } from "react-router-dom";
import InputField from "../../components/inputField/InputField";
import { register } from "../../api/authService";
import { getRegisterErrorMessage } from "../../api/utils/authErrorMessages";
import "./auth-page.css";

export default function RegisterPage() {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmedPassword, setConfirmedPassword] = useState("");
  const [errorMessage, setErrorMessage] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const isSubmittingRef = useRef(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();

    if (isSubmittingRef.current) {
      return;
    }

    setErrorMessage("");

    const trimmedEmail = email.trim();

    if (trimmedEmail === "") {
      setErrorMessage("Укажите почту.");
      return;
    }

    if (password === "") {
      setErrorMessage("Укажите пароль.");
      return;
    }

    if (password !== confirmedPassword) {
      setErrorMessage("Пароли не совпадают.");
      return;
    }

    isSubmittingRef.current = true;
    setIsSubmitting(true);

    try {
      await register({
        email: trimmedEmail,
        password: password
      });

      navigate("/confirm-email", {
```

```

    state: {
      email: trimmedEmail
    }
  });
} catch (error) {
  setErrorMessage(getRegisterErrorMessage(error));
} finally {
  isSubmittingRef.current = false;
  setIsSubmitting(false);
}
}

function handleLoginClick() {
  navigate("/login");
}

function handleCatalogClick() {
  navigate("/");
}

return (
  <div className="auth-page">
    <button
      type="button"
      className="auth-back-button"
      onClick={handleCatalogClick}
    >
      В каталог
    </button>

    <section className="auth-card auth-card-register">
      <h1 className="auth-title">Регистрация</h1>

      <form className="auth-form" onSubmit={handleSubmit}>
        <InputField
          id="register-email"
          label="Почта"
          type="email"
          value={email}
          onChange={setEmail}
        />

        <InputField
          id="register-password"
          label="Пароль"
          type="password"
          value={password}
          onChange={setPassword}
        />

        <InputField
          id="register-confirmed-password"
          label="Подтвердите пароль"
          type="password"
          value={confirmedPassword}
          onChange={setConfirmedPassword}
        />

        {errorMessage !== "" && (
          <p className="auth-error-message">{errorMessage}</p>

```

```

    })

    <div className="auth-actions">
      <button
        type="button"
        className="auth-link-button"
        onClick={handleLoginClick}
      >
        Вход
      </button>

      <button
        type="submit"
        className="auth-submit-button"
        disabled={isSubmitting}
      >
        {isSubmitting ? "Регистрация..." : "Зарегистрироваться"}
      </button>
    </div>
  </form>
</section>
</div>
);
}

```

LoginPage.tsx

// Страница LoginPage собирает данные, обработчики и компоненты для одного пользовательского сценария.

```

import { useState } from "react";
import type { FormEvent } from "react";
import { useNavigate } from "react-router-dom";
import InputField from "../../components/inputField/InputField";
import { login } from "../../api/authService";
import { getLoginErrorMessage } from "../../api/utls/authErrorMessages";
import { useUser } from "../../context/UserContext";
import "./auth-page.css";

export default function LoginPage() {
  const navigate = useNavigate();
  const { refreshUser } = useUser();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [errorMessage, setErrorMessage] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setErrorMessage("");

    const trimmedEmail = email.trim();

    if (trimmedEmail === "") {
      setErrorMessage("Укажите почту.");
      return;
    }

    if (password === "") {
      setErrorMessage("Укажите пароль.");
      return;
    }
  }
}

```

```

setIsSubmitting(true);

try {
  const response = await login({
    email: trimmedEmail,
    password: password
  });

  localStorage.setItem("accessToken", response.token);
  await refreshUser();
  navigate("/");
} catch (error) {
  if (isEmailNotConfirmedError(error)) {
    navigate("/confirm-email", {
      state: {
        email: trimmedEmail,
        autoSendCode: true
      }
    });
  }
  return;
}

setErrorMessage(getLoginErrorMessage(error));
} finally {
  setIsSubmitting(false);
}
}

function handleRegisterClick() {
  navigate("/register");
}

function handleCatalogClick() {
  navigate("/");
}

return (
  <div className="auth-page">
    <button
      type="button"
      className="auth-back-button"
      onClick={handleCatalogClick}
    >
      В каталог
    </button>

    <section className="auth-card">
      <h1 className="auth-title">Вход</h1>

      <form className="auth-form" onSubmit={handleSubmit}>
        <InputField
          id="login-email"
          label="Почта"
          type="email"
          value={email}
          onChange={setEmail}
        />

        <InputField

```

Продолжение приложения E

```
        id="login-password"
        label="Пароль"
        type="password"
        value={password}
        onChange={setPassword}
      />

      {errorMessage !== "" && (
        <p className="auth-error-message">{errorMessage}</p>
      )}

      <div className="auth-actions">
        <button
          type="button"
          className="auth-link-button"
          onClick={handleRegisterClick}
        >
          Регистрация
        </button>

        <button
          type="submit"
          className="auth-submit-button"
          disabled={isSubmitting}
        >
          {isSubmitting ? "Вход..." : "Войти"}
        </button>
      </div>
    </form>
  </section>
</div>
);
}

function isEmailNotConfirmedError(error: unknown): boolean {
  return typeof error === "object" &&
    error !== null &&
    "message" in error &&
    (error.message === "Email is not confirmed." || error.message === "Почта не подтверждена.");
}

ConfirmEmailPage.tsx
// Страница ConfirmEmailPage собирает данные, обработчики и компоненты для одного пользовательского
// сценария.

import { useEffect, useRef, useState } from "react";
import type { FormEvent } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import InputField from "../../components/inputField/InputField";
import { confirmEmail, resendEmailConfirmationCode } from "../../api/authService";
import { getConfirmEmailErrorMessage, getResendCodeErrorMessage } from "../../api/Utils/authErrorMessages";
import "./auth-page.css";

type ConfirmEmailLocationState = {
  email?: string;
  autoSendCode?: boolean;
};

export default function ConfirmEmailPage() {
  const navigate = useNavigate();
  const location = useLocation();
```


Продолжение приложения E

```
const state = location.state as ConfirmEmailLocationState | null;
const initialEmail = state?.email ?? "";

const [email, setEmail] = useState(initialEmail);
const [code, setCode] = useState("");
const [errorMessage, setErrorMessage] = useState("");
const [successMessage, setSuccessMessage] = useState("");
const [isSubmitting, setIsSubmitting] = useState(false);
const [isResending, setIsResending] = useState(false);
const didAutoSendCode = useRef(false);

useEffect(() => {
  if (!state?.autoSendCode || didAutoSendCode.current) {
    return;
  }

  didAutoSendCode.current = true;
  void sendConfirmationCode(initialEmail, "Код подтверждения отправлен на почту.");
}, [initialEmail, state?.autoSendCode]);

async function handleSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault();

  setErrorMessage("");
  setSuccessMessage("");

  const trimmedEmail = email.trim();
  const trimmedCode = code.trim();

  if (trimmedEmail === "") {
    setErrorMessage("Укажите почту.");
    return;
  }

  if (trimmedCode === "") {
    setErrorMessage("Укажите код подтверждения.");
    return;
  }

  setIsSubmitting(true);

  try {
    const response = await confirmEmail({
      email: trimmedEmail,
      code: trimmedCode
    });

    void response;
    navigate("/login");
  } catch (error) {
    setErrorMessage(getConfirmEmailErrorMessage(error));
  } finally {
    setIsSubmitting(false);
  }
}

async function handleResendCodeClick() {
  await sendConfirmationCode(email, "Новый код отправлен на почту.");
}
```

```
function handleBackClick() {
  navigate("/register");
}

return (
  <div className="auth-page">
    <section className="auth-card auth-card-confirm">
      <h1 className="auth-title">Подтверждение почты</h1>

      <form className="auth-form" onSubmit={handleSubmit}>
        <InputField
          id="confirm-email"
          label="Почта"
          type="email"
          value={email}
          onChange={setEmail}
        />

        <InputField
          id="confirm-code"
          label="Код"
          type="text"
          value={code}
          onChange={setCode}
        />

        {errorMessage !== "" && (
          <p className="auth-error-message">{errorMessage}</p>
        )}

        {successMessage !== "" && (
          <p className="auth-success-message">{successMessage}</p>
        )}

        <div className="auth-actions">
          <button
            type="button"
            className="auth-link-button"
            onClick={handleBackClick}
          >
            Назад
          </button>

          <button
            type="button"
            className="auth-link-button"
            onClick={handleResendCodeClick}
            disabled={isResending}
          >
            {isResending ? "Отправка..." : "Отправить код снова"}
          </button>

          <button
            type="submit"
            className="auth-submit-button"
            disabled={isSubmitting}
          >
            {isSubmitting ? "Подтверждение..." : "Подтвердить"}
          </button>
        </div>
      </form>
    </section>
  </div>
)
```

Продолжение приложения Е

```
        </div>
      </form>
    </section>
  </div>
);

async function sendConfirmationCode(emailValue: string, successText: string) {
  setErrorMessage("");
  setSuccessMessage("");

  const trimmedEmail = emailValue.trim();

  if (trimmedEmail === "") {
    setErrorMessage("Укажите почту.");
    return;
  }

  setIsResending(true);

  try {
    await resendEmailConfirmationCode({
      email: trimmedEmail
    });

    setSuccessMessage(successText);
  } catch (error) {
    setErrorMessage(getResendCodeErrorMessage(error));
  } finally {
    setIsResending(false);
  }
}
}

// Страница ProfilePage собирает данные, обработчики и компоненты для одного пользовательского сценария.

import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { getMyPerson, updateMyPerson } from "../../api/personService";
import { changeMyPassword } from "../../api/userService";
import { getUpdateProfileErrorMessage } from "../../api/utils/authErrorMessages";
import { getFriendlyApiErrorMessage } from "../../api/utils/apiError";
import Header from "../../components/header/Header";
import InputField from "../../components/inputField/InputField";
import type { ProfileFormData } from "../../contracts/responses/persons/profileFormData";
import "../../ProfilePage.css";
```

```
const emptyProfileFormData: ProfileFormData = {
  firstName: "",
  lastName: "",
  patronymic: "",
  passportSeries: "",
  passportNumber: "",
  email: "",
  birthDate: "",
  registrationAddress: "",
  actualAddress: "",
  phoneNumber: "",
  taxpayerId: "",
  bankName: "",
  currentAccountNumber: "",
}
```

Продолжение приложения E

```
correspondentAccountNumber: null,
bankIdentifierCode: ""
});

function formatLocalDateInput(date: Date): string {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");

  return `${year}-${month}-${day}`;
}

export default function ProfilePage() {
  const navigate = useNavigate();
  const today = formatLocalDateInput(new Date());

  const [initialData, setInitialData] = useState<ProfileFormData | null>(null);
  const [formData, setFormData] = useState<ProfileFormData>(emptyProfileFormData);
  const [isLoading, setIsLoading] = useState(true);
  const [isSaving, setIsSaving] = useState(false);
  const [isPasswordModalOpen, setIsPasswordModalOpen] = useState(false);
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [newPasswordConfirmation, setNewPasswordConfirmation] = useState("");
  const [isPasswordSaving, setIsPasswordSaving] = useState(false);
  const [passwordMessage, setPasswordMessage] = useState<{ text: string; type: "success" | "error" } | null>(null);
  const [statusMessage, setStatusMessage] = useState<{ text: string; type: "success" | "error" } | null>(null);

  useEffect(() => {
    void loadPersonData();
  }, []);

  async function loadPersonData() {
    try {
      const person = await getMyPerson();

      const data: ProfileFormData = {
        firstName: person.firstName,
        lastName: person.lastName,
        patronymic: person.patronymic ?? "",
        passportSeries: person.passportSeries ?? "",
        passportNumber: person.passportNumber ?? "",
        email: person.email ?? "",
        birthDate: person.birthDate ? person.birthDate.toString().split("T")[0] : "",
        registrationAddress: person.registrationAddress ?? "",
        actualAddress: person.actualAddress ?? "",
        phoneNumber: person.phoneNumber ?? "",
        taxpayerId: person.taxpayerId ?? "",
        bankName: person.bankName ?? "",
        currentAccountNumber: person.currentAccountNumber ?? "",
        correspondentAccountNumber: null,
        bankIdentifierCode: person.bankIdentifierCode ?? ""
      };

      setFormData(data);
      setInitialData(data);
      setStatusMessage(null);
    } catch (error: unknown) {
      if (isApiErrorWithStatus(error, 404)) {
        setFormData(emptyProfileFormData);
      }
    }
  }
}
```

Продолжение приложения Е

```
    setInitialData(emptyProfileFormData);
    setStatusMessage(null);
    return;
  }

  setStatusMessage({
    text: getFriendlyApiErrorMessage(error, "Не удалось загрузить профиль. Попробуйте ещё раз."),
    type: "error"
  });
} finally {
  setIsLoading(false);
}
}

const isChanged = JSON.stringify(initialData) !== JSON.stringify(formData);

function updateField(name: keyof ProfileFormData, value: string) {
  setFormData((currentData) => ({ ...currentData, [name]: value }));
}

async function handleSubmit(event: React.FormEvent) {
  event.preventDefault();

  if (!isChanged || isSaving) {
    return;
  }

  if (formData.birthDate && formData.birthDate > today) {
    setStatusMessage({ text: "Дата рождения не может быть в будущем", type: "error" });
    return;
  }

  const sanitizedData: ProfileFormData = {
    ...formData,
    firstName: formData.firstName.trim(),
    lastName: formData.lastName.trim(),
    patronymic: formData.patronymic?.trim() || null,
    email: formData.email?.trim() || null,
    birthDate: formData.birthDate || null,
    registrationAddress: formData.registrationAddress?.trim() || null,
    actualAddress: formData.actualAddress?.trim() || null,
    phoneNumber: formData.phoneNumber?.trim() || null,
    taxpayerId: formData.taxpayerId?.trim() || null,
    bankName: formData.bankName?.trim() || null,
    currentAccountNumber: formData.currentAccountNumber?.trim() || null,
    correspondentAccountNumber: null,
    bankIdentifierCode: formData.bankIdentifierCode?.trim() || null
  };

  setIsSaving(true);
  setStatusMessage(null);

  try {
    await updateMyPerson(sanitizedData);
    setFormData(sanitizedData);
    setInitialData(sanitizedData);
    setStatusMessage({ text: "Изменения сохранены", type: "success" });
  } catch (error: unknown) {
    setStatusMessage({ text: getUpdateProfileErrorMessage(error), type: "error" });
  } finally {
  }
}
```

Продолжение приложения Е

```
    setIsSaving(false);
  }
}

function openPasswordModal() {
  setCurrentPassword("");
  setNewPassword("");
  setNewPasswordConfirmation("");
  setPasswordMessage(null);
  setIsPasswordModalOpen(true);
}

function closePasswordModal() {
  if (isPasswordSaving) {
    return;
  }

  setIsPasswordModalOpen(false);
}

async function handlePasswordSubmit(event: React.FormEvent) {
  event.preventDefault();

  if (isPasswordSaving) {
    return;
  }

  if (currentPassword.trim() === "") {
    setPasswordMessage({ text: "Укажите текущий пароль.", type: "error" });
    return;
  }

  if (newPassword.length < 6) {
    setPasswordMessage({ text: "Новый пароль должен быть не короче 6 символов.", type: "error" });
    return;
  }

  if (newPassword !== newPasswordConfirmation) {
    setPasswordMessage({ text: "Новые пароли не совпадают.", type: "error" });
    return;
  }

  setIsPasswordSaving(true);
  setPasswordMessage(null);

  try {
    await changeMyPassword({
      currentPassword,
      newPassword
    });

    setIsPasswordModalOpen(false);
    setStatusMessage({ text: "Пароль изменён.", type: "success" });
    setCurrentPassword("");
    setNewPassword("");
    setNewPasswordConfirmation("");
  } catch (error: unknown) {
    setPasswordMessage({ text: getChangePasswordErrorMessage(error), type: "error" });
  } finally {
    setIsPasswordSaving(false);
  }
}
```

```

    }
  }

  if (isLoading) {
    return <div className="auth-page"><div className="auth-title">Загрузка...</div></div>;
  }

  return (
    <div className="catalog-wrapper">
      <Header showSearch={false}>
        <button
          className="header-icon-btn"
          onClick={() => navigate(-1)}
          title="Назад"
        >
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
            <line x1="19" y1="12" x2="5" y2="12"></line>
            <polyline points="12 19 5 12 12 5"></polyline>
          </svg>
        </button>
      </Header>

      <div className="profile-scroll-container">
        <div className="profile-content-card">
          <h1 className="auth-title">Профиль</h1>

          <form onSubmit={handleSubmit} className="auth-form">
            <InputField label="Фамилия" value={formData.lastName} onChange={(value) =>
updateField("lastName", value)} required />
            <InputField label="Имя" value={formData.firstName} onChange={(value) => updateField("firstName",
value)} required />
            <InputField label="Отчество" value={formData.patronymic ?? ""} onChange={(value) =>
updateField("patronymic", value)} />

            <div className="passport-row">
              <InputField label="Серия" value={formData.passportSeries} onChange={(value) =>
updateField("passportSeries", value.replace(/D/g, ""))} maxLength={4} required />
              <InputField label="Номер" value={formData.passportNumber} onChange={(value) =>
updateField("passportNumber", value.replace(/D/g, ""))} maxLength={6} required />
            </div>

            <InputField label="Дата рождения" type="date" value={formData.birthDate ?? ""} max={today}
onChange={(value) => updateField("birthDate", value)} />
            <InputField label="Email для связи" type="email" value={formData.email ?? ""} onChange={(value)
=> updateField("email", value)} />
            <InputField label="Адрес регистрации" value={formData.registrationAddress ?? ""}
onChange={(value) => updateField("registrationAddress", value)} maxLength={255} />
            <InputField label="Фактический адрес" value={formData.actualAddress ?? ""} onChange={(value) =>
updateField("actualAddress", value)} maxLength={255} />
            <InputField label="Телефон" value={formData.phoneNumber ?? ""} onChange={(value) =>
updateField("phoneNumber", value)} maxLength={20} />
            <InputField label="ИНН" value={formData.taxpayerId ?? ""} onChange={(value) =>
updateField("taxpayerId", value.replace(/D/g, ""))} maxLength={12} />
            <InputField label="Банк" value={formData.bankName ?? ""} onChange={(value) =>
updateField("bankName", value)} maxLength={100} />
            <InputField label="Счёт" value={formData.currentAccountNumber ?? ""} onChange={(value) =>
updateField("currentAccountNumber", value.replace(/D/g, ""))} maxLength={20} />
            <InputField label="БИК" value={formData.bankIdentifierCode ?? ""} onChange={(value) =>
updateField("bankIdentifierCode", value.replace(/D/g, ""))} maxLength={9} />
          </form>
        </div>
      </div>
    </div>
  )

```

Продолжение приложения Е

```
{statusMessage && (
  <div className={`form-message ${statusMessage.type}`} >
    {statusMessage.text}
  </div>
)}

<div className="auth-actions">
  <button
    type="button"
    className="secondary-button"
    onClick={openPasswordModal}
    disabled={isSaving}
  >
    Сменить пароль
  </button>
  <button
    type="submit"
    className="auth-submit-button"
    disabled={!isChanged || isSaving}
  >
    {isSaving ? "Сохранение..." : "Сохранить"}
  </button>
</div>
</form>
</div>
</div>

{isPasswordModalOpen && (
  <div className="profile-modal-backdrop" role="presentation">
    <form className="profile-password-modal" onSubmit={handlePasswordSubmit}>
      <div className="profile-modal-header">
        <h2>Смена пароля</h2>
        <button
          type="button"
          onClick={closePasswordModal}
          aria-label="Заккрыть"
          disabled={isPasswordSaving}
        >
          ×
        </button>
      </div>
      <InputField
        label="Текущий пароль"
        type="password"
        value={currentPassword}
        onChange={setCurrentPassword}
        autoComplete="current-password"
        required
      />
      <InputField
        label="Новый пароль"
        type="password"
        value={newPassword}
        onChange={setNewPassword}
        autoComplete="new-password"
        required
      />
    </form>
  </div>
)
```


Продолжение приложения E

```
<TextField
  label="Повторите новый пароль"
  type="password"
  value={newPasswordConfirmation}
  onChange={setNewPasswordConfirmation}
  autoComplete="new-password"
  required
/>

{passwordMessage && (
  <div className={`form-message ${passwordMessage.type}`}>
    {passwordMessage.text}
  </div>
)}

<div className="profile-modal-actions">
  <button
    type="button"
    className="secondary-button"
    onClick={closePasswordModal}
    disabled={isPasswordSaving}
  >
    Отмена
  </button>
  <button
    type="submit"
    className="auth-submit-button"
    disabled={isPasswordSaving}
  >
    {isPasswordSaving ? "Сохранение..." : "Сохранить"}
  </button>
</div>
</form>
</div>
)}
</div>
);
}
```

```
function isApiErrorWithStatus(error: unknown, status: number): boolean {
  return typeof error === "object" &&
    error !== null &&
    "status" in error &&
    error.status === status;
}
```

```
function getChangePasswordErrorMessage(error: unknown): string {
  if (typeof error === "object" && error !== null && "message" in error) {
    switch (error.message) {
      case "Current password is required.":
      case "Укажите текущий пароль.":
        return "Укажите текущий пароль.";

      case "New password is required.":
      case "Укажите новый пароль.":
        return "Укажите новый пароль.";

      case "New password is too short.":
      case "Новый пароль должен быть не короче 6 символов.":
        return "Новый пароль должен быть не короче 6 символов.";
    }
  }
}
```

Продолжение приложения E

```
        case "Current password is invalid.":
        case "Текущий пароль указан неверно.":
            return "Текущий пароль указан неверно.";
    }
}

return "Не удалось изменить пароль.";
}

DeparturesController.cs

[HttpPost("create-order")]
public async Task<IActionResult> CreateOrder(
    CreateDepartureRequest createDepartureRequest,
    CancellationToken cancellationToken)
{
    Claim? userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier);

    if (userIdClaim == null || !int.TryParse(userIdClaim.Value, out int userId))
        return Unauthorized();

    if (createDepartureRequest.TakeOffAirportId == createDepartureRequest.LandingAirportId)
        return BadRequest("Аэропорт вылета и аэропорт посадки не должны совпадать.");

    if (IsRequestedTakeOffDateTimeTooEarly(createDepartureRequest.RequestedTakeOffDateTime))
        return BadRequest("Дата и время вылета должны быть не раньше завтрашнего дня.");

    Plane? plane = await _context.Planes
        .Include(currentPlane => currentPlane.Airline)
        .FirstOrDefaultAsync(currentPlane => currentPlane.Id == createDepartureRequest.PlaneId, cancellationToken);

    if (plane == null)
        return NotFound("Самолёт не найден.");

    AirportGraph airportGraph = await _airportGraphCache.GetOrCreateAsync(_context, cancellationToken);

    PlaneCatalogResponse routeCalculation = _routePlanningService
        .CalculateCatalog(
            new[] { plane },
            airportGraph,
            createDepartureRequest.TakeOffAirportId,
            createDepartureRequest.LandingAirportId)
        .First();

    if (!routeCalculation.IsRouteFound)
        return BadRequest("Для выбранного самолёта маршрут не найден.");

    Departure departure = new()
    {
        CharterRequesterId = userId,
        PlaneId = createDepartureRequest.PlaneId,
        TakeOffAirportId = createDepartureRequest.TakeOffAirportId,
        LandingAirportId = createDepartureRequest.LandingAirportId,
        RequestedTakeOffDateTime = createDepartureRequest.RequestedTakeOffDateTime
    };

    AddRouteLegs(departure, routeCalculation.RouteLegs);
    ApplyRouteTotals(departure);

    departure.DepartureStatuses.Add(new DepartureStatus
    {
```

Продолжение приложения E

```
        StatusId = _context.Statuses.Min(status => status.Id),
        StatusSettingDateTime = DateTime.UtcNow
    });

    _context.Departures.Add(departure);
    await _context.SaveChangesAsync(cancellationToken);

    return NoContent();
}
[HttpPost("my/{departureId:int}/submit")]
public async Task<IActionResult> SubmitMyDeparture(
    int departureId,
    CancellationToken cancellationToken)
{
    int? userId = GetCurrentUserId();

    if (userId is null)
        return Unauthorized();

    Departure? departure = await GetManagementDepartureQuery()
        .FirstOrDefaultAsync(
            departure => departure.Id == departureId &&
            departure.CharterRequesterId == userId.Value,
            cancellationToken);

    if (departure is null)
        return NotFound();

    DepartureStatus? currentStatus = GetCurrentStatus(departure);

    if (currentStatus?.StatusId != (int)FlightStatusId.InCreation)
        return BadRequest("Заявку можно отправить только из статуса создания.");

    if (departure.People.Count == 0)
        return BadRequest("Добавьте хотя бы одного пассажира.");

    if (departure.DepartureRouteLegs.Count == 0)
        return BadRequest("Маршрут заявки не рассчитан.");

    AddDepartureStatus(departure, FlightStatusId.AwaitingApproval);
    await _context.SaveChangesAsync(cancellationToken);

    return NoContent();
}
[HttpPost("management/{departureId:int}/approve")]
[Authorize(Roles = ManagementEditorRoles)]
public async Task<IActionResult> ApproveManagementDeparture(
    int departureId,
    CancellationToken cancellationToken)
{
    Departure? departure = await GetEditableManagementDepartureAsync(departureId, cancellationToken);

    if (departure is null)
        return NotFound();

    DepartureStatus? currentStatus = GetCurrentStatus(departure);

    if (currentStatus?.StatusId != (int)FlightStatusId.AwaitingApproval)
        return BadRequest("Заявку можно одобрить только в статусе ожидания одобрения.");
}
```

Продолжение приложения E

```
AddDepartureStatus(departure, FlightStatusId.AwaitingContractSigning);
AddDepartureStatusChangedNotification(departure, FlightStatusId.AwaitingContractSigning);

await _context.SaveChangesAsync(cancellationToken);

return NoContent();
}
[HttpPost("management/{departureId:int}/route")]
[Authorize(Roles = ManagementEditorRoles)]
public async Task<IActionResult> SaveManagementDepartureRoute(
    int departureId,
    UpdateDepartureRouteRequest request,
    CancellationToken cancellationToken)
{
    Departure? departure = await GetEditableManagementDepartureAsync(departureId, cancellationToken);

    if (departure is null)
        return NotFound();

    DepartureStatus? currentStatus = GetCurrentStatus(departure);

    if (currentStatus?.StatusId != (int)FlightStatusId.AwaitingApproval)
        return BadRequest("Маршрут можно редактировать только до одобрения заявки.");

    ActionResult<ManualRouteCalculation> calculationResult = await TryCalculateManualRouteAsync(
        departure,
        request,
        allowSameAirportLegs: false,
        cancellationToken);

    if (calculationResult.Result is not null)
        return calculationResult.Result;

    ManualRouteCalculation calculation = calculationResult.Value!;

    if (!CreateManagementRoutePreviewResponse(departure.Plane, calculation.RouteLegs).CanFly)
        return BadRequest("Одно из плеч маршрута превышает безопасную дальность самолёта.");

    string previousRouteSignature = CreateRouteSignature(departure);

    _context.DepartureRouteLegs.RemoveRange(departure.DepartureRouteLegs.ToList());
    departure.DepartureRouteLegs.Clear();

    AddRouteLegs(departure, calculation.RouteLegs);
    ApplyRouteTotals(departure);

    bool routeChanged = HasRouteChanged(previousRouteSignature, calculation.RouteLegs);

    if (routeChanged)
        AddDepartureRouteChangedNotification(departure, calculation.RouteAirports);

    await _context.SaveChangesAsync(cancellationToken);

    if (routeChanged)
        await SendRouteChangedEmailAsync(departure, calculation.RouteAirports, cancellationToken);

    return NoContent();
}
[HttpPost("management/{departureId:int}/status")]
[Authorize(Roles = ManagementEditorRoles)]
```

Продолжение приложения Е

```
public async Task<IActionResult> UpdateManagementDepartureStatus(
    int departureId,
    [FromBody] UpdateManagementDepartureStatusRequest request,
    CancellationToken cancellationToken)
{
    int? userAirlineId = await GetCurrentUserAirlineIdAsync(cancellationToken);

    if (userAirlineId is null)
        return Forbid();

    if (!Enum.IsDefined(typeof(FlightStatusId), request.StatusId))
        return BadRequest("Выбран неизвестный статус вылета.");

    FlightStatusId nextStatusId = (FlightStatusId)request.StatusId;

    if (!IsOperationalStatus(nextStatusId))
        return BadRequest("Этот статус нельзя установить из управления вылетом.");

    Departure? departure = await GetManagementDepartureQuery()
        .FirstOrDefaultAsync(
            departure => departure.Id == departureId,
            cancellationToken);

    if (departure is null)
        return NotFound();

    if (departure.Plane.AirlineId != userAirlineId.Value)
        return Forbid();

    int? delayMinutes = null;
    Airport? redirectedAirport = null;

    if (nextStatusId == FlightStatusId.Delayed)
    {
        if (request.DelayMinutes is null or <= 0)
            return BadRequest("Укажите примерное время задержки вылета.");

        if (request.DelayMinutes > 72 * 60)
            return BadRequest("Примерная задержка не должна превышать 72 часа.");

        delayMinutes = request.DelayMinutes.Value;
    }

    if (nextStatusId == FlightStatusId.Redirected)
    {
        if (request.RedirectedAirportId is null)
            return BadRequest("Укажите аэропорт перенаправления вылета.");

        redirectedAirport = await _context.Airports
            .FirstOrDefaultAsync(
                airport => airport.Id == request.RedirectedAirportId.Value,
                cancellationToken);

        if (redirectedAirport is null)
            return NotFound("Аэропорт перенаправления не найден.");
    }

    DepartureStatus? currentStatus = GetCurrentStatus(departure);

    if (currentStatus is null || !IsActiveFlightStatus(currentStatus.StatusId))
```

Продолжение приложения Е

```
return BadRequest("Сейчас нельзя изменить статус вылета.");

bool shouldIncludePreviousStatuses = request.IncludePreviousStatuses &&
    nextStatusId != FlightStatusId.Delayed &&
    nextStatusId != FlightStatusId.Redirected;

if (shouldIncludePreviousStatuses)
{
    FlightStatusId currentSequenceStatusId =
        GetCurrentOperationalSequenceStatusId(departure) ??
        (FlightStatusId)currentStatus.StatusId;

    if (IsStatusAlreadyPastCatchUpTarget(
        departure,
        currentSequenceStatusId,
        nextStatusId,
        departure.DepartureRouteLegs.Count,
        request.TargetLegIndex))
    {
        return BadRequest("Текущий статус уже дальше рассчитанного статуса.");
    }

    IReadOnlyCollection<FlightStatusId> catchUpSequence = BuildOperationalStatusCatchUpSequence(
        departure,
        currentSequenceStatusId,
        nextStatusId,
        departure.DepartureRouteLegs.Count,
        request.TargetLegIndex);

    if (RequiresCrewBeforeDeparture(catchUpSequence) && departure.Employees.Count == 0)
        return BadRequest("Для вылета назначьте хотя бы одного члена экипажа.");

    foreach (FlightStatusId statusId in catchUpSequence)
    {
        AddDepartureStatus(departure, statusId);
        AddDepartureStatusChangedNotification(departure, statusId);
    }

    await _context.SaveChangesAsync(cancellationToken);
    return NoContent();
}

if (currentStatus.StatusId != request.StatusId)
{
    if (RequiresCrewBeforeDeparture(nextStatusId) && departure.Employees.Count == 0)
        return BadRequest("Для вылета назначьте хотя бы одного члена экипажа.");

    AddDepartureStatus(
        departure,
        nextStatusId,
        delayMinutes,
        redirectedAirport?.Id);

    AddDepartureStatusChangedNotification(
        departure,
        nextStatusId,
        delayMinutes,
        redirectedAirport);

    await _context.SaveChangesAsync(cancellationToken);
}
```

```

    }

    return NoContent();
}

[HttpPost("management/{departureId:int}/route")]
[Authorize(Roles = ManagementEditorRoles)]
public async Task<IActionResult> SaveManagementDepartureRoute(
    int departureId,
    UpdateDepartureRouteRequest request,
    CancellationToken cancellationToken)
{
    Departure? departure = await GetEditableManagementDepartureAsync(
        departureId,
        cancellationToken);

    if (departure is null)
        return NotFound();

    DepartureStatus? currentStatus = GetCurrentStatus(departure);

    if (currentStatus?.StatusId != (int)FlightStatusId.AwaitingApproval)
        return BadRequest("Маршрут можно редактировать только до одобрения заявки.");

    ActionResult<ManualRouteCalculation> calculationResult = await TryCalculateManualRouteAsync(
        departure,
        request,
        allowSameAirportLegs: false,
        cancellationToken);

    if (calculationResult.Result is not null)
        return calculationResult.Result;

    ManualRouteCalculation calculation = calculationResult.Value!;

    if (!CreateManagementRoutePreviewResponse(departure.Plane, calculation.RouteLegs).CanFly)
        return BadRequest("Одно из плеч маршрута превышает безопасную дальность самолёта.");

    string previousRouteSignature = CreateRouteSignature(departure);

    List<DepartureRouteLeg> existingRouteLegs = departure.DepartureRouteLegs.ToList();
    _context.DepartureRouteLegs.RemoveRange(existingRouteLegs);
    departure.DepartureRouteLegs.Clear();

    AddRouteLegs(departure, calculation.RouteLegs);
    ApplyRouteTotals(departure);

    bool routeChanged = HasRouteChanged(previousRouteSignature, calculation.RouteLegs);

    if (routeChanged)
        AddDepartureRouteChangedNotification(departure, calculation.RouteAirports);

    await _context.SaveChangesAsync(cancellationToken);

    if (routeChanged)
        await SendRouteChangedEmailAsync(departure, calculation.RouteAirports, cancellationToken);

    return NoContent();
}

```

Продолжение приложения E

```
[HttpPost("management/{departureId:int}/route-preview")]
[Authorize(Roles = ManagementEditorRoles)]
public async Task<ActionResult<ManagementRoutePreviewResponse>> PreviewManagementDepartureRoute(
    int departureId,
    UpdateDepartureRouteRequest request,
    CancellationToken cancellationToken)
{
    Departure? departure = await GetEditableManagementDepartureAsync(
        departureId,
        cancellationToken);

    if (departure is null)
        return NotFound();

    ActionResult<ManualRouteCalculation> calculationResult = await TryCalculateManualRouteAsync(
        departure,
        request,
        allowSameAirportLegs: true,
        cancellationToken);

    if (calculationResult.Result is not null)
        return calculationResult.Result;

    ManualRouteCalculation calculation = calculationResult.Value!;

    return Ok(CreateManagementRoutePreviewResponse(
        departure.Plane,
        calculation.RouteLegs,
        calculation.RouteAirports));
}

private async Task<ActionResult<ManualRouteCalculation>> TryCalculateManualRouteAsync(
    Departure departure,
    UpdateDepartureRouteRequest request,
    bool allowSameAirportLegs,
    CancellationToken cancellationToken)
{
    if (request.AirportIds.Count < 2)
        return BadRequest("В маршруте должно быть минимум два аэропорта.");

    int[] airportIds = request.AirportIds.ToArray();

    if (airportIds[0] != departure.TakeOffAirportId)
        return BadRequest("Первый аэропорт маршрута должен совпадать с аэропортом вылета.");

    if (airportIds[^1] != departure.LandingAirportId)
        return BadRequest("Последний аэропорт маршрута должен совпадать с аэропортом прилёта.");

    for (int airportIndex = 1; airportIndex < airportIds.Length; airportIndex++)
    {
        if (!allowSameAirportLegs && airportIds[airportIndex - 1] == airportIds[airportIndex])
            return BadRequest("Соседние плечи не могут использовать один и тот же аэропорт.");
    }

    TimeSpan?[] groundTimesAfterArrival = request.GroundTimesAfterArrival.ToArray();
    int legsCount = airportIds.Length - 1;

    if (groundTimesAfterArrival.Length > legsCount)
        return BadRequest("Количество стоянок не должно превышать количество плеч маршрута.");
}
```


Продолжение приложения E

```
foreach (TimeSpan? groundTimeAfterArrival in groundTimesAfterArrival)
{
    if (groundTimeAfterArrival is { } value
        && (value < TimeSpan.Zero || value > TimeSpan.FromHours(24)))
    {
        return BadRequest("Время стоянки должно быть от 0 до 24 часов.");
    }
}

List<Airport> airports = await _context.Airports
    .Where(airport => airportIds.Contains(airport.Id))
    .ToListAsync(cancellationToken);

if (airports.Count != airportIds.Distinct().Count())
    return BadRequest("Один или несколько аэропортов маршрута не найдены.");

Dictionary<int, Airport> airportById = airports.ToDictionary(airport => airport.Id);

List<Airport> orderedAirports = airportIds
    .Select(airportId => airportById[airportId])
    .ToList();

IReadOnlyCollection<RouteLegResponse> routeLegs =
    _routePlanningService.CalculateRouteLegs(
        departure.Plane,
        orderedAirports,
        groundTimesAfterArrival);

return new ManualRouteCalculation
{
    RouteAirports = orderedAirports,
    RouteLegs = routeLegs
};
}

private static ManagementRoutePreviewResponse CreateManagementRoutePreviewResponse(
    Plane plane,
    IReadOnlyCollection<RouteLegResponse> routeLegs,
    IReadOnlyCollection<Airport>? routeAirports = null)
{
    int maximumLegDistanceKm = RoutePlanningService.GetMaximumLegDistanceKilometers(
        plane.MaxDistance);

    bool canFly = routeLegs.All(routeLeg =>
        routeLeg.FromAirportId != routeLeg.ToAirportId &&
        routeLeg.DistanceKm <= maximumLegDistanceKm);

    TimeSpan flightTime = BaseOperationalDuration;

    foreach (RouteLegResponse routeLeg in routeLegs)
    {
        flightTime += routeLeg.FlightTime;

        if (routeLeg.GroundTimeAfterArrival is not null)
            flightTime += routeLeg.GroundTimeAfterArrival.Value;
    }

    return new ManagementRoutePreviewResponse
    {
        Distance = routeLegs.Sum(routeLeg => routeLeg.DistanceKm),
```

Продолжение приложения E

```

FlightTime = flightTime,
Price = routeLegs.Sum(routeLeg => routeLeg.FlightCost),
Transfers = Math.Max(0, routeLegs.Count - 1),
CanFly = canFly,
RouteAirports = routeAirports is null
    ? Array.Empty<AirportSearchResponse>()
    : routeAirports.Select(CreateAirportResponse).ToArray(),
RouteLegs = routeLegs
    .Select(routeLeg => new ManagementRoutePreviewLegResponse
    {
        FromAirportId = routeLeg.FromAirportId,
        ToAirportId = routeLeg.ToAirportId,
        DistanceKm = routeLeg.DistanceKm,
        FlightTime = routeLeg.FlightTime,
        FlightCost = routeLeg.FlightCost,
        GroundTimeAfterArrival = routeLeg.GroundTimeAfterArrival,
        CanFly = routeLeg.FromAirportId != routeLeg.ToAirportId &&
            routeLeg.DistanceKm <= maximumLegDistanceKm,
        MaximumLegDistanceKm = maximumLegDistanceKm
    })
    .ToArray()
};
}

private static void AddRouteLegs(
    Departure departure,
    IReadOnlyCollection<RouteLegResponse> routeLegs)
{
    int routeLegSequenceNumber = 1;

    foreach (RouteLegResponse routeLeg in routeLegs)
    {
        departure.DepartureRouteLegs.Add(new DepartureRouteLeg
        {
            SequenceNumber = routeLegSequenceNumber,
            FromAirportId = routeLeg.FromAirportId,
            ToAirportId = routeLeg.ToAirportId,
            Distance = routeLeg.DistanceKm,
            FlightTime = routeLeg.FlightTime,
            FlightCost = routeLeg.FlightCost,
            GroundTimeAfterArrival = routeLeg.GroundTimeAfterArrival
        });

        routeLegSequenceNumber++;
    }
}

private static void ApplyRouteTotals(Departure departure)
{
    RouteTotals routeTotals = CreateRouteTotals(departure);

    departure.Distance = routeTotals.Distance;
    departure.FlightTime = routeTotals.FlightTime;
    departure.Price = routeTotals.Price;
    departure.Transfers = routeTotals.Transfers;
}

private static RouteTotals CreateRouteTotals(Departure departure)
{
    DepartureRouteLeg[] routeLegs = departure.DepartureRouteLegs

```

Продолжение приложения E

```
.OrderBy(routeLeg => routeLeg.SequenceNumber)
.ToArray();

if (routeLegs.Length == 0)
{
    return new RouteTotals
    {
        Distance = departure.Distance,
        FlightTime = departure.FlightTime,
        Price = departure.Price,
        Transfers = departure.Transfers
    };
}

TimeSpan flightTime = BaseOperationalDuration;

foreach (DepartureRouteLeg routeLeg in routeLegs)
{
    flightTime += routeLeg.FlightTime;

    if (routeLeg.GroundTimeAfterArrival is not null)
        flightTime += routeLeg.GroundTimeAfterArrival.Value;
}

return new RouteTotals
{
    Distance = routeLegs.Sum(routeLeg => routeLeg.Distance),
    FlightTime = flightTime,
    Price = routeLegs.Sum(routeLeg => routeLeg.FlightCost),
    Transfers = Math.Max(0, routeLegs.Length - 1)
};
}

private static IReadOnlyCollection<FlightStatusId> BuildOperationalStatusCatchUpSequence(
    Departure departure,
    FlightStatusId currentStatusId,
    FlightStatusId targetStatusId,
    int routeLegCount,
    int? targetLegIndex)
{
    IReadOnlyList<FlightStatusId> sequence = BuildOperationalStatusSequence(
        routeLegCount,
        targetLegIndex,
        targetStatusId);

    int currentIndex = FindCurrentStatusIndex(sequence, departure, currentStatusId);
    int targetIndex = FindLastStatusIndex(sequence, targetStatusId);

    if (targetIndex < 0)
        return Array.Empty<FlightStatusId>();

    if (currentIndex < 0)
        currentIndex = -1;

    if (targetIndex <= currentIndex)
        return Array.Empty<FlightStatusId>();

    return sequence
        .Skip(currentIndex + 1)
        .Take(targetIndex - currentIndex)
```

```

        .ToArray();
    }

private static bool IsStatusAlreadyPastCatchUpTarget(
    Departure departure,
    FlightStatusId currentStatusId,
    FlightStatusId targetStatusId,
    int routeLegCount,
    int? targetLegIndex)
{
    IReadOnlyList<FlightStatusId> sequence =
        BuildOperationalStatusSequence(routeLegCount, targetLegIndex, targetStatusId);

    int currentIndex = FindCurrentStatusIndex(sequence, departure, currentStatusId);
    int targetIndex = FindLastStatusIndex(sequence, targetStatusId);

    if (targetIndex < 0)
        return true;

    if (currentIndex < 0)
        return true;

    return currentIndex > targetIndex;
}

private static IReadOnlyList<FlightStatusId> BuildOperationalStatusSequence(
    int routeLegCount,
    int? targetLegIndex,
    FlightStatusId targetStatusId)
{
    List<FlightStatusId> fullSequence = new()
    {
        FlightStatusId.Scheduled,
        FlightStatusId.Planned,
        FlightStatusId.RegistrationOpen,
        FlightStatusId.RegistrationClosing,
        FlightStatusId.RegistrationClosed,
        FlightStatusId.AwaitingBoarding,
        FlightStatusId.Boarding,
        FlightStatusId.GateOpen,
        FlightStatusId.GateClosed,
        FlightStatusId.BoardingCompleted,
        FlightStatusId.EnRoute
    };

    int normalizedRouteLegCount = Math.Max(1, routeLegCount);
    int normalizedTargetLegIndex = Math.Clamp(
        targetLegIndex ?? 0,
        0,
        normalizedRouteLegCount - 1);

    for (int legIndex = 0; legIndex < normalizedTargetLegIndex; legIndex++)
    {
        fullSequence.Add(FlightStatusId.IntermediateStop);
        fullSequence.Add(FlightStatusId.EnRoute);
    }

    if (targetStatusId == FlightStatusId.IntermediateStop)
    {
        fullSequence.Add(FlightStatusId.IntermediateStop);
    }
}

```

Продолжение приложения Е

```
}
else if (targetStatusId == FlightStatusId.Landed)
{
    fullSequence.Add(FlightStatusId.Landed);
}

return fullSequence;
}

[HttpGet("{departureId:int}/ticket")]
public async Task<IActionResult> DownloadTicket(
    int departureId,
    CancellationToken cancellationToken)
{
    Claim? userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier);

    if (userIdClaim == null || !int.TryParse(userIdClaim.Value, out int userId))
        return Unauthorized();

    Departure? departure = await _context.Departures
        .Include(departure => departure.Plane)
        .ThenInclude(plane => plane.Airline)
        .Include(departure => departure.TakeOffAirport)
        .Include(departure => departure.LandingAirport)
        .Include(departure => departure.Employees)
        .Include(departure => departure.People)
        .FirstOrDefaultAsync(
            departure => departure.Id == departureId,
            cancellationToken);

    if (departure == null)
        return NotFound();

    int? userAirlineId = await GetCurrentUserAirlineIdAsync(cancellationToken);

    bool isRequester = departure.CharterRequesterId == userId;
    bool isAirlineEmployee = CanCurrentUserAccessAirlineDeparture(
        departure,
        userId,
        userAirlineId);
    bool canDownloadAsAirline = isAirlineEmployee && CanCurrentUserEditManagementDepartures();

    if (!isRequester && !canDownloadAsAirline)
        return Forbid();

    DeparturePdfData departurePdfData = _departurePdfDataFactory.Create(departure);
    byte[] pdfBytes = _ticketPdfService.Generate(departurePdfData);

    return File(pdfBytes, "application/pdf", $"ticket-{departureId}.pdf");
}

[HttpGet("{departureId:int}/contract")]
public async Task<IActionResult> DownloadContract(
    int departureId,
    CancellationToken cancellationToken)
{
    Claim? userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier);

    if (userIdClaim == null || !int.TryParse(userIdClaim.Value, out int userId))
        return Unauthorized();
}
```

```

Departure? departure = await _context.Departures
    .Include(departure => departure.Plane)
    .ThenInclude(plane => plane.Airline)
    .Include(departure => departure.TakeOffAirport)
    .Include(departure => departure.LandingAirport)
    .Include(departure => departure.DepartureStatuses)
    .Include(departure => departure.DepartureRouteLegs)
    .Include(departure => departure.People)
    .FirstOrDefaultAsync(
        departure => departure.Id == departureId,
        cancellationToken);

if (departure == null)
    return NotFound();

int? userAirlineId = await GetCurrentUserAirlineIdAsync(cancellationToken);

bool isRequester = departure.CharterRequesterId == userId;
bool isAirlineEmployee = CanCurrentUserAccessAirlineDeparture(
    departure,
    userId,
    userAirlineId);

if (!isRequester && !isAirlineEmployee)
    return Forbid();

User? signingUser = await _context.Users
    .Include(user => user.Person)
    .Include(user => user.Role)
    .FirstOrDefaultAsync(user => user.Id == userId, cancellationToken);

if (signingUser is null)
    return Unauthorized();

ContractPdfDataResult contractDataResult =
    _contractPdfDataFactory.Create(departure, signingUser);

if (!contractDataResult.IsSuccess)
{
    return BadRequest(
        CreateContractMissingDataMessage(contractDataResult.MissingFields));
}

byte[] pdfBytes = _contractPdfService.Generate(contractDataResult.Data!);

return File(pdfBytes, "application/pdf", $"contract-{departureId}.pdf");
}

```

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 19.701-90 Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения = Unified system for program documentation. Data, program and system flowcharts, program network charts and system resources charts. Documentation symbols and conventions for flowcharting : межгосударственный стандарт : издание официальное утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26.12.90 № 3294 : дата введения : 1992-01-01 / разработан Государственным комитетом СССР по вычислительной технике и информатике. - Москва : Стандартинформ, 2010. – Текст : непосредственный.

2 ГОСТР 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам = Unified system for design documentation. General requirements for textual documents : национальный стандарт Российской Федерации : издание официальное утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 29 апреля 2019 г. № 175-ст : введен впервые : дата введения : 2020-02-01 : издание с изменением № 1 (ИУС 3—2021) / разработан ФГУП «СТАНДАРТИНФОРМ». - Москва : Стандартинформ, 2021. – Текст : непосредственный.

3 ГОСТ Р 59793- 2021. Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания = Information technology. Set of standards for automated systems. Stages of development : национальный стандарт Российской Федерации : издание официальное утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 25 октября 2021 г. № 1285-ст : введен впервые : дата введения : 2022-01-30 / разработан ООО «ИАВЦ». - Москва : Российский институт стандартизации, 2021. – Текст :

					40.С2454-2026 09.02.07 ДП-ПЗ	Лист
						135
Изм.	Лист	№ докум.	Подпись	Дата		

непосредственный.

4 Галиаскаров, Э. Г. Анализ и проектирование систем с использованием UML : учебник для вузов / Э. Г. Галиаскаров, А. С. Воробьев. — Москва : Издательство Юрайт, 2025. — 125 с. — (Высшее образование). — ISBN 978-5-534-14903-6. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/568178> (дата обращения: 09.04.2026).

5 Аврамчиков, В. М. Цифровая трансформация в авиационной отрасли: возможности и перспективы / В. М. Аврамчиков, А. С. Тимохович, И. П. Рожнов // Вестник евразийской науки. — 2024. — Т. 16. — № 3. — Текст : электронный. — URL: <https://esj.today/PDF/04ECVN324.pdf> (дата обращения: 09.04.2026).

6 Давыдов, А. Д. Развитие сервисов обслуживания пассажирских авиаперевозок с использованием цифровых экосистем / А. Д. Давыдов, В. Г. Даудов, С. А. Горохова // Московский экономический журнал. — 2024. — № 2. — Текст : электронный. — URL: <https://www.iacj.ru/en/storage/download/144363> (дата обращения: 09.04.2026).

7 Кореньков, В. В. Технологии баз данных: проектирование реляционных баз данных : учебное пособие / В. В. Кореньков, И. А. Филозова, О. В. Иванцова. — Москва : КУРС, 2024. — 128 с. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/144825.html> (дата обращения: 09.04.2026).

8 Мельник, А. А. Основные принципы построения и проектирования интерфейса API // Вестник науки. — 2025. — Текст : электронный // КиберЛенинка : [сайт]. — URL: <https://cyberleninka.ru/article/n/osnovnyye-printsipy-postroeniya-i-proektirovaniya-interfeysa-api> (дата обращения: 09.04.2026).

9 Мищенко, Я. В. Базы данных: современные тенденции в области баз данных и их значимость для различных сфер деятельности // Вестник науки. — 2024. — Текст : электронный // КиберЛенинка : [сайт]. — URL:

<https://cyberleninka.ru/article/n/bazy-dannyh-sovremennye-tendentsii-v-oblasti-baz-dannyh-i-ih-znachimost-dlya-razlichnyh-sfer-deyatelnosti> (дата обращения: 09.04.2026).

10 Останакулов, Х. А. У. Проектирование информационной базы для интеллектуальной среды // Science and Education. — 2025. — Текст : электронный. — URL: <https://openscience.uz/index.php/sciedu/article/view/7604> (дата обращения: 09.04.2026).

11 Понин, Ф. Н. Методология проектирования и создания баз данных для современного программного обеспечения // Universum: технические науки. — 2024. — Текст : электронный // КиберЛенинка : [сайт]. — URL: <https://cyberleninka.ru/article/n/metodologiya-proektirovaniya-i-sozdaniya-baz-dannyh-dlya-sovremennogo-programmnogo-obespecheniya> (дата обращения: 09.04.2026).

12 Попцов, П. В. Разработка алгоритма выбора паттерна API с учетом имеющихся требований к информационному обмену / П. В. Попцов, А. В. Козлова // Вестник Воронежского государственного технического университета. — 2025. — Т. 21. — № 2. — С. 91–99. — Текст : электронный. — URL: <https://journals.rcsi.science/1729-6501/article/view/408219> (дата обращения: 09.04.2026).

13 Стружкин, Н. П. Базы данных: проектирование : учебник для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. — Москва : Издательство Юрайт, 2025. — 477 с. — ISBN 978-5-534-11635-9. — Текст : электронный // Образовательная платформа Юрайт : [сайт]. — URL: <https://urait.ru/bcode/566509> (дата обращения: 09.04.2026).

14 ASP.NET Core web API documentation [Электронный ресурс] // Microsoft Learn : [сайт]. — URL: <https://learn.microsoft.com/aspnet/core/web-api/> (дата обращения: 09.04.2026). — Текст : электронный.

15 Entity Framework Core documentation [Электронный ресурс] // Microsoft

Learn : [сайт]. — URL: <https://learn.microsoft.com/ef/core/> (дата обращения: 09.04.2026). — Текст : электронный.

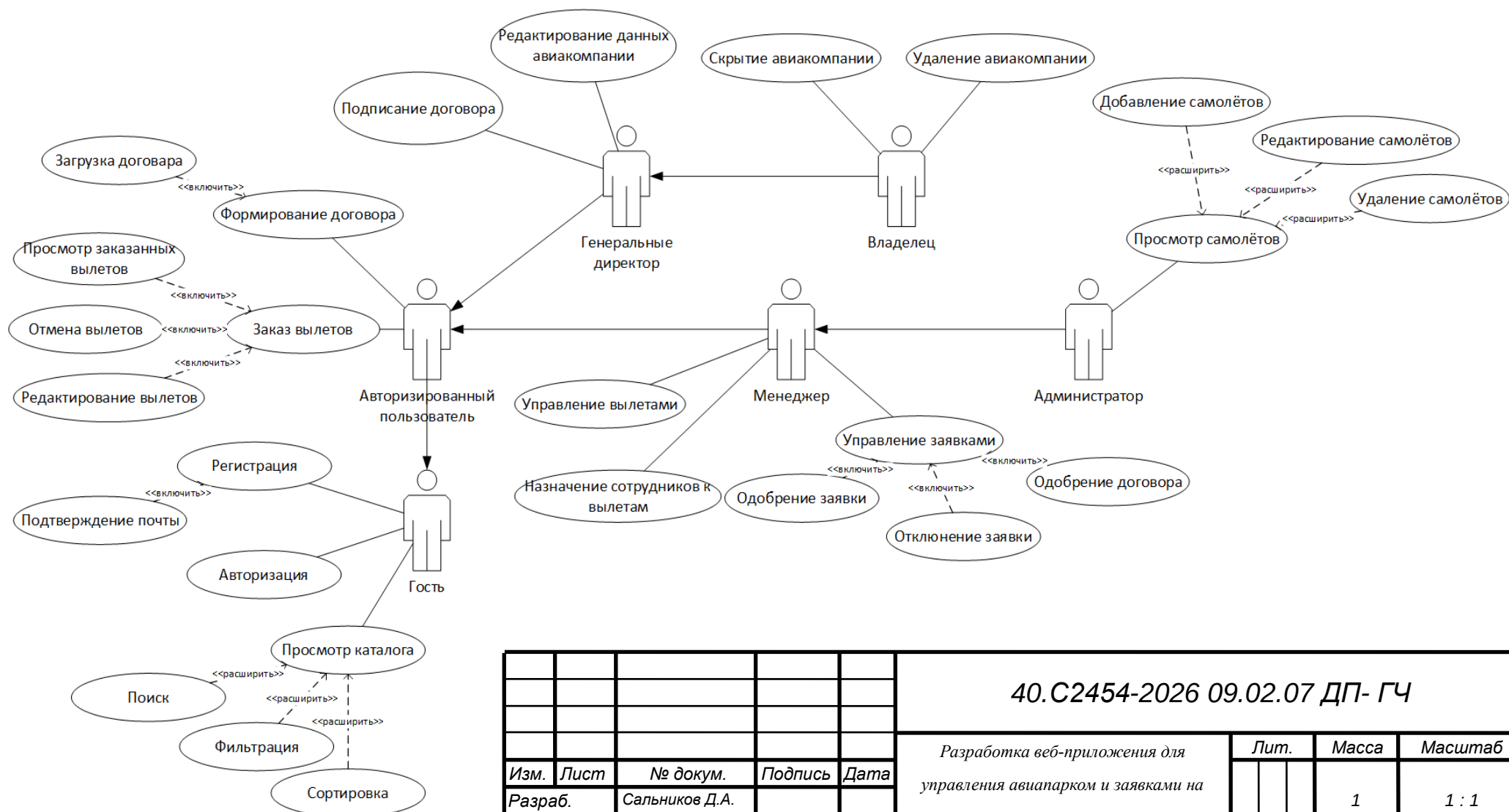
16 IATA Airline and Airport Code Search [Электронный ресурс] // International Air Transport Association : [сайт]. — URL: <https://www.iata.org/en/publications/directories/code-search/> (дата обращения: 09.04.2026). — Текст : электронный.

17 MySQL 8.4 Reference Manual [Электронный ресурс] // Oracle MySQL : [сайт]. — URL: <https://dev.mysql.com/doc/refman/8.4/en/> (дата обращения: 09.04.2026). — Текст : электронный.

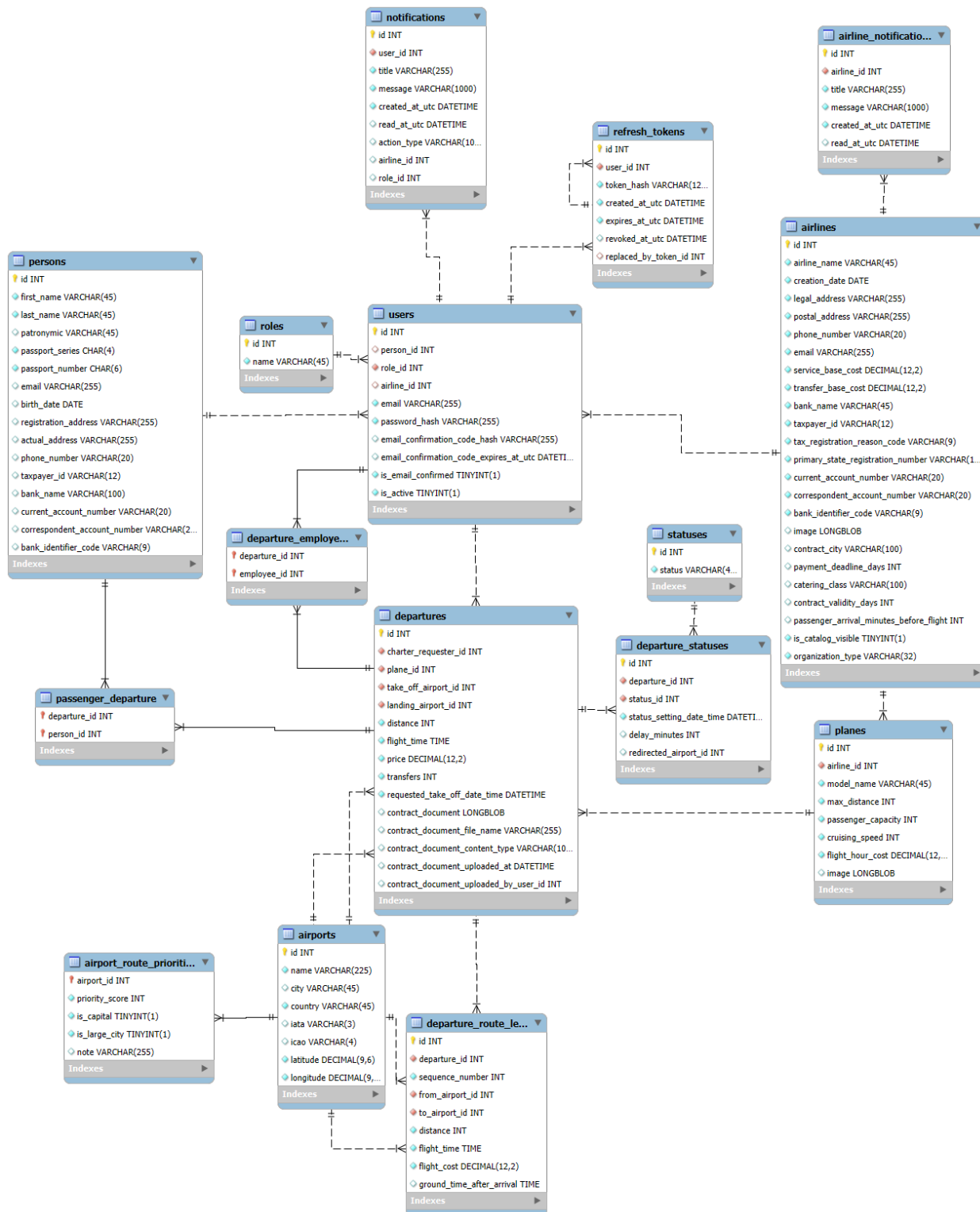
18 OpenAPI Specification v3.1.0 [Электронный ресурс] // OpenAPI Initiative : [сайт]. — URL: <https://spec.openapis.org/oas/v3.1.0.html> (дата обращения: 09.04.2026). — Текст : электронный.

19 React documentation [Электронный ресурс] // React : [сайт]. — URL: <https://react.dev/learn> (дата обращения: 09.04.2026). — Текст : электронный.

20 Vite documentation [Электронный ресурс] // Vite : [сайт]. — URL: <https://vite.dev/guide/> (дата обращения: 09.04.2026). — Текст : электронный.

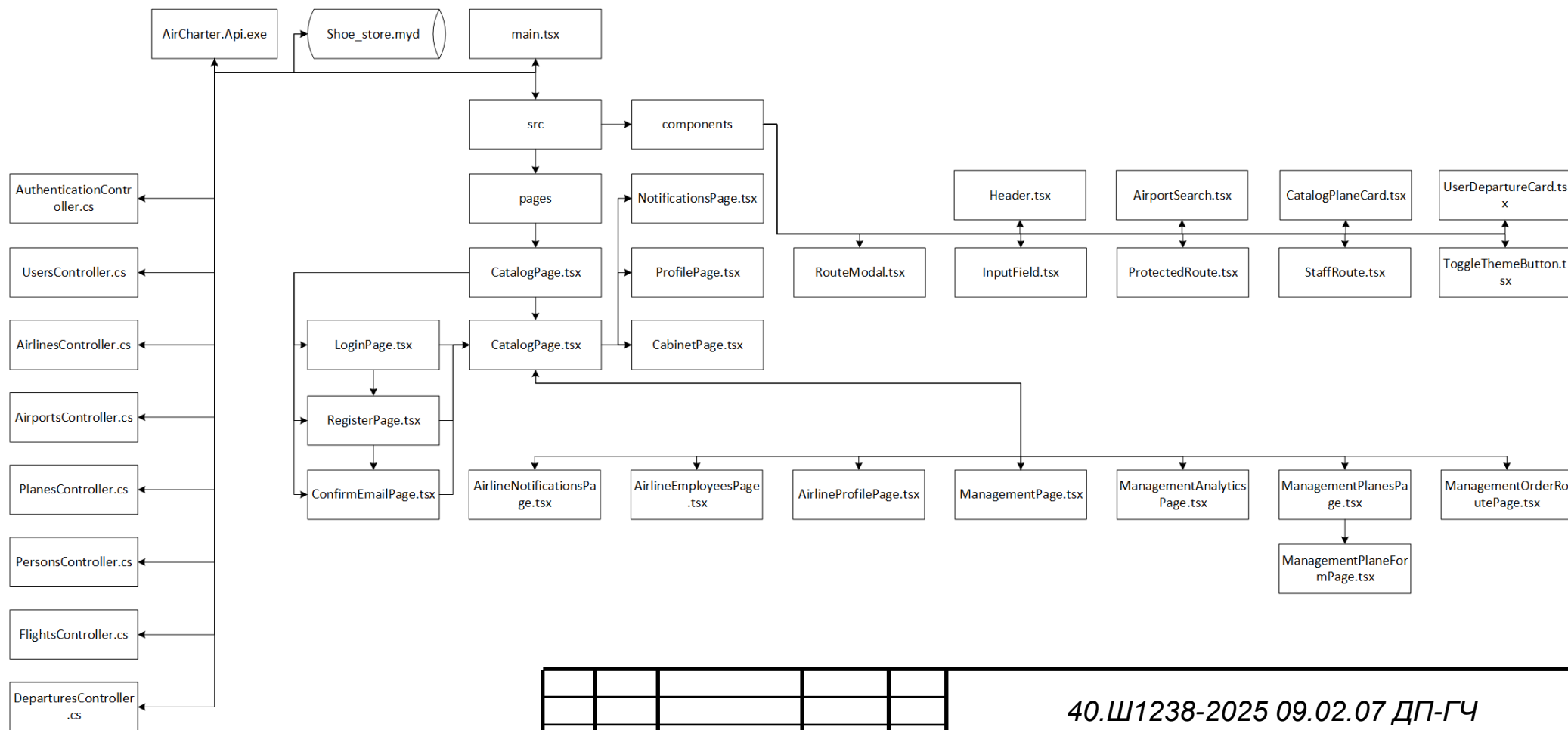


					40.С2454-2026 09.02.07 ДП- ГЧ						
					Разработка веб-приложения для управления авиапарком и заявками на вылет Диаграмма вариантов использования	Лит.			Масса	Масштаб	
Изм.	Лист	№ докум.	Подпись	Дата					1	1 : 1	
Разраб.		Сальников Д.А.									
Провер.		Шагибалова З.Р.									
Т. Контр.											
Реценз.		Будилов В.В.				Лист 1			Листов 3		
Н. Контр.		Каримова Р.Ф.				УКСИВТ 22П-1					
Утверд.		Курмашева З.З.									



40.С2454-2026 09.02.07 ДП-ГЧ

					Лит.			Масса	Масштаб
Изм.	Лист	№ докум.	Подпись	Дата	Разработка веб-приложения для управления авиапарком и заявками на вылет Схема базы данных			1	1 : 1
Разраб.	Сальников Д.А.								
Провер.	Шагибалова З.Р.								
Т. Контр.									
Реценз.	Будилов В.В.								
Н. Контр.	Каримова Р.Ф.				Лист 2			Листов	3
Утверд.	Курмашева З.З.				УКСИВТ 22П-1				



					40.Ш1238-2025 09.02.07 ДП-ГЧ											
					Разработка веб-приложения для управления авиапарком и заявками на вылет Модульная схема приложения											
Изм.	Лист	№ докум.	Подпись	Дата	Лит.			Масса			Масштаб					
Разраб.		Сальников Д.А.							1			1 : 1				
Провер.		Шагибалова З.Р.														
Т. Контр.					Лист			3			Листов			3		
Реценз.		Будилов В.В.			УКСИВТ 22П-1											
Н. Контр.		Каримова Р.Ф.														
Утверд.		Курмашева З.З.														